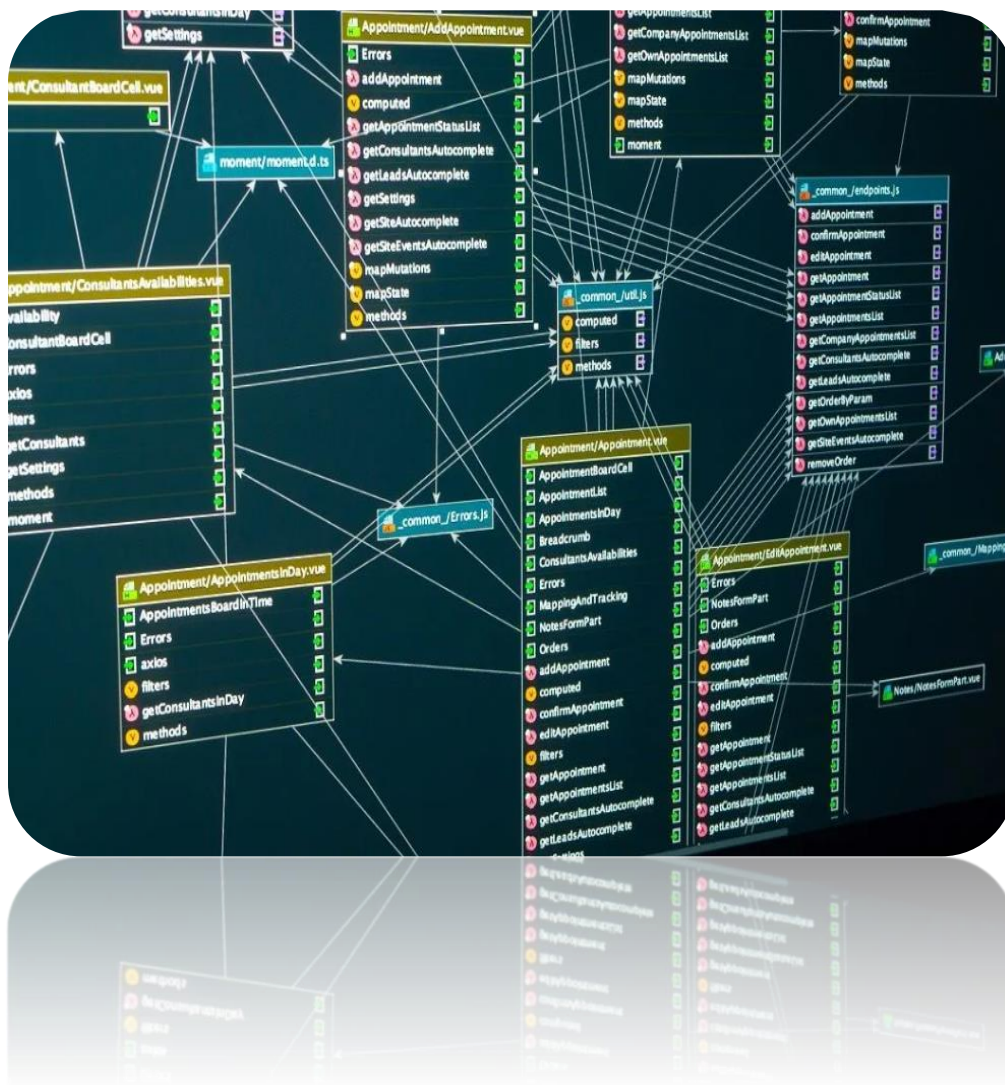


MYSQL КАКО АЛАТКА ЗА РАЗВОЈ И ОДРЖУВАЊЕ НА РЕЛАЦИОНИ БАЗИ НА ПОДАТОЦИ



УНИВЕРЗИТЕТ „ГОЦЕ ДЕЛЧЕВ”- ШТИП

ФАКУЛТЕТ ЗА ИНФОРМАТИКА

Компјутерско инженерство и технологии

СЕМИНАРСКА РАБОТА

MySQL како алатка за развој и одржување на релациони бази на податоци



Ментор,

Проф. Д-р Цвета Мартиновска Банде

Студент,

Мерсие Адемова

број на индекс:102856

Штип, јуни 2025

ПРЕДГОВОР

Оваа семинарска работа претставува сеопфатен преглед на основните и напредни концепти поврзани со базите на податоци, со фокус на практична примена преку MySQL. Семинарската работа започнува со вовед во податоци и нивна организација, како и со класификација на бази на податоци според моделот на податоци. Понатаму, детално се анализира релациониот модел на бази на податоци, објаснувајќи го начинот на организација на податоците преку табели и релации.

Следните поглавја се фокусираат на системите за управување со бази на податоци (DBMS), со посебен акцент на MySQL. Описот на инсталација, конфигурација и користење на MySQL серверот и Workbench GUI алатката се детално разработени. Се обработуваат и SQL наредбите, синтаксата и практични примери за креирање бази, табели, внесување, ажурирање, и пребарување на податоци.

Дополнително, семинарската работа опфаќа и теми како нормализација на бази, перформанси и индексирање, бекап и враќање на податоци, како и практична апликација преку симулација на реален информациски систем. Целта е не само теоретско познавање, туку и развивање на практични вештини за дизајн, имплементација и одржување на бази на податоци.

СОДРЖИНА

предговор	3
СОДРЖИНА	4
1. ВОВЕД ВО БАЗИ НА ПОДАТОЦИ	5
1.1. РЕЛАЦИОНЕН МОДЕЛ	6
1.2. DBMS	8
1.3. SQL	9
1.4. MYSQL	9
2. ИНСТАЛАЦИЈА И ПОКРЕНУВАЊЕ НА MYSQL СЕРВЕРОТ	11
2.1. ИНСТАЛАЦИЈА НА MYSQL	11
2.2. ПОДЕСУВАЊЕ НА СЕРВЕРОТ	12
2.3. ИНСТАЛИРАЊЕ И ПОКРЕНУВАЊЕ НА MYSQL СЕРВИСОТ	14
2.4. MYSQL MONITOR	16
2.5. СИСТЕМСКИ БАЗИ НА ПОДАТОЦИ	18
1. ПРОЕКТИРАЊЕ НА БАЗА НА ПОДАТОЦИ	19
3.1. ФАЗИ ВО ПРОЕКТИРАЊЕТО НА БАЗА НА ПОДАТОЦИ	19
3.1.1. Концептуален дизајн	20
3.1.2. Моделирање на ентитети	21
3.1.3. Моделирање на релации	23
4. MYSQL WORKBENCH	25
4.1. ИНСТАЛАЦИЈА НА MYSQL WORKBENCH	25
4.2. КРЕИРАЊЕ НА КОНЕКЦИЈА	27
4.3. SQL EDITOR	28
4.4. РАБОТА СО МОДЕЛИ	30
4.4.1. Reverse Engineering	30
4.4.2. Forward Engineering	31
5. КРЕИРАЊЕ НА БАЗА НА ПОДАТОЦИ	34
5.1. КРЕИРАЊЕ НА ТАБЕЛИ	35
5.1.1. Типови табели	36
6. ТИПОВИ ПОДАТОЦИ	39
6.1. НУМЕРИЧКИ ТИПОВИ	40
6.2. ТЕКСТУАЛНИ ТИПОВИ - СТРИНГОВИ	42
6.3. ПОДАТОЦИ ЗА УПРАВУВАЊЕ СО ВРЕМЕ И ДАТУМ	45
7. ПРИМАРНИ И НАДВОРЕШНИ КЛУЧЕВИ	47
7.1. ПРИМАРЕН КЛУЧ (PRIMARY KEY)	50
7.2. НАДВОРЕШЕН КЛУЧ (FOREIGN KEY)	53
7.3. РЕФЕРЕНЦИЈАЛЕН ИНТЕГРИТЕТ (FOREIGN KEY CONSTRAINT)	55
8. ВОВЕД ВО SQL	59
8.1. SQL СИНТАКСА	60
8.1.1. Текстуални константи	60
8.1.2. Коментари	61

8.2. НАРЕДБИ	63
9. НАРЕДБИ ЗА ДЕФИНИЦИЈА (DATA DEFINITION LANGUAGE - DDL).....	64
10. ОСНОВНИ НАРЕДБИ ЗА ПРЕБАРУВАЊЕ (SELECT)	69
11. ПОВРЗУВАЊЕ НА ПОДАТОЦИТЕ ОД ПОВЕЌЕ ТАБЕЛИ	74
11.1. СТУДИЈА НА СЛУЧАЈ	75
12. НАРЕДБИ ЗА ИЗМЕНА НА ПОДАТОЦИТЕ (INSERT, UPDATE, DELETE)	79
13. ИНДЕКСИ	86
13.1. МЕХАНИЗАМ НА ИНДЕКСИРАЊЕ	87
13.1.1. Кластериран и некластериран индекс	88
14. ПОГЛЕДИ (VIEWS)	89
14.1. КРЕИРАЊЕ НА ПОГЛЕДИ	90
14.2. АЖУРИРАЊЕ НА ИЗВОРНИТЕ ПОДАТОЦИ	92
14.3. ПРАВА	93
15. СКЛАДИРАНИ РУТИНИ.....	94
15.1. STORED PROCEDURES VS. USER-DEFINED FUNCTIONS	95
15.2. СКЛАДИРАНИ ПРОЦЕДУРИ	96
15.2.1. Креирање на складирани процедури	96
15.2.2. Активирање на процедура	99
15.2.3. Ажурирање на процедура	100
15.2.4. Бришење на процедура.....	101
15.2.5. Параметризација на процедури	101
15.3. КОРИСНИЧКИ ДЕФИНИРАНИ ФУНКЦИИ (UDF)	104
15.4. КОНТРОЛА НА ТЕКОТ	108
6. ВГРАДЕНИ ФУНКЦИИ	113
6.1. СКАЛАРНИ ФУНКЦИИ	113
6.2. АГРЕГАТНИ ФУНКЦИИ.....	113
17. КУРСОРИ И ТРИГЕРИ	114
7.1. BEFORE ТРИГЕРИ	115
7.2. AFTER ТРИГЕРИ.....	117
7.3. КУРСОРИ.....	118
18. ТРАНСАКЦИИ	122
18.1. КОНТРОЛА НА ТРАНСАКЦИИТЕ	122
18.1.1. Контрола autocommit mode	123
18.1.2. START TRANSACTION и ROLLBACK/COMMIT	124
18.2. КОНТРОЛНИ ТОЧКИ	125
18.3. КОНКУРЕНЦИЈА.....	126
18.3.1. Заклучување (Locks).....	127
18.3.2. Трансакции и заклучување.....	129
18.3.3. Timeout и брзина на пристап	132
18.3.4. Изолициони нива (Isolated Levels).....	134
19. КОРИСНИЦИ И ПРАВА НА ПРИСТАП	135

19.1. КРЕИРАЊЕ НА КОРИСНИК	136
19.2. ПРАВА НА ПРИСТАП	137
20. БЕЗБЕДНОСТ ВО MYSQL.....	142
20.1. ПОРТА	142
20.2. КОРИСНИЦИ	143
20.3. АПЛИКАЦИЈА	144
20.4. ПОДАТОЦИ	145
21. КОНЕКЦИИ	147
21.1. КРЕИРАЊЕ НА КОНЕКЦИЈА	147
21.2. ПРЕГЛЕД НА КОНЕКЦИЈАТА.....	150
21.3. ЗАТВАРАЊЕ НА КОНЕКЦИЈАТА	152
21.4. ПОДЕСУВАЊЕ НА КОНЕКЦИЈАТА.....	152
22. РЕПЛИКАЦИЈА	154
22.1. КОРИСТЕЊЕ НА РЕПЛИКАЦИЈА.....	155
22.2. ФУНКЦИОНИРАЊЕ НА РЕПЛИКАЦИЈА	156
22.3. КРЕИРАЊЕ НА РЕПЛИКАЦИЈА.....	157
23. ВАСКУР И МИГРАЦИЈА	162
23.1. КОПИРАЊЕ НА ФАЈЛОВИ.....	163
23.2. DUMP ТАБЕЛИ ВО ТЕКСТ ФАЈЛ.....	164
23.3. DUMP БАЗИ ВО ФАЈЛ	165
23.4. INCREMENTAL BACKUP	167
ЗАКЛУЧОК.....	170
КОРИСТЕНА ЛИТЕРАТУРА	170

1. ВОВЕД ВО БАЗИ НА ПОДАТОЦИ

Податок претставува секвенца од симболи, која со специфична интерпретација добива значење. Дигитални податоци се оние над кои операциите се извршуваат на компјутер и се сместени во некој вид на мемориски единици (магнетни, оптички, механички медиуми) и кои се пренесуваат во форма на сигнали.

Во компјутерските науки поимот **податок** се однесува на сè она што се наоѓа во форма погодна за користење на компјутер. Податоците може да бидат:

- Неструктурирани
 - Слики и музика
 - Содржини од директориуми
 -
- Структурирани
 - Табели

Базата на податоци претставува колекција на податоци организирани на начин погоден за манипулација над податоците. Општо гледано, базите на податоци може да се класифицираат според типот на податоци кои ги содржат: текстуални, нумерички или некои други типови на податоци.

Компјутерските бази на податоци најчесто се класифицираат во зависност од нивниот организационен пристап. Организациониот пристап поинаку се нарекува **модел на базата на податоци**. Моделот на базата на податоци ја одредува нејзината логичка структура, како и начинот на кои податоците се складирали, организирани и начинот на кој се манипулира со нив. Постојат различни модели на бази на податоци:

- Flet база
- Релациона база
- Индексер и инверзен индекс
- Document
- Key value store
- Graph
- ...

1.1. РЕЛАЦИОНЕН МОДЕЛ

Релациониот модел е модел во кој податоците се организирани во форма на подредени листи (ordered list) и се групирани во релации. Подредените листи, всушност се табели со редови и колони, а поврзаноста меѓу табелите се нарекува релација.

За да го појасниме ова, ќе замислиме еден пример. Доколку е потребно во базата на податоци да чуваме листа од корисници, при што за секој корисник ќе се чува име, презиме и држава во која живее, табелата на релационата база ќе изгледа вака:

First name	Last name	Country
Lukas	Peeters	Belgium
Demetrios	Christopoulos	Greece

Да претпоставиме сега, што би се случило доколку во базата сакаме да бележиме и некои дополнителни податоци во врска со државата. На пример, интернет доменот и главниот град. Доколку сакаме да го постигнеме тоа со користење на постоечката табела, би се појавиле неколку проблеми. Првиот е постоење на колони кои не се стриктно поврзани со корисникот, а другиот е повторување на податоците. За секој од корисниците кои би имале иста држава на живеење,

непотребно би се повторувале податоци за називот на државата, главниот град и top-level доменот.

Една од главните цели на користењето на релациони модели на бази на податоци е намалувањето на опишаното повторување (редундантност). Тоа се постигнува со воведување на релации.

Да погледнеме сега како би изгледал примерот. Со оглед на тоа дека за државата сакаме да чуваме повеќе податоци, овие податоци стануваат реален кандидат за креирање на нова табела на податоци. Таа табела би изгледала вака:

Country name	Country domain	Capital
Belgium	.be	Brussels
Greece	.gr	Athens

Јасно може да воочиме дека табелите во релационите бази на податоци, како и сите други табели се составени од колони и редици. Колоните ги обединуваат сите податоци од ист тип, додека редиците чинат еден запис или слог. Може да се каже дека колоните дефинираат кои податоци ќе ги внесуваме во редиците, а редовите претставуваат поединечни податоци на одредена (дефинирана) колона.

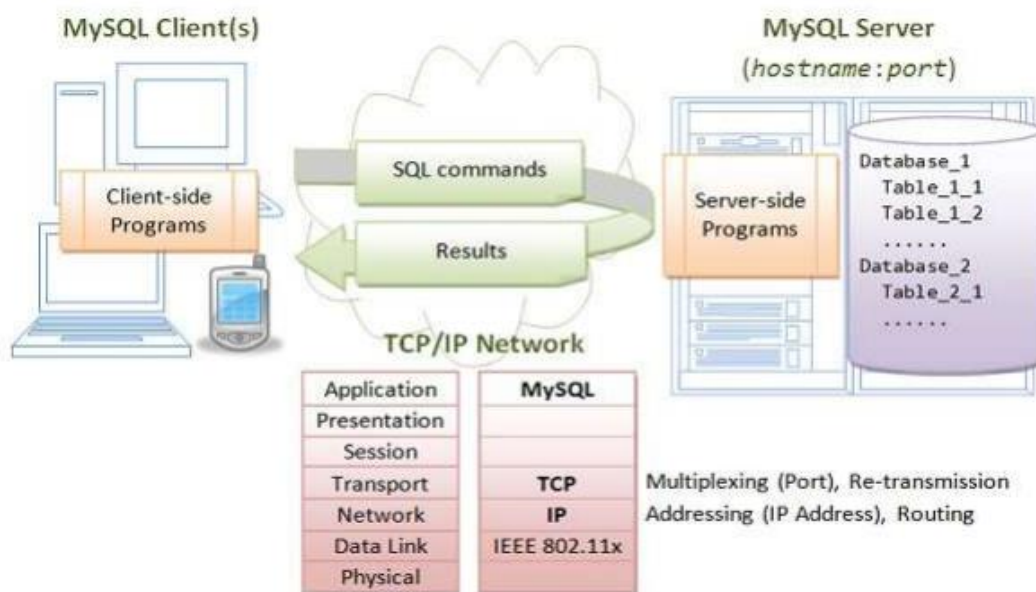
Бидејќи сега имаме две табели, неопходно е да се дефинира релацијата меѓу нив. Дефинирањето на релацијата меѓу две табели се постигнува со употреба на една идентификациона колона. Целта на оваа колона е еднозначно да го дефинира редот кој е внесен во табелата.

Во нашата табела на држави, како идентификациона колона може да се употреби колоната Country name. Она што е особено важно е дека мора да се води сметка за тоа податокот во оваа колона да биде единствен за секој внес, односно за секој ред во табелата. Бидејќи не постојат две држави со ист назив, оваа колона може да се земе за идентификациона колона, а тоа значи дека Country колоната во табелата на корисниците ќе покажува на податок во табелата на државите. Ова значи дека Lukas Peeters живее во државата чии податоци може да го пронајдеме во табелата со државите таму каде што вредноста на колоната Country name Belgium.

First name	Last name	Country	Country name	Country domain	Capital
Lukas	Peeters	Belgium	Belgium	.be	Brussels
Demetrios	Christopoulos	Greece	Greece	.gr	Athens

1.2. DBMS

Систем за управување со база на податоци – database management systems (DBMS) е специфично дизајнирана софтверска апликација која овозможува интеракција меѓу корисниците, апликациите и самата база на податоци. Со други зборови релациониот систем за управување со бази на податоци (Relational Database Management System - RDBMS) овозможува дефинирање, креирање, пребарување, ажурирање и администрација на една база на податоци.



Нај
поз
нат
и
сис
тем
и
за
упр

авување со бази на податоци се: MicrosoftSQLServer, Oracle, MySQL, PostgreSQL, SQLite, MariaDB, SAP HANA, dBASE, FoxPro, IBM DB2, LibreOffice Base, FileMaker Pro, Microsoft Access и др.

1.3. SQL

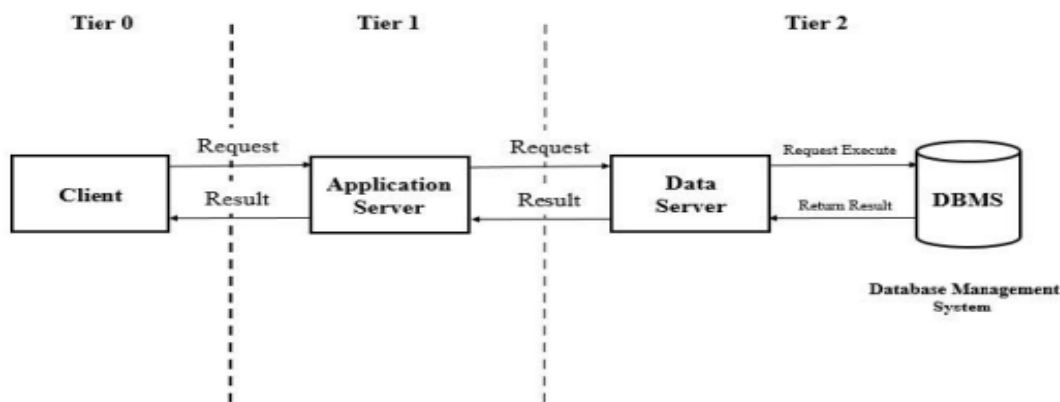
SQL (Structured Query Language) е јазик за управување со податоци во посредство со систем за управување на бази на податоци. SQL е стандардизиран во 1986 година, но и покрај тоа не е во потполност портабилен меѓу различни системи за управување со бази на податоци. SQL поседува комплетна сопствена синтакса со која може да се манипулира со базата на податоци.

1.4. MYSQL

Во овој проект ќе го разгледаме системот за управување со бази на податоци MySQL. MySQL е еден од најкористените системи од овој вид. Тој е проект со отворен код, креиран од страна на шведската компанија MySQL AB, понатаму преземен од Sun Microsystems, а потоа од Oracle.

Овде ќе го разгледаме конзолното задавање на команди, но и алатката со графички кориснички интерфејс MySQL Workbench.

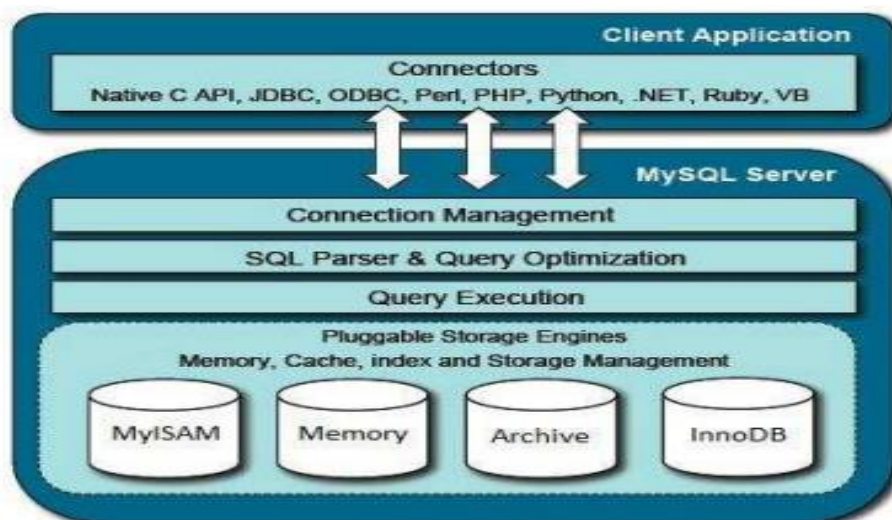
Опкружувањето на MySQL се состои од два дела: сервер, кој управува со податоците и клиент, кој од серверот бара извршување на одредени акции. Серверот всушност претставува еден сервис на Windows оперативниот систем кој е во состојба да прима наредби и да врши складирање на податоците, но кај другите оперативни системи, серверот претставува софтвер кој прима наредби и ги извршува. Од друга страна, клиент е секој софтвер кој му се обраќа на серверот. Клиентот и серверот комуницираат преку request (барање) и response (одговор) во кој се наоѓаат резултатите од барањето.



На сликата е претставен продукционен пример, каде клиентот се обраќа до апликацискиот сервер, кој понатаму комуницира со серверот на податоците, т.е. со database management системот, кој пак ги враќа назад податоците по истата линија на комуникација. Податоците главно се формираат во апликацискиот сервер и му се враќаат на клиентот како информации.

Кога базата на податоци се користи во некоја апликација, била да станува збор за web, desktop или мобилна, тогаш самата апликација е клиент на базата на податоци. За да може креираната апликација да комуницира со MySQL серверот треба да користи т.н. конектори (connectors), кои се разликуваат во зависност од платформата и програмскиот јазик со кој ќе бидат употребувани. Конекторите се всушност библиотеки напишани на изворен јазик за кој се наменети и содржат готови методи за комуникација.

На следната слика е претставен пример за комуникација меѓу клиентска апликација и MySQL серверот преку конектори. Овде може да се забележат повеќе слоеви на MySQL серверот каде првиот слој е задолжен за конекцијата, вториот за парсирање и оптимизација, следниот за егзекуција, и во последниот слој се случува самото извршување, кеширањето, меморијата и сл.



2. ИНСТАЛАЦИЈА И ПОКРЕНУВАЊЕ НА MYSQL СЕРВЕРОТ

Во продолжение ќе ја разгледаме постапката за инсталација на MySQL серверот, како и со постапката за негово покренување. Ќе ја разгледаме и основната клиентска алатка за управување со базата и со податоците, MySQL Monitor.

Од суштинска важност е да не се инсталира MySQL серверот, доколку во нашиот компјутер тој веќе постои, бидејќи тоа може да доведе до различни проблеми во работата, посебно доколку се користи некоја од варијантите на WAMP.

2.1. ИНСТАЛАЦИЈА НА MYSQL

Преземањето на инсталациониот фајл може да се изврши од следната адреса:

<http://dev.mysql.com/downloads/mysql/>

Од менито Select platform е потребно да се избере платформата на која ќе го инсталираме MySQL и во зависност од типот на оперативниот систем (32-bit/64-bit) и саканиот формат за инсталациониот фајл (installer/archive), па download.

За инсталација на MySQL на Windows платформата се избира Go to Download page на Windows (x86, 64-bit), MySQL Installer MSI.

На првиот прозорец се клика No thanks, just start my download, за да избегнеме креирање на кориснички налог на Oracle.

По преземањето започнува инсталацијата на фајлот. Се прифаќаат условите на лиценцата и се избира тастерот Next.

Во следниот прозорец се избира типот на инсталацијата. Соодветниот тип е Developer Default, со што ќе бидат инсталирани сите потребни компоненти за развој. Со избор на овој тип на инсталација ќе бидат инсталирани компонентите:

- MySQL Server;
- MySQL Workbench, интегрирано опкружување за работа со серверот и податоците;

- Конектори, кои служат за користење на MySQL од апликации кои се пишуваат во различни програмски јазици;
- Samples and Examples – тест база на податоци.

2.2. ПОДЕСУВАЊЕ НА СЕРВЕРОТ

По завршувањето на инсталацијата се отвара автоматски дијалог за подесување на серверот. Првиот прозорец (High Availability) се остава на дефолтните подесувања и се клика на тастерот Next. Следниот прозорец се однесува на конфигурацијата на мрежата и типот на конфигурација, при што најсоодветно за тип на конфигурација е да се избере Development Computer.

За интересите на овој проект ќе користиме исклучиво TCP/IP протокол за конектирање. Дефолтната порта на која работи MySQL серверот е 3306. Преку опцијата Open firewall port for network access, предупредуваме за проблемите кои може да се јават доколку firewall оперативниот систем се обиде да ја блокира портата.

Следниот дијалог прозорец се однесува на автентикационите методи, кои исто така ќе ги оставиме на дефолтните подесувања (користење на строга и квалитетна енкрипција на лозинки), па кликаме Next.

Наредниот дијалог прозорец е за креирање на кориснички налог. Иницијално, MySQL серверот доаѓа инсталиран со еден корисник. Станува збор за root корисникот, кој претставува super administrator на серверот. Првично корисникот нема лозинка и овде е потребно да внесеме лозинка. Ова е апсолутна препорака, бидејќи во спротивност ова претставува безбедносен пропуст, особено во продукциска работа.

Освен опцијата за внесување на лозинка за root корисникот, во овој дијалог прозорец постои и опција за креирање на додатни корисници. Корисниците и нивните привилегии понатаму ќе бидат подетално објаснети.

Понатаму се отвара дијалог прозорецот за подесување на Windows сервисот. Како што веќе напоменавме, на Windows оперативните системи MySQL серверот е всушност еден сервис кој функционира во позадина. Прикажаниот прозорец се однесува на подесување на името на овој сервис, како и неговото покренување при подигнувањето на Windows - от.

Со ова е завршено подесувањето на серверот, по што ќе биде отворен тест база на податоци, кој ќе го прескокнеме за подоцна да го објасниме и подесиме рачно.

По завршувањето на инсталацијата неопходните фајлови за функционирање на MySQL ќе се наоѓаат на една од овие две патеки:

C:\Program Files (x86)\MySQL\

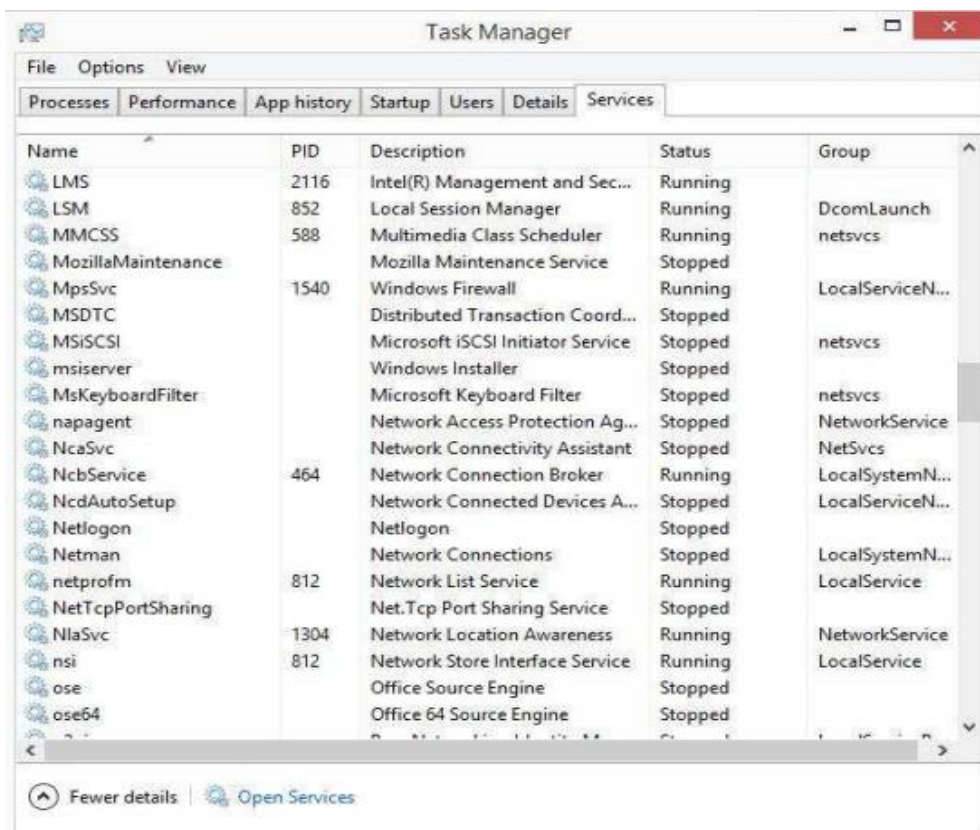
C:\Program Files\MySQL\

2.3. ИНСТАЛИРАЊЕ И ПОКРЕНУВАЊЕ НА MYSQL СЕРВИСОТ

Во најголем број на ситуации, доколку MySQL е инсталиран на корисничкиот налог со административни привилегии, при инсталацијата, ќе дојде до покренување на сервисот. Во некои ситуации до тоа нема да дојде, посебно доколку креираниот налог нема административни привилегии или доколку постои некоја претходна инсталација на MySQL.

Во вакви ситуации, потребно е сервисот рачно да биде стартуван. Ова е операција која бара административни привилегии, така што ова е предуслов за извршување на процедурите кои ќе бидат изложени во продолжение.

За да провериме дали MySQL сервисот е стартиран на нашиот компјутер, може да го искористиме Task Manager, каде може да извршиме увид во листата на сервиси.



Од сликата може да се забележи дека нема инсталиран MySQL сервис.

Постапката за негово додавање е следна:

1. Се отвара Command Prompt со административни права – најлесно ова може да се постигне со отворање на Start meni-то, cmd, па десен клик на Command Prompt со избор run as administrator.
2. Доколку bin фолдерот од MySQL од инсталационата патека ја имаме додадено во системската варијабла PATH, доволно е да внесеме

`mysql.exe -install`

Во спротивно, потребно е да се позиционираме во bin фолдерот на инсталационата патека, која обично е:

`C:\Program Files (x86)\MySQL\MySQL Server x.x\bin`

а потоа да ја активираме истата команда. На местото на x.x се поставува бројот на верзијата на инсталираниот MySQL, кој може да се види во фолдерот Program Files (x86) на дискот C: (доколку не е ставена друга патека при инсталацијата).



Доколку сервисот е успешно инсталиран, ќе добиеме порака Service successfully installed.

Ако сега ја погледнеме листата на инсталирани сервиси во нашиот компјутер, ќе ја добиеме следната слика:



Од сликата исто така може да се забележи дека иако сервисот е инсталиран, тој не е покренат, бидејќи покрај сервисот стои натпис Stopped.

Сервисот може да го стартираме со десен клик на него и избор на опцијата Start.

2.4. MYSQL MONITOR

Крајните корисници никогаш не го гледаат MySQL на начинот на кој ги гледаат и користат другите програми, туку за манипулација со базите и податоците користат програми, специјално наменети за таа цел. Тие програми се нарекуваат клиенти, со оглед на тоа дека MySQL се заснова на клиент-сервер архитектура, каде клиентите се конектираат на серверот за да извршат операции над базата и податоците во неа. Постојат голем број на клиентски апликации, од кои некои

имаат и графички кориснички интерфејс, но основниот програм за управување со базата на податоци (клиент) е конзолната апликација MySQL Monitor.

Тоа е mysql.exe апликација, која се наоѓа во bin инсталациониот фолдер.

Уште еден важен поим, кога станува збор за управувањето со MySQL сервисот се корисниците. Еден MySQL сервер може да има дефинирано произволен број на корисници. Секој MySQL сервер доаѓа со автоматски креиран еден корисник со назив root и овој корисник се смета за системски администратор. За да се логираме на MySQL серверот како root корисник, потребно е во конзолата да внесеме:

```
mysql.exe -u root -p
```

По ова ќе ни биде побарана лозинка која ја имаме дефинирано при инсталацијата на серверот. Доколку при инсталацијата не е дефинирана лозинка за root корисникот, доволно е да ја внесеме следната наредба:

```
mysql.exe -u root
```

Ваквото стартирање е можно под услов bin фолдерот во кој се наоѓа mysql.exe фајлот да го имаме додадено во системската варијабла PATH. Во спротивно, преку конзолата се отвара фолдерот bin и потоа се внесува горенаведената команда.

Ако ги тестираме сега функционалностите на инсталираниот сервер:

За добивање на моменталниот статус се пишува командата status;

За добивање на моменталниот датум и време се пишува командата select now();

За добивање на сите бази на податоци кои постојат во серверот се пишува командата show databases;

2.5. СИСТЕМСКИ БАЗИ НА ПОДАТОЦИ

Може да забележиме дека иако немаме креирано ниту една база на податоци, на серверот постојат три бази на податоци:

- `information_schema`;
- `mysql`;
- `performance_schema`;
- `sys`.

`Information_schema` овозможува пристап до информациите за податоците, т.е. метаподатоци. Кога самостојно ќе креираме некоја база на податоци и во склоп на некоја нејзина табела, во `information_schema` базата ќе може да пронајдеме информации како што се назив на базата, тип на податоците во колоните и права на пристап. Всушност, со наредбата `show databases;` , информациите кои ги добиваме за моменталните бази на серверот се добиени од базата `information_schema`. Со други зборови, `information_schema` е информациона база на податоци која постои на серверот. Во неа постојат неколку табели од кои може само да се чита, но не и да се пишува.

MySQL е база која ги содржи сите серверски информации, како корисници, релации, привилегии и сл.

`Performance_schema` претставува функционалност која овозможува следење на извршувањето на серверот на ниско ниво. Оваа функционалност е имплементирана преку `performance_schema` базата на податоци. Логиката на оваа функционалност се заснова на следење и бележење на серверските настани. Настан на серверот претставува се она што серверот го извршува и потребно е одредено време за извршување. Целта на оваа функционалност е можноста за добивање на информации за ефикасноста на серверот без загрозување на неговите перформанси.

Системската база `Sys` претставува сет на објекти кој на програмерите им помага при интерпретацијата на податоците соберени од страна на `Performance_schema`

базата, со цел подобрување на работата на серверот (фино подесување на серверот).

1. ПРОЕКТИРАЊЕ НА БАЗА НА ПОДАТОЦИ

При имплементацијата на нова база на податоци лесно може да паднеме во замка, со цел побрзо да ја сработиме работата, без претходно да потрошиме време на процесот на дизајн, односно проектирање. Колку е значајна оваа фаза, најдобро може да се види по завршувањето на проектот, кога поради лошо дизајнирана база на податоци често е неопходно да се извршуваат скапи редизајни и реимплементации. Проектирањето на база на податоци е слично на проектирањето на куќа. Нереално е да се започне со зидање без претходно да биде изработен детален план. Исто така, добриот дизајн овозможува оригиналната градба покасно да биде проширена, без потреба за рушење на претходно изграденото.

3.1. ФАЗИ ВО ПРОЕКТИРАЊЕТО НА БАЗА НА ПОДАТОЦИ

Проектирањето на база на податоци се изведува во неколку фази:

- **Анализа на барањата**

Пред да разгледаме како нешто да направиме, потребно е прво да согледаме што е потребно да направиме. Тоа најдобро може да го изведеме со собирање на што е можно повеќе информации и нивна анализа.

- **Концептуален дизајн**

Откако ги имаме соберено и анализирано барањата, потребно е овие барања да ги преточиме во некој вид на формален, концептуален дизајн. За таа цел најчесто се користат разни дијаграми за моделирање на ентитети и релации меѓу нив.

- **Логички дизајн**

На крај, со логичкиот дизајн се врши мапирање на концептуалниот дизајн на конкретни табели и релации во избраниот систем за бази на податоци.

3.1.1. КОНЦЕПТУАЛЕН ДИЗАЈН

Креирањето на концептуален модел подразбира дефинирање на ентитети и релации меѓу нив. Во релациониот модел наречен **модел на ентитети и зависности**, односно **Entity Relationship Model**, лесно може да воочиме два клучни поими: **ентитети** и **зависности**. Овие поими ја чинат самата срж на креирањето на концептуален модел, како и на базата на податоци воопшто.

Базата на податоци чува информации од одредени објекти, кои поинаку се нарекуваат ентитети, а односите меѓу овие објекти се релации. На пример, базата на податоци на еден универзитет може да чува информации за студентите и предметите, како и информации за уписите на студентите на одредени курсеви. Студентите и курсевите би биле ентитети, додека уписите би биле релација меѓу тие два ентитети. Слично на тоа, една база на податоци за продажба би можела да содржи информации за производите, клиентите и продажбите. Производите и клиентите би биле ентитети, додека продажбите би биле однос, т.е. релации меѓу клиентите и производите.

Популарен пристап на концептуалниот дизајн е користење на моделирани релации, односно креирање на таканаречен ER модел.

3.1.2. МОДЕЛИРАЊЕ НА ЕНТИТЕТИ

За визуелизација на дизајнот при моделирање на релациите и ентитетите се цртаат дијаграми на ентитети и зависности (Entity Relationship diagram).

Customer

Product

Типично, базата на податоци се користи за претставување на одредени карактеристики, односно атрибути на веќе споменатите ентитети. Во базата на податоци за продажбите, за секој клиент се чуваат информации како што се име, презиме, e-mail адреса и датум на раѓање. Можно е овие информации да бидат и значајно многу повеќе. Накратко, може да се каже дека атрибутите ги опишуваат ентитетите на кои им припаѓаат.

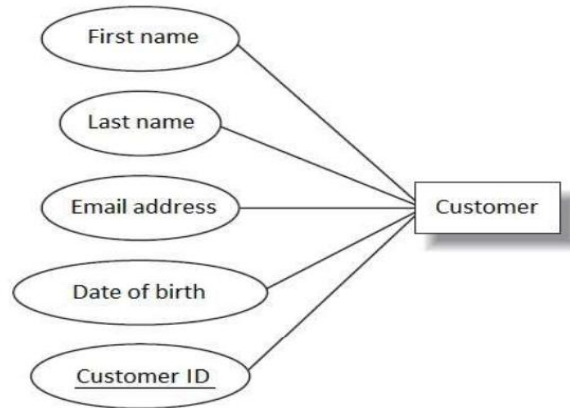
Атрибутите се користат и за разликување на повеќе ентитети од ист тип. За разликување на клиентите може да се користи нивното име, но доколку два клиенти имаат исто име, атрибутот име не може да го користиме како идентификатор на ентитетот.

Да разгледаме кои други атрибути ни се достапни. Ако го разгледаме атрибутот e-mail адреса, ќе забележиме дека таа е единствена и на прв поглед се наметнува како кандидат за идентификација на ентитетот. Проблемот настанува во тоа што секој клиент би морал да има e-mail адреса, поинаку не би било можно да се чуваат информации за сите клиенти во базата. Исто така, проблем би се појавил и кај клиентите со повеќе e-mail адреси, па информациите од овој тип не би можеле да служат како уникатен идентификатор

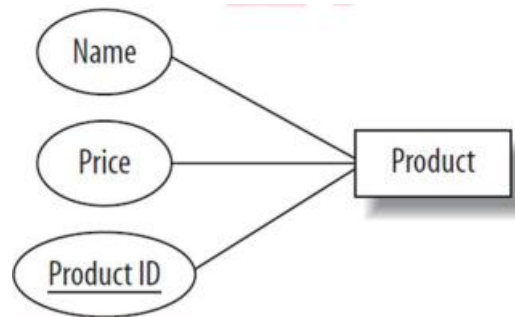
Недостатокот на адекватен атрибут за идентификација се решава со воведување на атрибут чија специфична намена е идентификација на ентитетот. Во некои случаи може да се земат и повеќе атрибути, т.е. да се креира сложен атрибут за идентификација.

Идентификациониот атрибут (или атрибути) се нарекува примарен клуч.

Во ER дијаграмот, атрибутите се претставуваат во овални облици со називот на атрибутот, кои се поврзани со соодветниот ентитет. Атрибутите кои претставуваат примарни клучеви се прикажани со подвлечен назив.



Да го разгледаме сега ентитетот Product. По барање кое го имаме добиено од клиент, потребно е базата на податоци која ја моделираме да чува информации за името и цената на производот. Па, ентитетот Product ќе поседува атрибути Name и ProductID, како и атрибут неопходен за идентификација на ентитетот:

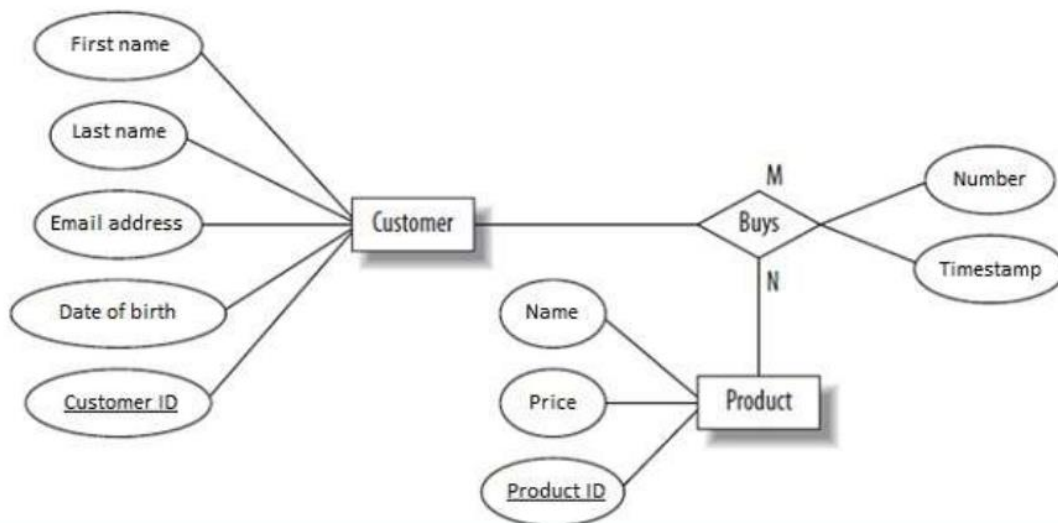


3.1.3. МОДЕЛИРАЊЕ НА РЕЛАЦИИ

Ентитетите партиципираат во однос на други ентитети при што поседуваат одредени зависности, допирни точки, односно релации. На пример, клиентот може да купи производ или студентот може да посетува одреден курс, музичарот може да сними албум, писателот може да напише книга. Сето тоа се примери за односи меѓу ентитетите.

Во нашиот случај, како релација меѓу клиентот и производот, ќе дефинираме продажба, со оглед дека клиентот може да купи производ. Исто како и ентитетите,

и релациите може да поседуваат атрибути. Така, релацијата продажба може да поседува атрибути како што се време на продажба и број на продажби:



Релациите меѓу ентитетите се прикажуваат со помош на ромб. Карактерите M и N покрај ромбот ја означуваат релацијата. Овие карактери ја дефинираат таканаречената **кардиналност на релациите**.

Различен број на ентитети може да се појават на различни страни од релацијата. На пример, секој клиент може да купи било колку производи, и секој производ може да биде купен од страна на било колку клиенти. Тоа е т.н. релација many-to-many (повеќе спрема повеќе).

Освен овој вид на релација, постојат и други два вида на релации, како one-to-many (еден спрема повеќе) или one-to-one (еден спрема еден). Една личност може да има повеќе кредитни картички, а секоја од тие кредитни картички припаѓа само на една личност. Тоа е класичен пример за one-to-many (еден спрема повеќе) релација.

На крај, серискиот број на моторот на автомобилот е пример за релација one-to-one (еден спрема еден). Моторот на автомобилот може да има еден сериски број и серискиот број на автомобилот може да припаѓа само на еден мотор.

За означување на овие релации се користи следната нотација:

- **1:1 (one-to-one),**
- **1:N (one-to-many),**
- **M:N (many-to-many).**

4. MYSQL WORKBENCH

Во продолжение ќе разгледаме интегрирано опкружување за администрација на MySQL серверот, MySQL Workbench, кое поседува графички кориснички интерфејс, за разлика од MySQL Monitor.

MySQL Workbench им овозможува на администраторите и програмерите интегрирано опкружување со сите потребни алатки за:

- Дизајн и моделирање на базата на податоци;
- SQL развој;
- Администрирање;
- Миграција на бази.

4.1. ИНСТАЛАЦИЈА НА MYSQL WORKBENCH

MySQL Workbench може да се инсталира во склоп со пакетот, заедно со серверот и останатите компоненти, но може да се инсталира и одвоено. Пожелно е да се инсталира одвоено во случај при користење на WAMP, LAMP, MAMP или XAMPP пакет, затоа што не мора да се инсталира серверот, туку доволно е само да се инсталира MySQL Workbench.

Во продолжение ќе го разгледаме процесот за преземање и инсталација на оваа апликација, самостојно, т.е. одвоено од целокупниот пакет.

MySQL Workbench може да се преземе од адресата:

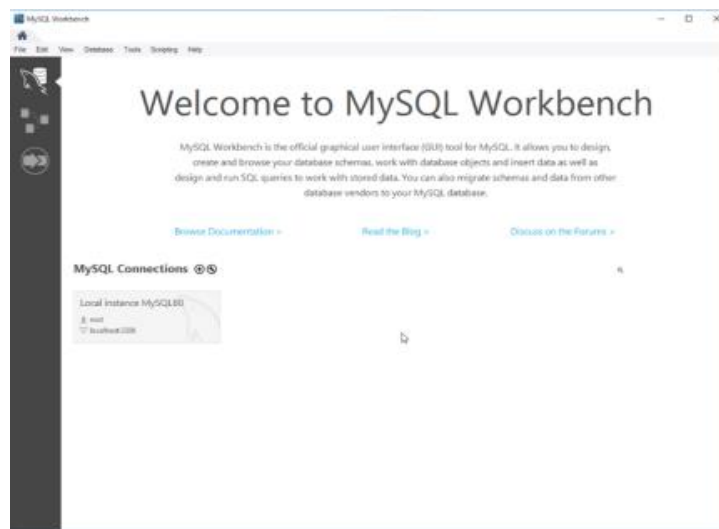
<http://dev.mysql.com/downloads/workbench/>

По отворањето на наведениот линк, ќе добиеме опции за избор на платформата и со притискање на тастерот Download го вршиме преземањето, по што ќе ни се понуди опција за најава, која ќе ја избегнеме и ќе преминеме директно на преземањето.

По преземањето на апликацијата ќе ја покренеме инсталацијата со следните чекори:

1. Next >
2. Next >
3. Complete: Next >
4. Install

По покренувањето на MySQL Workbench ќе се отвори прозорец:



Од сликата може да забележиме дека MySQL Workbench автоматски има креирано конекција на MySQL серверот. Сепак, тоа не е секогаш случај и најчесто конекцијата ќе ја креираме сами.

4.2. КРЕИРАЊЕ НА КОНЕКЦИЈА

Под конекција се подразбира пријавување (најава) на MySQL серверот со адекватни пристапни податоци. Во случајот со MySQL Monitor, конекцијата ја извршувавме со командата:

```
mysql.exe -u root -p
```

и ова значеше дека на серверот се имаме конектирано како root корисник.

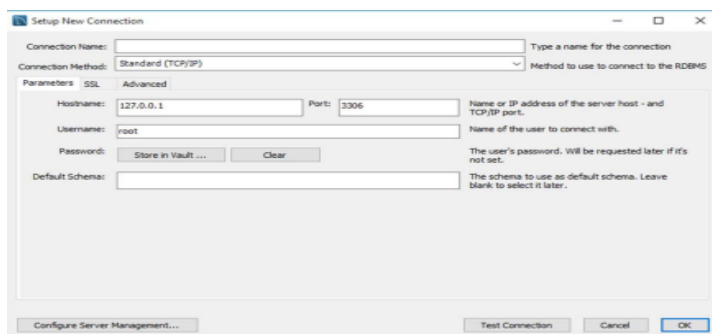
Сега, за да креираме нова конекција, потребно е да го избереме тастерот за додавање на нова конекција (plus taster).

Во новоотворениот прозорец потребно е да ги внесеме најважните параметри за да успееме да ја креираме конекцијата:

- Назив на конекцијата;
- hostname;
- port;
- username;
- password.

За назив на конекцијата може да се избере било што, додека за hostname потребно е да се внесе адресата од компјутерот на која се наоѓа инстанца од MySQL серверот која е оперативна. Со оглед на тоа дека работиме со локална база на податоци, потребно е да ја внесеме localhost адресата 127.0.0.1 или едноставно да напишеме localhost.

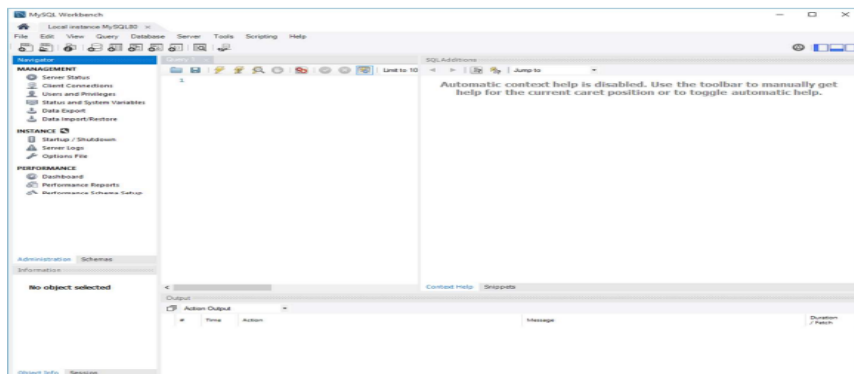
Подразбирливата порта на која работи MySQL серверот е port 3306 или 3307. Корисничкото име (username) е root, додека лозинката иницијално е празна.



Од сликата може да воочиме дека има и поле за внес на подразбирливата база на податоци од креираната конекција – Default Schema. Во ова поле може да го внесеме називот на некоја база на податоци, која ќе биде автоматски избрана при конектирање на дефинираната конекција. Оваа опција во позадина ја повикува use наредбата.

4.3. SQL EDITOR

При креирањето на конекција се отвара SQL Editor:



Во левиот дел од работната површина може да се најдат опции кои се логички поделени во четири категории:

- Management;
- Instance;
- Performance;
- Schemas.

Во одделот **Management** може да се добијат основните информации за статусот на серверот, конектираните клиенти и корисниците. Исто така, во овој оддел се наоѓаат и опциите за креирање на нови корисници и додавање на права на корисниците. Тука припаѓаат и уште две многу важни опции, а тоа се увозот и извозот на податоци. Со користење на овие опции може да се увезат податоци од друга компатибилна база на податоци или да се извезат податоци од дефинираната база.

Одделот **Instance** содржи опции за подесување на инстанците на MySQL серверот.

Следниот оддел, **Performance** се однесува на функционалностите за следење на перформансите на различни операции кои се извршуваат на серверот.

За нас најважен оддел е одделот **Schemas**, кој ги прикажува базите на податоци кои постојат на серверот.

Централниот дел од работната површина од SQL Editor-от е за внес на упити, т.е. SQL код за управување со базите на податоци.

Под делот за внес на упити се наоѓа **Output panel** во кој се прикажуваат пораки кои се однесуваат на успешноста на командите кои ги задаваме на серверот. Доколку дојде до некоја грешка при извршување на некој упит, текстот за грешката ќе биде впишан во тој панел.

Во крајниот десен дел од работната површина се наоѓа **SQL Additions**, панел кој служи за приказ на некој вид на помош при пишувањето на упитите. Тука може да се најде документација за начинот на кој се пишуваат некои основни SQL упити и сл.

4.4. РАБОТА СО МОДЕЛИ

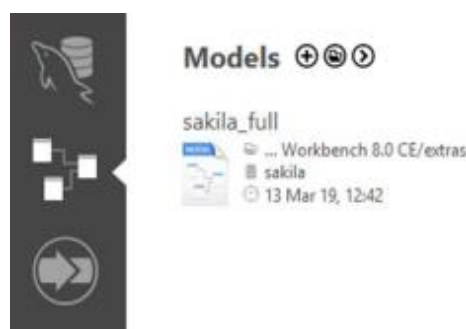
Со користење на MySQL Workbench алатките за моделирање може да се креираат и ER модели на бази на податоци кои потоа може автоматски да се конвертираат во жива база на податоци. Ваквиот пристап, каде од моделите се добива

конкретен код, се нарекува **Forward Engineering**. Можно е и обратното сценарио, т.е. креирање на модел од постоечка база на податоци и таквиот процес се нарекува **Reverse Engineering**.

4.4.1. REVERSE ENGINEERING

Со оглед дека сеуште немаме креирано ниту една база на податоци, не сме во состојба да извршиме Reverse Engineering(бидејќи ни е потребна база која постои за од неа да добиеме модел).

Одделот за манипулација со моделите иницијално изгледа како на сликата:



Се забележуваат три тастера. Станува збор за тастери за манипулација со моделите, и тоа: за креирање на нови модели, за отворање на постоечки модели и за генерирање на модели од постоечки бази или скрипти.

4.4.2. FORWARD ENGINEERING

Почеста пракса е прво да се креира моделот, а потоа од него да се направи базата на податоци. Тоа го овозможува Forward Engineering. Приоритет имаат анализите на барањата и функционалниот дизајн, на чија основа може да го креираме логичкиот дизајн. Препорачливо е да се придржуваме до овој редослед .

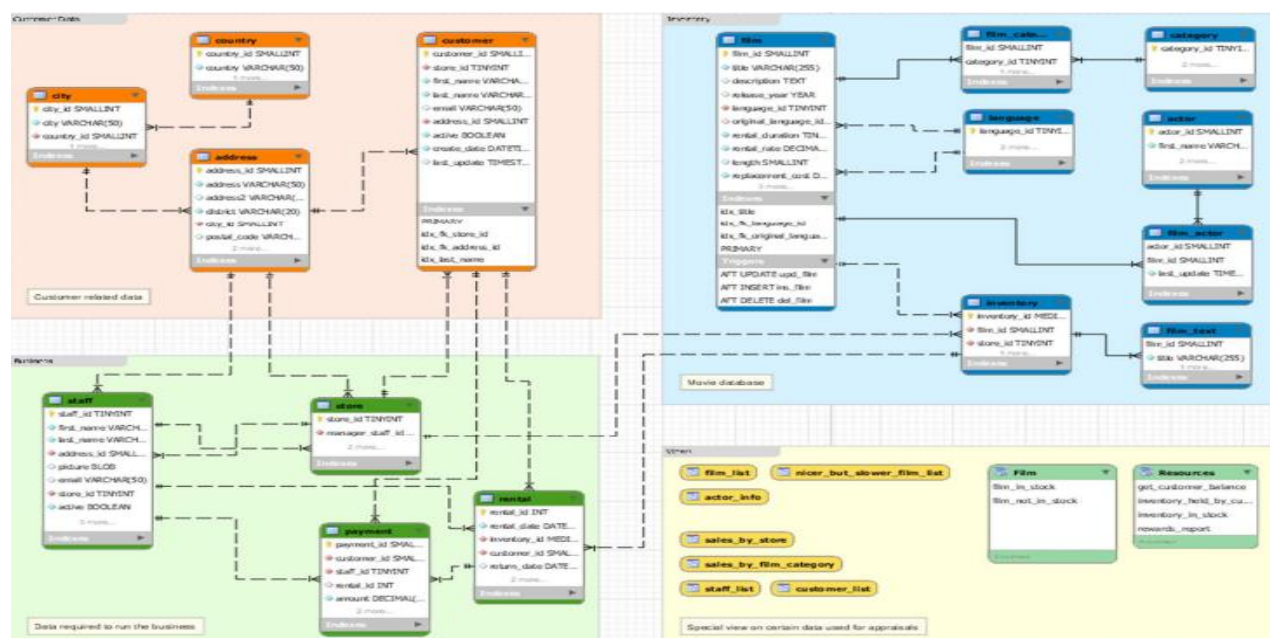
За да го тестираме Forward Engineering, може да ја преземеме и инсталираме базата на податоци која е наменета за учење и тестирање и која ќе ни послужи во понатамошната работа.

Станува збор за базата **Sakila** и може да ја преземеме (модел, структура и податоци) од следната адреса: <https://dev.mysql.com/doc/index-other.html>.

По преземањето на .zip фајлот и неговото распакување, би требало да добиеме три датотеки:

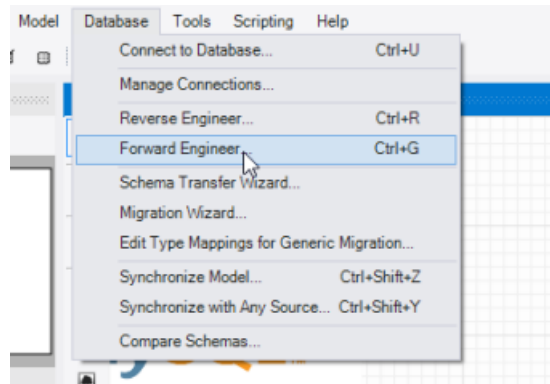


Во Workbench се избира File од менито, а потоа ставката Open Model. Потоа се клика распакуваниот .zip фајл и се клика фајлот sakila.mwb, па Open. По целата оваа процедура треба да добиеме ER дијаграми од комплетната база на податоци:

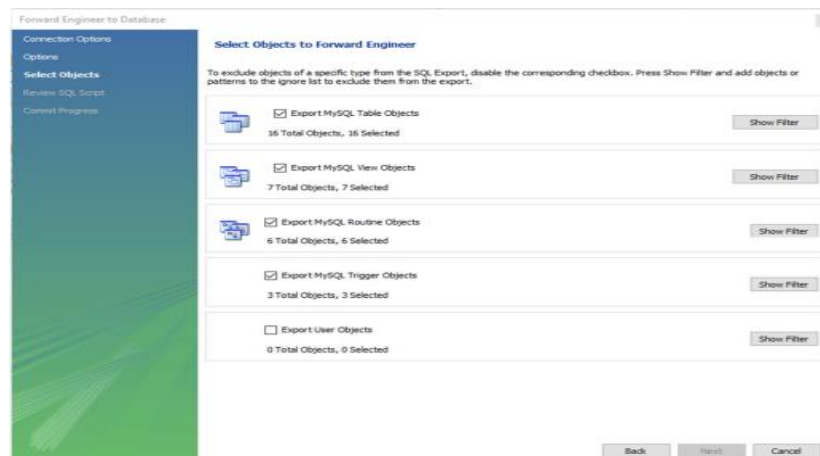


Оваа база ќе ја користиме за демонстрирање на упитите, затоа ни е од голема важност нејзиното правилно креирање.

За да добиеме конкретна база на податоци од овој модел, ќе ја покренеме опцијата Forward Engineering:



Во новоотворениот прозорец, првенствено одредуваме преку кои конекции (кој сервер) ќе ја креираме базата на податоци од дијаграмот, односно каде таа на крајот ќе се најде. Овој прозорец е идентичен со оној што го добиваме при креирањето на нова конекција. Откако ќе кликнеме Next во другиот прозорец може да избереме некоја од опциите за базата која се креира, кои во овој случај ќе ги оставиме на дефолтните вредности. По следното Next во новиот прозорец, од значајност ни е dialog Select objects, каде може да избереме кои се елементи од дијаграмот сакаме да го претвориме во конкретни делови од базата на податоци.

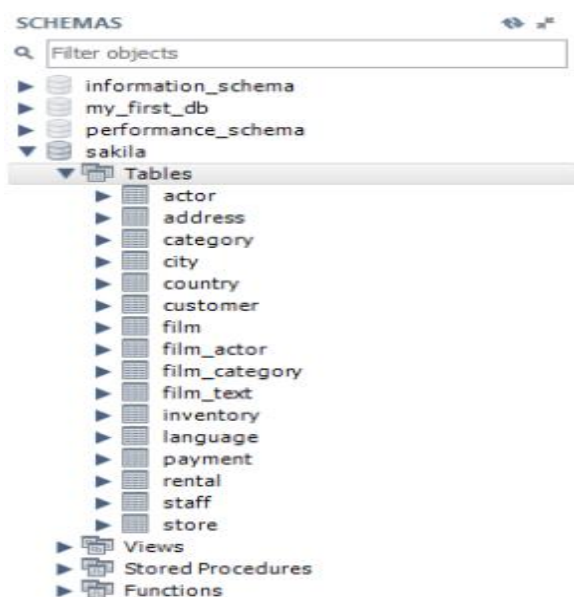


Се избираат практично сите елементи, односно табели, погледи, рутини, тригери.

Во следниот дијалог ќе се прикаже генерирана скрипта која ќе ја создаде базата на податоци со сите пропратни елементи. Со процесот Forward Engineering дијаграмите од моделот се преведуваат во адекватен SQL код, кој по извршувањето на серверот, создава база со табели, погледи, тригери и сл.

Со избор на тастерот Next, генерирањето на базата започнува.

По успешниот Forward Engineering, во одделот за базите може да се најде Sakila базата на податоци.



Вака креираната база не содржи никакви податоци. За да ги внесеме податоците, потребно е од .zip фолдерот да го пронајдеме фајлот sakila-data.sql.

Фајловите кои имаат екстензија .sql, се документи кои содржат комплетни SQL упити или т.н. SQL скрипти. Документот sakila-schema.sql содржи упити за креирање на база на податоци со комплетна структура, т.е. база каква што ние моментално имаме (база без податоци). Документот sakila-data.sql содржи податоци кои се наоѓаат во табели, а тоа е она што сега ни е потребно.

За извршување на упитот кој е дефиниран во документот потребно е да ја избереме опцијата Open a script file in this editor, која се наоѓа над просторот за внес на упити на SQL едиторот. Тоа е првата икона во низата икони, прикажани на сликата:



По изборот на оваа опција, потребно е да се пронајде фајлот со дефинирани упити, а во нашиот случај тоа е фајлот sakila-data.sql.

5. КРЕИРАЊЕ НА БАЗА НА ПОДАТОЦИ

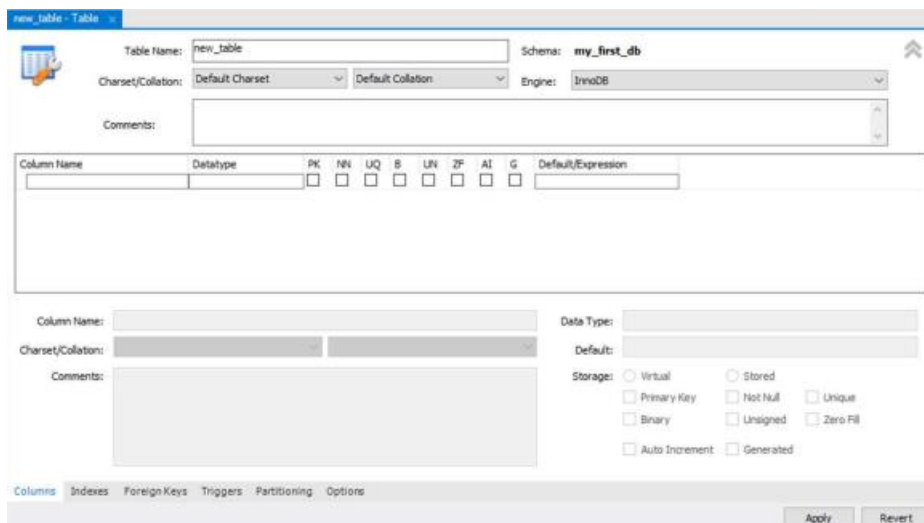
Наједноставен начин за креирање на база на податоци е со користење на MySQL Workbench. За креирање на база на овој начин не потребно дури ни познавање на SQL упити.

Во десниот дел од прозорецот во одделот SCHEMAS, каде веќе постојат системски бази на податоци, со десен клик во празниот простор се избира опцијата CREATE SCHEMA. На работната површина ќе се појави прозорец во кој е потребно да се внесе називот на базата на податоци и да се избере комбинација Charset/Collation. Понатаму, со притискање на тастерот Apply ќе се појави прозорец со прикажан DDL упит за креирање на база.

Во следниот прозорец ќе добиеме известување за успешноста на креирањето на базата и притискаме Finish, по што ќе имаме креирано база на податоци и таа ќе биде прикажана во одделот SCHEMAS.

5.1. КРЕИРАЊЕ НА ТАБЕЛИ

За креирање на табела во некоја база на податоци, потребно е од контекстуалното мени кое се добива со десен клик над базата во која сакаме да ја креираме табелата, да ја избереме опцијата Create Table. Во прозорецот кој се отвара потребно е да внесеме основни податоци за табелата:



Во прозорецот за креирање на нови табели, меѓу достапните опции може да се види и опција со назив Engine. Ова е опција за избор на storage engine, т.е. типот на табелата.

5.1.1. ТИПОВИ ТАБЕЛИ

Постојат многу типови кои може да и се доделат на една табела, во што може да се увериме од контекстуалното мени на оваа ставка. Двата најважни типови за нас ќе бидат MyISAM и InnoDB.

MyISAM типот е стабилен, зрел и лесен за користење. Претставува механизам кој податоците ги складира на едноставен принцип на бинарно дрво. Ваквиот тип на табели податоците физички ги сместува во три различни видови на документи:

- frm фајловите се описни фајлови на табелите; во нив се наоѓаат информации за колоните на табелите, индексите на табелите, типот на механизмот на складирање и сл.;
- MYD фајловите ги содржат податоците од табелите;
- MYI фајловите ги содржат индексите од табелите.

Овие фајлови треба да стојат во ист фолдер со db.opt фајлот на самата база на податоци.

InnoDB претставува модерна алтернатива на претходно опишаниот тип на табела. Основната мерна единица во базата на податоци со ваков тип на табели, претставува страна. Сите податоци се складираат на страна со величина до 16 KB. Одредена количина од такви страни (64) сочинува еден опсег на страна (extend), додека одредена количина од опсези чинат еден сегмент. Комплетниот простор на базата, всушност е збир од сегменти.

Физички, податоците од InnoDB табелите се складираат во посебен фајл под назив `ibdata1`, кој се наоѓа надвор од фолдерот на базата (во родителскиот фолдер). InnoDB исто така подразбира уште два фајла: `ib_logfile0` и `ib_logfile1`, во кои се складираат логовите.

За секоја табела во базата на податоци може независно да се избере нејзиниот тип. MyISAM се препорачува да се користат кога нема потреба од трансакции и странски клучеви, а важни ни се брзината и зафатеноста на меморијата. Генерално, MyISAM се добри за мали бази по количина и захтевност.

Од друга страна, InnoDB, ги користиме кога се потребни трансакции, поголема сигурност или кога на табелата ќе и се пристапува од страна на повеќе различни корисници кои би вршеле измени практично симултано.

Трансакциите и странските клучеви се клучни за одлуката за механизмот на складирање. Но, MySQL има можност за измена на механизмот за складирање по креирањето на носечките податоци, па дури и по самото складирање. Исто така, може да се користат и повеќе механизми во една база.

Во следната табела се прикажани комплетните разлики меѓу двата видови на табели:

Feature	MyISAM	InnoDB
Transaction Support	No	Yes
Recovery From Crash	Complete rebuild tables/indexes	Recover from logs
Performance with Data Growth	Slow down dramatically	Performance remain almost unchanged
Log (journal) Flush	Run on the top of the journaled file system	Run after each transaction
Foreign Key Support	No	Yes
Locking Granularity	Full table	Row
Full Text Search Index	Yes	Yes
Cluster Indexes	No	Yes
Data Caches	No	Yes
Set as default engine	Version prior to version 5.5	Version 5.5 and later
Storage of Table	One table stored in 3 separate files .FRM for table format .MYD for data .MYI for indexes	Table stored in table space: consisting several files (or raw disk partitions)
Type of cache	Filesystem block cache (virtual memory space)	Within the engine (main memory)
How records are stored?	In roughly sorted order	Ordered by primary key or first unique key
Could A36 Compressed Table Be Updated?	No	Yes

По ова, може да продолжиме со креирање на табела. Следниот чекор е дефинирање на колоните. Секоја колона од една табела е дефинирана со нејзиното име и тип на податок.

Column Name:
Collation:
Comments:

Data Type:
Default:
☒ Primary ☒ Not Null ☐ Unique
☐ Binary ☐ Unsign ☐ Zero Fill
☐ Auto Increment

Вежба 1: Потребно е да се направи база на податоци со назив test_db. Базата мора да има подразбирлива колација и сет на карактери за јапонски јазик.

Решение:

```
create database test_db character set = ujis collate = ujis_japanese_ci
```

Вежба 2: Потребно е да се направи база на податоци со назив test_db2 со подразбирлив грчки сет на карактери и колација. Скриптата мора да поседува механизам кој нема да доведе до грешка во случај на креирање на нова база, доколку таа веќе постои.

Решение:

6. ТИПОВИ ПОДАТОЦИ

Една табела се состои од повеќе колони, а секоја колона мора да има дефинирано тип на податок кој ќе го чува. За да знаеме на адекватен начин да ги креираме колоните, ќе ги разгледаме достапните типови на податоци во MySQL.

Податоците се сместуваат во ќелии, а податоците во една колона мора да бидат од ист тип:

Cardinal number	Name	Date of birth
1	John	01-01-1980
2	David	01-01-1970

Очигледно е дека овие три колони чуваат различни типови на податоци. Во првата колона (Cardinal number) се складираат само броеви; во втората колона (Name) текст; а во третата (Date of birth) датуми.

Ова е и генералната поделба на типови во MySQL: броеви (нумерички типови), текст (стрингови) и датуми (datetime). Но, овие типови имаат множества подтипови и всушност на управувањето со овие подтипови се базира добрата табеларна конструкција во MySQL и базите на податоци воопшто.

Доколку не очекуваме повеќе записи во листата, тогаш доволни се неколку броеви, но ако очекуваме милиони записи, тогаш потребни се повеќе броеви и, со самото тоа, нумеричкиот тип на колоната мора да биде поголем.

Да претпоставиме дека типот на колоната е таков што може да прифати милиони записи, а во табелата да имаме само два записи. Во тој случај, MySQL би имал резервиран простор во меморијата за нешто што никогаш нема да го исполни тој простор, или пак обратното, табелата да има можност за прифаќање на два записи, а се појавуваат милион; табелата, едноставно нема да биде во можност да ги прифати тие податоци.

Истото важи и за текстуалните податоци (стрингови). Во една колона може да очекуваме одредена количина на карактери по поле. Содржината на полето може да биде, на пример, да или не, што е во било кој случај – два карактера. Но, може да биде и поголем текст од повеќе илјади карактери. За двата случаи потребни се различни големини на полето, односно колоната.

MySQL серверот не одговорен за типовите податоци со кои ќе ракува, туку за тоа се задолжени механизмите за складирање.

6.1. НУМЕРИЧКИ ТИПОВИ

Нумеричките типови се типови кои прифаќаат исклучиво бројни вредности, без оглед на тоа дали станува збор за цели или децимални броеви.

Најпознат нумерички тип е `int` (integer). Тоа е целоброен тип кој може да прифати вредност од -2147483648 до 2147483647 доколку е означен како `signed`, или од 0 до 4294967295 ако е означен како `unsigned`. Означените броеви се оние кои дозволуваат негативни вредности, додека неозначените прифаќаат само позитивни вредности. Овој тип има четири бајта, што значи дека неговата максимална величина така што ќе ги изброиме бајтите и ќе го помножиме со бројот на битови (еден бајт содржи осум бита), и целиот тој број ќе го ставиме како експонент на бројот два.

На пример, `int` прифаќа 4 бајти. $4 \text{ бајти} \times 8 \text{ бита} = 32 \text{ бита} \Rightarrow 2^{32} = 4294967295$.

Освен основниот тип `int`, постојат и варијации на целобројните нумерички типови:

Тип	Капацитет	Опсег
<code>Tinyint</code>	1 бајт	Од -128 до 127 (неозначено до 256)
<code>Smallint</code>	2 бајти	Од -32768 до 32767 (неозначено до 65535)
<code>Mediumint</code>	3 бајти	Од -8388608 до 8388607 (неозначено до 16777215)
<code>Int</code>	4 бајти	Од -2147483648 до 2147483647 (неозначено до 4294967295)
<code>bigint</code>	8 бајти	Од -9223372036854775808 до 9223372036854775807 (неозначено до 18446744073709551615)

При иницијализирањето на овие типови во табелата, може да им се додадат и некои карактеристики кои се однесуваат на нивното прикажување. Може, на пример, да се дефинираат минимум прикажани цифри: `int (2)` -ќе прикажува минимум две цифри (иако во меморијата ќе биде складиран цел број). Ова е посебно важно кога се користи `zerofill`, кој додава нули за дополнување на бројот до саканиот број на цифри - `zerofill (ZF)`.

На пример, ако типот е `int (5)`, а внесената вредност е 1, конечниот број ќе биде 00001.

`Unsigned (UN)` – оваа опција ја поставуваме доколку сакаме бројот да биде неозначен и во тој случај бројот не може да прими негативна вредност.

Освен целите, постојат и типови со подвижна запирка, од кои разликуваме неколку типови:

- `float`;
- `double`;
- `decimal`.

float / double е основната верзија на бројот со подвижна запирка, која подразбира еднострана прецизност. Оваа прецизност обезбедува четири бајти за вредноста: 2,3; 100.45.

float, исто така прифаќа параметри, при што може да се форматира форматот на децималниот запис. На пример, ако како тип ставиме: `float (6,2)`, вредноста 1525 ќе биде емитувана како 1525.00. На ист начин функционира и типот `double`, со тоа што има величина од 8 бајти и двојно поголема прецизност.

decimal управува со броеви со максимална прецизност и мора да прифаќа параметри како: `decimal(5,2)`. Овој број ќе прифати вредност до 5 цифри, кој може да има до 2 децимали. Значи, ако го внесеме бројот 123.45 ќе биде прифатен. Бројот 123 ќе биде запишан во базата со додадени децимали – 123.00, а додека бројот 123.119 ќе биде зачуван како 123.12. Броевите со

повеќе од 5 цифри нема да бидат прифатени и серверот ќе пријави грешка. Максималниот број на цифри во децималниот тип е 65, при што од десната страна на запирката може да бидат не повеќе од 30 цифри (decimal (65,30)).

Типот **bit** служи за складирање на еднобитни вредности 1 или 0. Овој тип не е вистински еднобитен тип, туку само го репрезентира податокот. Овој тип во базата е складиран како tinyint(1), а од верзија 5.0.3 MySQL е посебен бинарен тип. Типот bit служи исклучиво за складирање на логички вредност (true или false, 1 или 0, вклучено или исклучено), па така може да прифаќа и бројна вредност (1/0) или Boolean вредност (true / false).

6.2. ТЕКСТУАЛНИ ТИПОВИ - СТРИНГОВИ

MySQL познава пет типови на текстуални типови:

- char;
- binary;
- blob;
- enum;
- set.

Од сите овие типови најчесто користени се char и blob.

char / varchar типот на податок прифаќа низа карактери (бајти). При креирањето прифаќа параметар кој означува колку бајти прифаќа колоната, односно полето: char(5) –прифаќа 5 карактери и зафаќа исто толку бајти, односно двојно, доколку станува збор за UNICODE карактери.

Заклучно со верзија 4.0 капацитетот на полето се изразува во бајти, така што за чување на текстуални UNICODE вредности од 20 карактери било потребно да се наведе должината од 40 бајта. Од верзија 5 капацитетот на полето се изразува во карактери, па за чување на UNICODE текстуални вредности од 20 карактери

доволно е да се наведе 20. Тоа значи дека, кога креираме ваков тип и во него ја сместиме вредноста 'x', тој таа вредност во базата ќе ја зачува како x, но во меморијата ќе заземе простор од четири карактери: `varchar (5)` – вака дефиниран тип, исто така прифаќа до пет бајти / карактери, но доколку има помалку од пет карактери, во меморијата ќе биде зафатен простор онолку колку што изнесува вредноста: 'a' = 1 bajt; 'abcde' = 5 bajt.

Поради можноста да не зафаќе повеќе меморија од што му е потребно, `varchar` е еден од најкористените текстуални типови во базите на податоци.

Типот **binary / varbinary** функционира исто како `char` и `varchar`, но наместо како низа карактери, тој стринговите ги бележи како бинарни вредности, поради што не постои контрола над кодниот распоред и сортирањето: `varbinary (5)`.

Типовите **blob (binary large object) / text** може да се сфатат како некој вид на надградба на типовите `char` и `binary`. Тие се користат за ракување со многу големи вредности. Основната разлика меѓу `blob/text` и `binary/char` е во тоа што `blob` и `text` не ги сместуваат вредностите во табелата во која се наоѓаат, туку во посебни складишта. Поради тоа, наместо 65 535 бајти, колку што може да прифатат `char` и `binary` типовите, овој тип може да прифати многу поголеми вредности.

`blob` и `text`, исто така имаат подтипови:

- `tinytext/tinyblob` – 255 карактери/бајти;
- `text/blob` – 65 535 карактери/бајти;
- `medimtext/mediumblob` – 16 777 215 карактери/бајти;
- `longtext/longblob` – 4 294 967 295 карактери/бајти.

Разликата меѓу `blob` и `text` типовите е тоа што `blob` се користи за сместување на бинарни податоци (на пример, фотографии), додека `text` се користи за сместување на текст.

Не треба да заборавиме дека овие големини не важат за UNICODE карактери. Во тој случај, овие количини се намалуваат двојно, бидејќи еден UNICODE карактер е два бајта.

enum е тип кој прифаќа само некоја од вредностите претходно дефинирани во дефиницијата на самиот тип:

```
enum('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday')
```

Ако колоната е вака дефинирана, вредноста во полето може да прифати само една од наведените вредности.

Ова всушност е низа од вредности и секоја од нив е индексирана со одреден број, па вредностите од низата може да се репрезентираат и со броеви. Но, и покрај тоа, повторувањето на вредностите во низата не е дозволено. Еден enum тип може да прифати 65 535 вредности.

Типот **set** функционира слично како типот enum, со таа разлика што една колона може да содржи повеќе вредности.

На пример, ако дефиницијата на колоната е: `set('a', 'b')` , тогаш вредноста на таа колона може да биде 'a' или 'b' или 'a', 'b'; но, доколку вредноста ја поставиме на 'a', 'b', 'b', 'b' или слична во која доаѓа до повторување на вредностите, вредноста и понатаму ќе биде 'a', 'b'.

6.3. ПОДАТОЦИ ЗА УПРАВУВАЊЕ СО ВРЕМЕ И ДАТУМ

Во MySQL се разликуваат пет типови за управување со времето и датумот:

- date;
- datetime;
- time;
- timestamp;

- `year`.

Типот **date** ги подразбира сите датуми од 01 јануари 1000 година до 31 декември 9999 година. Кога еднаш ќе ја дефинираме колоната, вредноста може да ја внесеме преку два различни типа: датум (на пример, со функцијата `curdate()`) или стринг (2009-10-02).

Типот **datetime** функционира слично како `date`, со тоа што може да прифати и време за одреден датум. Исто така, може да прифати вредност преку еквивалентен тип (на пример, функцијата `now()`) или стрингот ('GGGG-MM-DD SS:MM:ss').

Поради тоа управувањето со датумот и времето со `datetime` типот на податок е често во употреба. Покрај тоа, додавањето на датум и време во базата на податоци преку програмските јазици, користејќи го `datetime` е многу едноставно, бидејќи повеќето програмски јазици поседуваат можност за добивање на датумот и времето во идентичен облик.

Типот **timestamp** претставува број на секунди поминати од 01 јануари 1970 година. Овој тип често се користи за мерење на времето за некоја операција.

Типот **time** прифаќа време од -838:59:59 до 838:59:59 по шемата SS:MM:ss. Како и останатите формати на датум и овој тип може да прифати вредност спакувана во стринг.

Типот **year** претставува година (во зависност од параметрите при дефинирањето) во периодот од 1901 до 2155. Вредноста може да биде внесена во вид на стринг или датум.

Вежба 3: Потребно е да се креира табела со имињата и презимињата на корисниците, матични броеви, број на телефон, шифра и e-mail.

Пред креирање на табелата, потребно е во спецификацијата точно да се нагласи кои типови на податоци ќе ги содржи табелата.

Решение:

Табелата ќе содржи поле ID, кое во зависност од фреквенцијата на корисници ќе биде `smallint`, `medium int` или `int`.

За полето име и презиме може да се користи типот `varchar(50)`, бидејќи ни името ни презимето нема да ја преминат таа граница. За ова поле никако не треба да се користи типот `text`. Типот `varchar(50)` може да се искористи и за e-mail адресата и телефонскиот број, додека за матичен број ќе искористиме тип `varchar(13)`. Конечниот сет на податоци би бил:

```
id int
name varchar(50)
surname varchar(50)
email varchar(50)
password varchar(50)
id_number varchar(13)
```

Вежба 4: Потребно е да се креира табела `avtomobil`. Оваа табела ќе содржи податоци за кубикажата на автомобилот, коњските сили, називот, максималната брзина и годината на производство. Пред креирањето на табелата, потребно е да се одреди кои типови на податоци ќе бидат користени.

Решение:

Кубикажата не може да биде поголема од неколку илјади, па `smallint` е доволен тип, бројот на коњски сили и максималната брзина се уште помали, па повторно ќе користиме `smallint`, бидејќи `tinyint` е помал од што ни е потребно. За годината на производство можеме да искористиме тип `year`, `date`, `timestamp`, па дури и `smallint`.

```
id int
volume smallint
horse_power smallint
year_of_production year
```

Вежба 5: Потребно е да се креира табела за продавница на книги. За секоја книга во табелата потребно е да се чуваат називот, цената, авторот, краток опис, ISBN бројот и датумот на издавање.

Решение:

Називот на книгата може да биде `varchar(256)`, под претпоставка да не постои назив подолг од 256 карактери. Цената може да биде тип `decimal`, со шест цифри од кои две ќе бидат децимални (под претпоставка дека ниту една книга нема да чини повеќе од 9999). За краток опис може да се искористи и долг `varchar(4000)` или `text`. За датумот на издавање ќе искористиме `timestamp`.

```
id int
name varchar(256)
```

price decimal(6,2)
description varchar(4000)
date Timestamp

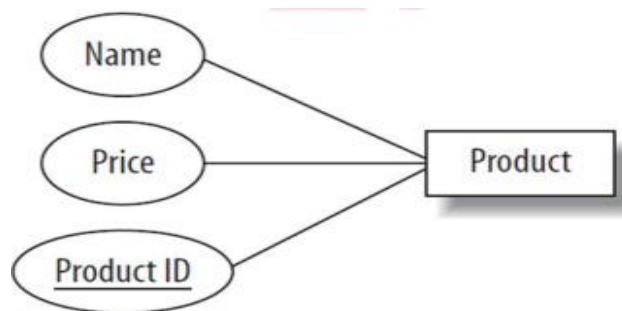
7. ПРИМАРНИ И НАДВОРЕШНИ КЛУЧЕВИ

Ограничувањата се посебни ознаки во табелите со кои се контролира содржината на нејзините колони, односно редови:

- Primary key
- Foreign key
- Unique
- Default
- Not null
- Unsigned
- ...

Клучевите се системи кои овозможуваат обезбедување на референцијален интегритет во базата на податоци. Клучевите се делат на примарни и надворешни. Примарните клучеви може да подразбираат повеќе од една колона (композитни/сложени клучеви).

За да продолжиме со креирањето на табелата на адекватен начин, потребно е да се запознаеме уште со некои поими. Да се потсетиме како е потребно да изгледа табелата која веќе претходно започнавме да ја креираме:



Од дијаграмот гледаме дека е потребно да креираме три колони:

Column Name	Datatype
product_id	INT
name	VARCHAR(45)
price	DECIMAL(10,4)

Колоните ги креираме така што ги опишуваме нивните називи, во колоната Column Name. потоа ги дефинираме нивните типови. Со оглед дека колоната product_id е идентификационата колона во која ќе се чуваат целобројни вредности, за тип на податок за оваа колона ќе избереме тип INT.

Првата причина за ова е очигледна, а тоа е пребројување на слоговите (редовите) во самата база. Другата причина е дефинирањето на примарен клуч или да ја споиме табелата со друга табела со помош на странски клуч – што ќе го разгледаме во продолжение.

Во колоната со назив Name ќе се чуваат називите на производите, па за тип на оваа колона ќе поставиме VARCHAR. Поставена е и големина на типот VARCHAR на 45, под претпоставка дека називот на производот нема да надмине должина од 45 карактери.

За колоната со назив Price поставен е тип на податок Decimal. Decimal е тип на податок кој е најпогоден за чување на валутна репрезентација и прифаќа параметри. Затоа во заграда се додадени параметри кои укажуваат на тоа дека во таа колона од табелата ќе може да се сместуваат децимални вредности од шест цифри од левата страна и четири цифри од десната страна на подвижната запирка.

Кога дефинираме табела со ваква структура, таа ќе биде валидна, но со прилично слаба употребливост. Нејзиниот основен проблем е тоа што ќе можат да постојат два или повеќе производи со исто име, цена и идентификационен број – што би значело непотребно повторување на податоците, а тоа секогаш и во секој случај сакаме да го избегнеме во базите на податоци.

Значи, на оваа табела и е потребен некој механизам кој ќе забрани внес на дупликации на идентификациони броеви. Во оваа табела имаме креирано колона со назив `product_id`, која служи токму за идентификација. Оваа колона ќе служи и како ограничувачки фактор, не дозволувајќи повторување на исти внесови. Ваквата колона се нарекува **примарен клуч**.

7.1. ПРИМАРЕН КЛУЧ (PRIMARY KEY)

Примарниот клуч е основен ограничувачки фактор во една табела. Тоа е својство на една колона кое спречува вредностите во неа да се повторуваат. На овој начин, лесно може да се идентификува било кој ред од колоната, бидејќи во секој од нив се наоѓа уникатен примарен клуч. Примарниот клуч е карактеристика на скоро секоја табела, иако постои можност да се избегне неговото поставување.

Понекогаш табелата може да има и неколку колони кои заедно го сочинуваат примарниот клуч. На пример, доколку би сакале ниту еден производ и неговата цена да не се повторуваат, би можеле да дефинираме примарен клуч на колоната `price` и `name`. Но, за нашиот случај подобра опција е дефинирање примарен клуч само на една колона.

Од сето досега изложено може да заклучиме дека најадекватен избор за примарен клуч во нашиот случај е колоната `product_id`. Тоа може да го постигнеме со чекирање на опцијата PK во истиот ред во кој ја имаме дефинирано колоната `product_id`:

Column Name	Datatype	PK	NN	UQ	BIN	UN	ZF	AI
 product_id	INT	☒	☒	☐	☐	☐	☐	☐

Data Type:

Default:

Storage: ☐ Virtual ☐ Stored

☒ Primary Key ☒ Not Null ☐ Unique

☐ Binary ☐ Unsigned ☐ Zero Fill

☐ Auto Increment ☐ Generated

Ја чекиравме ставката Primary Key, што оваа колона ја дефинира како колона за примарен клуч. Потполно е логично дека една ваква колона автоматски ќе го добие својството Not Null. На сликата може да видиме дека освен опцијата за прогласување на одредена колона да биде примарен клуч, постојат и мноштво додатни опции кои може да се применат на една колона.

Поставувањето вредност во **Default** полето претставува дефинирање на дефолтна вредност доколку не биде наведена никаква вредност за ова поле.

Not Null опцијата дефинира дека ќе биде неопходно ваквата колона при внес да податоците да добие вредност. Тоа значи дека, на пример, ако се обидеме да внесеме нов производ во табелата без дефинирање на примарниот клуч, ќе дојде до грешка. MySQL ќе пријави грешка, што значи дека на овој начин се осигуруваме дека полето ќе добие вредност.

Колоната која е примарен клуч автоматски добива својство Not Null, што е потполно логично, со оглед на тоа дека оваа колона ќе служи за идентификација на податоците. Дури и ако се обидеме да го исклучиме ова својство за оваа колона, ќе бидеме приморани во неа да внесуваме вредност секој пат, доколку е подесена како примарен клуч.

Со сетување на опцијата **Unique** се постигнува сличен ефект како и со својството Primary Key. Во оваа колона нема да може да се внесуваат податоци со исти вредности. За разлика од примарниот клуч, колоната која има само Unique својство ќе може да прифаќа и Null вредности. Примарниот клуч е автоматски и Unique и Not Null.

Активацијата на опцијата **Binary** ја претвора колоната во бинарен тип. Идентично како употребата на varbinary тип.

Опцијата **Unsigned** може да се чекира на колоните за кои како тип на податок е избран некој од нумеричките типови. Со оваа опција се овозможува внес само на позитивни нумерички вредности.

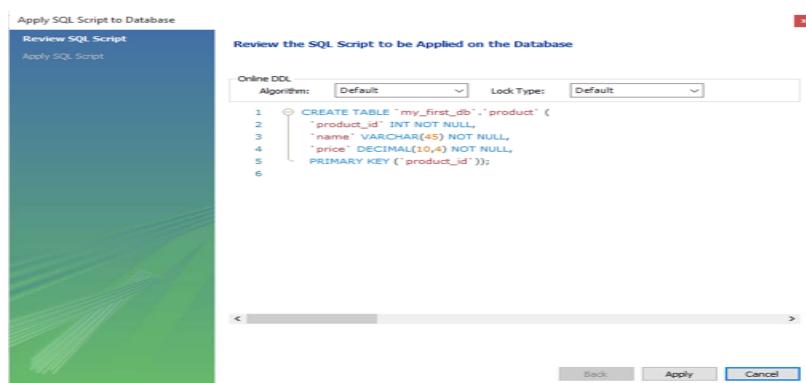
Активацијата на **Zero Fill** опцијата над некоја колона влијае на приказот на податоците. Исто како и во претходниот случај, и ова својство може да се примени само на нумерички типови податоци.

Својството Auto Increment автоматски внесува вредности во колоната со примарен клуч, при што секој пат ги инкрементира за еден.

Ако сумираме, изгледот на креираната табела product со сите потребни податоци би бил:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G
 product_id	INT	☒	☒	☐	☐	☐	☐	☐	☐
 name	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐
 price	DECIMAL(10,4)	☐	☒	☐	☐	☐	☐	☐	☐
		☐	☐	☐	☐	☐	☐	☐	☐

По ова со клик на копчето Apply, ќе се прикаже прозор во кој може да го видиме генерираниот упит на основа на овие подесување. Ќе напоменеме дека креирањето на табела на овој начин, со користење на MySQL Workbench, претставува само полесен начин да се дојде до упитот кој е неопходен да се изврши на MySQL серверот за да се добие саканата табела.



Со потврдување на овој упит тој ќе биде извршен на серверот и со самото тоа табелата ќе биде креирана.

По креирањето на табелата product, потребно е да ги креираме и останатите табели, по ER дијаграмот. Наредната табела која ќе ја креираме е табелата за сместување на информациите за клиентите customer:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
customer_id	INT	☒	☒	☐	☐	☐	☐	☒	☐	
first_name	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
last_name	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
email	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
date	DATE	☐	☒	☐	☐	☐	☐	☐	☐	

7.2. НАДВОРЕШЕН КЛУЧ (FOREIGN KEY)

Досега ги креиравме табелите за ентитетите Customer и Product. Останува да ја разгледаме релацијата меѓу овие два ентитета. Со оглед на тоа дека релацијата поседува сопствени атрибути кои ја опишуваат поблиску, а исто така постои потреба од чување на информации за продажбите, се наметнува логичен заклучок дека релацијата во овој случај ќе биде нова табела. Релациите повеќе-на-повеќе скоро секогаш креира нова табела.

Релацијата ги поседува атрибутите Number (за бележење на бројот на продажбата) и Timestamp (за чување на времето на купување). Логично е да чуваме информации за тоа кој го извршил купувањето и што купил. Ова значи дека е потребно да забележуваме кој клиент (customer) кој производ (product) го купил. Освен овие две колони кои ќе ги добиеме со мапирање на атрибутите ќе креираме уште две колони за идентификација на клиентот и производот.

Се наметнува прашањето кој податок да го користиме за идентификација на клиентот и производот? Како најдобар кандидат за оваа цел се наметнуваат идентификационите колони на табелите customer и product.

Колоните кои претставуваат референци до некои други табели се нарекуваат **надворешни клучеви**.

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
buy_id	INT	☒	☒	☐	☐	☐	☐	☒	☐	
number	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
time	DATETIME	☐	☒	☐	☐	☐	☐	☐	☐	
customer_id	INT	☐	☒	☐	☐	☐	☐	☐	☐	
product_id	INT	☐	☒	☐	☐	☐	☐	☐	☐	

Колоните `customer_id` и `product_id` всушност претставуваат вредности на овие колони за конкретни клиенти и производи. Тоа се колоните преку кои се остварува релацијата меѓу табелите, т.е. надворешни клучеви. Сепак, никаде не ја дефиниравме оваа поврзаност. Колоните `customer_id` и `product_id` се обични колони како и сите други. Ова значи дека на никој начин не е специјално нагласено дека овие колони се надворешни клучеви. MySQL овие колони ќе ги третира како и останатите.

Да го генерираме сега упитот за креирање на оваа табела со притискање на тастерот Apply.

По креирањето на табелата, исклучиво демонстративно во неа ќе извршиме еден внес:

```
INSERT INTO my_first_db.buy (number, time, customer_id, product_id)
```

```
VALUES ('234/23', now(), 2, 5);
```

Во табелата `buy` внесовме еден запис, т.е. ред. За број на продажбата внесовме `234/23`, за време на продажбата го поставивме моменталното време, со користење на функцијата `now()`, за ID на клиентот наведовме `2`, а за ID на производот наведовме `5`, но во табелите за клиенти и производи немаме никаков внес. Ова значи дека во табелата за продажба додадовме продажба која се одиграла без никакви актери: со непостоечки клиенти и непостоечки производи.

Притоа, откако ќе го извршиме ваквиот внес, не е пријавена никаква грешка, бидејќи не постои систем кој ќе ни каже дека клиентот под број 2 и производот под број 5 не постојат. Таквите грешки (логички грешки) се многу опасни, бидејќи не го спречуваат извршувањето на програмата, но програмата не работи правилно и може да биде тешко да се воочи.

Најдоброто решение за вакви проблеми е да се искористат вградените механизми кои постојат во одредени системи за управување со бази на податоци. Тоа се системите за одржување на референцијален интегритет, кој се спроведува со дефинирање на ограничувања со надворешни клучеви (Foreign Key Constraint).

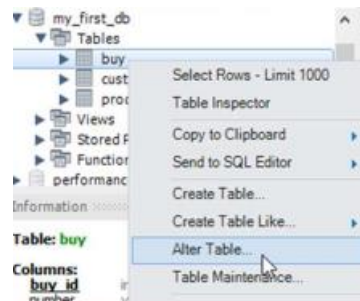
7.3. РЕФЕРЕНЦИЈАЛЕН ИНТЕГРИТЕТ (FOREIGN KEY CONSTRAINT)

Надворешниот клуч е симбол на релационата база на податоци. Ограничувањето надворешен клуч (Foreign Key Constraint) ја одржува физичката релација меѓу две табели. Едноставно кажано, не дозволува да биде внесена вредност во едната табела, доколку таквата вредност не постои во другата табела на поврзаната колона, односно, не го дозволува она што пред малку сакавме да го направиме.

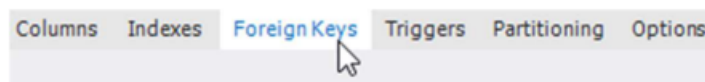
Треба да имаме во предвид дека ограничувањата не функционираат на MyISAM механизмите за складирање.

Сега ќе ја покренеме опцијата за измена ALTER на табелата buy. Препорака е прво да се креира табелата со се што треба да содржи, освен надворешните клучеви, па да се креира со Apply, па дури потоа да се покрене Alter и да се додадат надворешните клучеви. На овој начин ќе се избегнат проблеми околу тоа да MySQL Workbench не ги препознае колоните, индексите, односно местата каде сакаме да ги поставиме надворешните клучеви.

Измената на постоечка табела во MySQL Workbench покренуваме со избор на опцијата Alter Table од контекстното мени кое се добива со десен клик на табелата.



За да го додадеме надворешниот клуч, доволно е од просторот за креирање на табели да го избереме табот Foreign Key:



Во новоотворениот прозорец се додава назив на надворешниот клуч во колоната Foreign Key Name и се избира референтната табела (табелата со која постои релација). Во нашиот случај тоа ќе биде табелата customer.

Foreign Key Name	Referenced Table
fk_customer_id	`my_first_db`.`customer`

По ова во десниот дел од прозорецот ќе може да избереме конкретни колони од табелата која се креира и референцираната табела. Во нашиот случај, двете колони имаат идентичен назив customer_id, но тоа не секогаш е случај.

Column	Referenced Column
☐ buy_id	
☐ number	
☐ time	
☒ customer_id	customer_id
☐ product_id	

Не крај, може (а и не мора) да поставиме додатни параметри на foreign key ограничувачот. Тоа се On Delete и On Update особините:

Foreign Key Options

On Update: NO ACTION ▼

On Delete: NO ACTION ▼

☐ Skip in SQL generation

Може да забележиме дека овие особини иницијално се поставени на вредност NO ACTION.

Прво е важно да објасниме во кој контекст се важни овие параметри. Ќе претпоставиме дека во базата која ја креираваме имаме стотина записи за клиенти и исто толку за производи и дека во базата имаме регистрирано голем број на клиенти кои имаат купувано различни производи. Што би се случило доколку избришеме некој клиент од табелата customer? Во сите записи во табелата за продажби во кои се појавува овој клиент ќе останат податоци кои покажуваат на непостоечки клиент. Всушност овие параметри се однесуваат на прашањето што ќе се случи со поврзаните податоци во вакви ситуации.

Ќе го погледнеме значењето на достапните опции за овие параметри:

RESTRICT: Во опишаната ситуација, бришењето на клиентот или измена на идентификационата колона нема да бидат дозволени.

NO ACTION: Оваа опција е идентична на претходно опишаната RESTRICT, т.е. има ист ефект. Ова е подразбирливата опција доколку ништо не се подесува.

CASCADE: За параметар On Update оваа опција претставува многу добро решение. Доколку е избрана, едитирањето на примарниот клуч на клиентот ќе доведе до идентична промена во сите поврзани табели, т.е. секаде каде што се појавува клиентот со оваа идентификација.

Доколку оваа опција се користи за параметарот On Delete, може да биде многу опасна, бидејќи може да предизвика несакано губеше на податоци. Во нашиот случај, доколку би го избришале клиентот автоматски би се избришале и сите негови податоци од табелата за продажби. Ова не е резултатот кој го сакаме, па во вакви ситуации оваа опција не се препорачува.

SET NULL: Подобра варијанта од претходно опишаната во случаи на бришење е опцијата SET NULL. Кога оваа опција е вклучена, доколку би го избришале клиентот, во табелата за продажби, податоците за овој клиент би се нулирале.

Навраќајќи се на креирањето на надворешните клучеви, на ист начин го креираме и другиот надворешен клуч, со оглед на тоа дека релацијата buy е во однос со двете табели – product и customer.

Кога станува збор за надворешните клучеви, важно е да го напоменеме и дека треба да се избегнува внесување на податоци во табелата која сакаме да ја поврземе со надворешен клуч пред истото да го направиме, бидејќи може да се случи самата платформа да не ни дозволи да креираме надворешен клуч. Истото може да се случи доколку се обидуваме да споиме две табели, но колоните да имаат нешто различно(да речеме различна големина на податоци). Тоа значи дека колоните кои ги дефинираме мора да имаат идентични поставки. И последното нешто што се однесува на овој сегмент е делот со индексите. За да постави надворешен клуч, MySQL Workbench, креира индекси на тие колони пред поставувањето на надворешниот клуч. Понекогаш може да се случи да не се креираат индекси, па препорака е да ги креираме сами и дури тогаш да ги поставиме надворешните клучеви.

По додавањето на надворешните клучеви, креиран е основниот логички дизајн, т.е. извршивме мапирање на концептуалниот дизајн во конкретни табели со релации.

8. ВОВЕД ВО SQL

SQL е единствен јазик за бази на податоци кој е во широка примена. Откако е претставен во раните 70 години на минатиот век, бил критикуван, менуван и прошируван, за потоа да биде прифатен од сите релевантни извршители на базите на податоци.

SQL станал стандард на Американскиот национален завод за стандардизација (ANSI) 1986 година и Меѓународната организација за стандардизација (ISO) 1987 година.

И покрај стандардизацијата, SQL кај различни системи за управување со бази на податоци не е во потполност компатибилен, бидејќи секој произведувач го имплементира SQL со одредени отстапувања и применувајќи специфично проширување.

SQL примарно се користи за формулирање на инструкции за системите на бази на податоци кои опфаќаат наредби за манипулација со базата (структурата) и податоците.

8.1. SQL СИНТАКСА

Карактеристики на SQL синтаксата се:

- SQL не е чувствителен на мали и големи букви;
- Секој SQL упит (доколку ги има повеќе) мора да завршува со ; (овој сепаратор може да се промени);
- Стринговите се пишуваат исклучиво во наводници; MySQL прифаќа и единечни и двојно наводници за стринг, но најдобра пракса е да се користат единечните;
- Објектите во SQL може да се стават во Backtick наводници (select * from 'my table'), меѓутоа тоа не е задолжително;
- Променливите во SQL секогаш започнуваат со знакот @.

8.1.1. ТЕКСТУАЛНИ КОНСТАНТИ

Стринг претставува секвенца од бајти или карактери, ограничени со единечни (') или двојни наводници ("):

'a string'

"another string"

Препорака е стринговите да се означуваат со единечни наводници, поради ANSI QUOTES модот кој може да биде активиран на серверот, по кој стринговите со двојни наводници се интерпретираат како идентификатори.

Треба да напоменеме дека само стринговите, односно секвенците од карактери се ставаат во наводници, а останатите делови, како бројни или други вредности, објекти како табели, погледи и сл., не се ставаат под наводници. Како што претходно веќе спомнавме објектите може да ги ставаме под Backtick наводници.

Генерално, постојат два типа на стрингови во MySQL, а тоа се бинарни стрингови и небинарни стрингови, односно стрингови кои се составени од повеќе карактери. Разликата меѓу овие два стринга е во основната градивна единица, која за бинарни стрингови е бајт, додека за небинарни стрингови е карактер. Уште една многу битна разлика е тоа што за бинарни стрингови не може да се дефинира сет карактери или колација, со оглед дека се чуваат како секвенца од бајтови. Во пракса најчесто се користат небинарни стрингови базирани на карактери.

Освен стандардниот текст, односно стандардните карактери (букви, броеви, стандардни специјални карактери како извичници, прашалници и слично), постои одредена група на карактери која специјално се интерпретира во стринг константи, слично како во другите програмски јазици:

Escape Sequence	Репрезентација
\0	Null или 0x00
\'	Единечен наводник
\"	Двоен наводник
\b	Backspace
\n	Newline
\r	Carriage return
\t	Tab
\\	Backslash
_	Долна црта
\%	Процент

8.1.2. КОМЕНТАРИ

MySQL серверот подржува три вида на коментари:

- Еднолиниски коментар со наведување на карактерот # (се од овој карактер до крајот на линијата се смета за коментар):

```
mysql> SELECT 1+1; # This comment continues to the end of line
```

- Еднолиниски коментар со наведување на карактерот -- (подржано од верзија 3.23.3):

```
mysql> SELECT 1+1; -- This comment continues to the end of line
```

- Повеќелиниски коментар кој е креиран исто како и во програмскиот јазик C, со користење на ознаката /* и */, каде се меѓу овие ознаки претставува коментар:

```
mysql> SELECT 1 /* this is an in-line comment */ +1;
```

```
mysql> SELECT 1+
```

```
/*
```

```
this is a
```

```
multiple-line comment
```

```
*/
```

```
1;
```

MySQL серверот познава и еден вид на специјални коментари, кои се нарекуваат условни коментари, а ги препознава и извршува само MySQL серверот, додека другите сервери го гледаат како најобичен коментар. Нивната синтакса е следна:

```
/*! MySQL-specific code */
```

На пример,

```
SELECT /*! STRAIGHT JOIN*/ col1 FROM table1, table2 WHERE.....
```

8.2. НАРЕДБИ

Сите дијалекти на SQL се карактеризираат со три сета на наредби:

Наредби за манипулација (Data Manipulation Language - DML)

Овој сет на наредби е срцето на SQL. Тоа се наредби кои се користат најчесто и во големи количини. Наредби за манипулација со податоците се: SELECT, INSERT, UPDATE и DELETE.

За манипулационите наредби се вели дека се CRUD наредби, што претставува скратеница од CREATE, READ, UPDATE и DELETE.

Наредби за дефиниција (Data Definition Language - DDL)

Во овој сет на наредби спаѓаат CREATE, ALTER и DROP. Овие наредби се користат за ракување со објектите на серверот, па тие најчесто му се достапни само на администраторот на серверот.

Пример за дефинициона наредба е:

```
CREATE database my_db;
```

или

```
create table my_table
```

```
(id int primary key auto_increment,
```

```
first_name varchar(50),
```

```
last_name varchar(50)
```

```
);
```

Наредби за контрола (Data Control Language - DCL)

Наредбите за контрола се користат за управување со правата на серверот. Тоа се наредбите GRANT и REVOKE. Со наредбата GRANT се доделуваат права, а со наредбата REVOKE се одземаат привилегиите на корисниците на серверот.

9. НАРЕДБИ ЗА ДЕФИНИЦИЈА (DATA DEFINITION LANGUAGE - DDL)

Збирот од дефинициони наредби (DDL наредби) го сочинуваат три наредби: CREATE, ALTER и DROP. Тие се однесуваат исклучиво на структурата, а не на самите податоци.

Ако сакаме преку SQL скрипта да креираме база на податоци или некои нејзини ентитети ќе го користиме овој сет на наредби.

CREATE

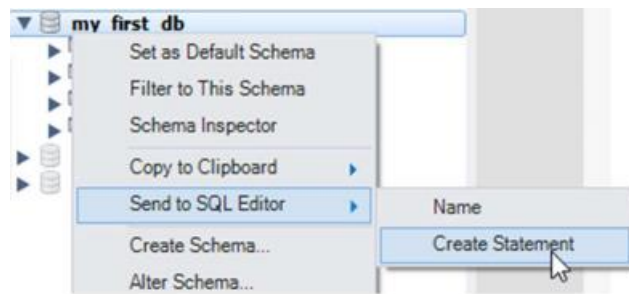
Секој објект кој се креира во базата на податоци, вклучувајќи ја и самата база на податоци, се креира со оваа наредба.

Синтаксата е едноставна:

CREATE *object_type* *object_name*

Меѓутоа, и овде некои работи зависат од природата на објектот. Затоа CREATE може да се согледа и како подгрупа на дефиниционите наредби, а не само како една наредба.

На пример, ние веќе со помош на MySQL Workbench креиравме база, кој понатаму генерираше упит. Како изгледа овој упит, може да погледнеме и сега, по креирањето на базата, со избор на опцијата Send to SQL Editor, па Create Statement од контекстното мени кое се добива со десен клик на базата на податоци.



Упитот кој ќе биде прикажан во SQL едиторот е:

```
CREATE DATABASE my_first_db /*!40100 DEFAULT CHARACTER SET utf8 COLLATE utf8_unicode_ci*/ /*!80016 DEFAULT ENCRYPTION='N' */;
```

Синтаксата на секоја CREATE наредба (и сите останати SQL наредби) може да се најдат во документацијата.

ALTER

Наредбата ALTER служи за модификација и ажурирање на структурата. Таа се користи во еден од двата облици: во едниот облик таа се однесува како наредбата CREATE и прифаќа комплетно нова дефиниција на објектот со измените. Во другата, потребно е експлицитно да се нагласат измените и само тие да се внесат преку соодветните модификатори. Самите облици зависат од типот на објектот над кој оваа наредба се извршува.

На пример, доколку погледот е креиран со следнава скрипта:

```
CREATE VIEW my_view AS select 'my view'
```

Може да биде изменет со следнава скрипта:

```
ALTER VIEW my_view as select 'my edited view'
```

Забележуваме дека е внесена цела скрипта со модификации.

Од друга страна, доколку направиме табела:

```
CREATE TABLE my_table (id int, name varchar(50))
```

А потоа сакаме да ја измениме нејзината структура така што ќе добиеме и колона за презиме, нема да можеме да ја измениме структурата на табелата со наредбата:

```
ALTER TABLE my_table (id int, name varchar(50), surname varchar (50))
```

Туку би морале експлицитно да нагласиме што менуваме и на кој начин:

```
CREATE TABLE my_table ADD COLUMN surname varchar (50))
```

Бидејќи станува збор за додавање на колона, го искористивме операторот ADD, а потоа типот на објект COLUMN, по што го структуриравме и самиот објект.

Доделувањето на надворешен клуч на колоната има сосема различна синтакса:

```
ALTER TABLE my_table ADD CONSTRAINT fk_cityid FOREIGN KEY
```

```
fk_cityid(cityid) REFERENCES city(city)
```

DROP

За разлика од ALTER и CREATE, оваа наредба нема многу широк спектар на опции. Се користи за бришење на објекти и како параметри го прифаќа типот и називот на објектот:

```
DROP TABLE my_table
```

```
DROP DATABASE my_first_db
```

```
DROP INDEX my_idx
```

Наредбата DROP често се среќава и во ваква комбинација:

```
DROP DATABASE IF EXISTS my_database;
```

Доколку се обидеме да избришеме непостоечка база:

```
DROP DATABASE my_database
```

Серверот би пријавил грешка:

Can't drop database 'my_database'; database doesn't exist

Оваа грешка не е проблематична доколку сакаме само да избришеме некоја база или некој друг објект, но често сакаме да избришеме објект за да направиме нов истоимен објект. Во тој случај може да дојде до неколку грешки:

Се обидуваме да избришеме база која не постои;

Се обидуваме да креираме база која веќе постои.

Доколку употребиме ваков блок, може да бидеме сигурни дека нема да дојде до грешка:

```
DROP DATABASE IF EXISTS my_database;
```

```
CREATE DATABASE my_database;
```

На истиот начин може да креираме и некој друг вид на објект – на пример табела:

```
DROP TABLE IF EXISTS my_table;
```

```
CREATE TABLE my_table(id int primary key auto_increment, name varchar(50));
```

DDL на сите креирани објекти во базата може да ги видиме со наредбата SHOW CREATE, по која ќе го напишеме типот и називот на објектот кој сакаме да го видиме:

```
SHOW CREATE DATABASE my_first_db;
```

```
SHOW CREATE TABLE my_table;
```

```
SHOW CREATE VIEW my_view;
```

Вежба 6: Потребно е да се креира табела cars која ќе ги чува следните податоци: ID на автомобилот кој ќе се зголемува автоматски за еден при секој внес и кој ќе биде примарен клуч на табелата, назив и година на производство.

Решение:

```
create table cars
(
  car_id int primary key auto_increment,
  car_name varchar(256),
  car_production_year year
)
```

Вежба 7: Потребно е да се креира табела users која ќе ги чува следните податоци: ID на корисникот кој ќе се зголемува автоматски за еден при секој внес и кој ќе биде примарен клуч на табелата, име и презиме на корисникот.

Потребно е да се креира и дополнителен упит кој на креираната табела ќе и додаде уште едно поле со шифра на корисникот.

Решение:

```
create table users
(
  user_id int primary key auto_increment,
  user_firstName varchar(50),
  user_lastName varchar(50)
)
Alter table users add column password varchar(15)
```

Вежба 8: Потребно е да се креира скрипта за креирање на база на податоци application_06. Оваа скрипта ќе ја брише постоечката база доколку постои. Исто така потребно е да се креира скрипта за креирање на табела computer_parts, така што ќе ја избрише постоечката истоимена табела доколку постои. Оваа табела треба да содржи податоци за ID на уредот, шифра на уредот, тип на уредот, назив и цена.

Решение:

```
drop database if exists application_06;
create database application_06;

drop table if exists computer_parts;
create table computer_parts
(
  computer_part_id int primary key auto_increment,
  computer_part_code varchar(5),
```

```
computer_part_type int,  
computer_part_name varchar(256),  
computer_part_price decimal(7,2),  
)
```

10. ОСНОВНИ НАРЕДБИ ЗА ПРЕБАРУВАЊЕ (SELECT)

SELECT

Наредбата SELECT е наредба која емитува табеларен резултат на основа на внесени параметри. Иако наредбата често се користи за прикажување на податоците, не е задолжена за логичкото преземање на тие податоци, туку само за нивниот приказ. SELECT го поседува изворот на податоците и преземањето на резултатите од тој извор го подразбира и клучното зборче FROM на кој му следи изворот (табелата):

```
SELECT * FROM film;
```

РЕСТРИКЦИЈА НА КОЛОНИ

Оваа наредба ги прикажува сите редови и сите колони од табелата филм. Но, честопати нема да бидеме заинтересирани за приказ на сите колони од една табела, бидејќи доколку ја внесеме наредбата:

```
SELECT * FROM actor;
```

Ќе ги добиеме податоците од сите колони за сите актери запишани во табелата actor. Понатаму, доколку би сакале да ги добиеме само имињата на сите актери во базата, ќе напишеме:

```
SELECT first_name FROM actor;
```

А за приказ на имињата и презимињата, ќе напишеме:

```
SELECT first_name, last_name FROM actor;
```

Секогаш треба да ги прикажуваме податоците само од оние колони чии податоци ни се потребни. На тој начин се растеретува базата на податоци, клиентскиот програм, како и самата мрежа преку која патуваат податоците.

COUNT

Често се јавува потреба да се утврди вкупниот број на записи во некоја табела. Ова се постигнува со следната наредба:

```
SELECT count(film_id) FROM film;
```

Во наведената наредба може да се користи било која колона од табелата film наместо користење на колоната film_id. Може да се користи и ѕвездичка (*) за сите колони. Резултатот ќе биде ист.

DISTINCT

За утврдување на бројот на единствени записи се користи клучното зборче DISTINCT. На пример:

```
SELECT COUNT (DISTINCT first_name) FROM actor;
```

Со оваа наредба се утврдува колку актери постојат во базата кои имаат единствено име. Секако, повеќе актери може да имаат идентично име, па бројот на актери со единствено име ќе биде помал од вкупниот број на актери во базата.

LIMIT

За лимитирање на бројот на резултатите кои сакаме да бидат прикажани се користи клучното зборче LIMIT:

```
SELECT first_name, last_name FROM actor LIMIT 2;
```

Наредбата LIMIT може да се користи и со дефинирање на таканаречен offset. Со помош на offset се дефинира од кој внес базата ќе ја започне претрагата:

```
SELECT first_name, last_name FROM actor LIMIT 10, 5;
```

Со помош на наведената наредба ќе бидат прескокнати првите 10 записи и ќе бидат прикажани наредните 5.

ORDER BY

Основното сортирање се извршува со ORDER BY. Доколку би сакале да ги добиете имињата и презимињата на сите актери, а покрај тоа да бидат и сортирани според името од А до Z, ќе напишеме:

```
SELECT first_name, last_name FROM actor ORDER BY first_name;
```

Подразбирливото сортирање, при употреба на клучното зборче ORDER BY, е сортирањето од помала вредност кон поголема, и доколку сакаме сортирањето да биде во обратен редослед (од поголема вредност кон помала), тоа треба експлицитно да го нагласиме со клучното зборче DESC (од поголема кон помала) и ASC (од помала кон поголема):

```
SELECT first_name, last_name FROM actor ORDER BY first_name DESC;
```

WHERE

Најчесто нема да нѝ бидат потребни сите записи од некоја табела, па дури ни дел од нив, што го постигнавме со употреба на лимитирање. Во повеќето случаи ќе нѝ бидат потребни записи кои ќе задоволуваат точно дефинирани критериуми. Овие критериуми се наведуваат со клучното зборче WHERE.

На пример, доколку би сакале да ги прикажеме називите на сите филмови од табелата филм кои во колоната rating имаат вредност PG(Parental Guidance). Тоа ќе го постигнеме преку наредбата:

```
SELECT title FROM film WHERE rating = 'PG';
```

За споредување не мора да се користи операторот за еднаквост. За приказ на називите на сите филмови кои се подолги од 2 часа, ќе напишеме:

```
SELECT title FROM film WHERE length>120;
```

LIKE

За пребарување на основа на само некој дел од некој податок, може да се користи клучното зборче LIKE. За приказ на сите актери чие име започнува со карактерите 'an', ќе напишеме:

```
SELECT first_name, last_name FROM actor WHERE first_name LIKE 'an%';
```

На сличен начин може да утврдиме дали записот од некоја колона завршува на одреден начин:

```
SELECT first_name, last_name FROM actor WHERE first_name LIKE '%en';
```

Може да утврдиме дали записот од некоја колана воопшто содржи некоја секвенца карактери било каде (на почетокот, на крајот, во средината):

```
SELECT first_name, last_name FROM actor WHERE first_name LIKE '%en%';
```

AND

Условените изрази во кои се користи повеќе од еден услов со поврзуваат со логички оператори. На пример, доколку би сакале да го селектираме сите филмови чии називи започнуваат на буквата 'B' и чие траење е подолго од 3 часа, ќе напишеме:

```
SELECT title, length FROM film WHERE title LIKE 'B%' AND length > 180;
```

Во условен израз може да го ставиме и самиот упит:

```
SELECT title, length FROM film WHERE title LIKE 'B%' AND length > (SELECT 180);
```

GROUP BY

SQL овозможува синтакса со која може да се врши групирање на резултатите според одредени колони. Тоа се постигнува со употреба на клучното зборче GROUP BY:

```
SELECT rating FROM sakila.film
```

```
GROUP BY rating;
```

Наредбата GROUP BY скоро секогаш се употребува со агрегатни функции.

```
SELECT rating, count (title) FROM sakila.film
```

```
GROUP BY rating;
```

HAVING

За да извршиме филтрација на резултатите по групирањето се користи клучното зборче HAVING:

```
SELECT rating, count (title) FROM sakila.film
```

```
GROUP BY rating HAVING count (title) > 200;
```

Вежба 9: Потребно е да се прземат сите податоци за сите филмови кои во својот назив го содржат зборот bird и имаат rental_duration поголем од 5.

Решение:

```
SELECT * FROM sakila.film WHERE title LIKE '%bird%' AND rental_duration > 5;
```

Вежба 10: Потребно е да се селектираат сите називи на филмови со rental_rate поголем од 3, подолги од 2 часа и пократки од 3 часа.

Решение:

```
SELECT title FROM film WHERE rental_rate > 3 AND LENGTH > 120 AND length < 180;
```

Вежба 11: Потребно е да се утврди колку различни рејтинзи на филмови постојат. Станува збор за Parental Guidance рејтингот, кој е дефиниран во колоната rating.

Решение:

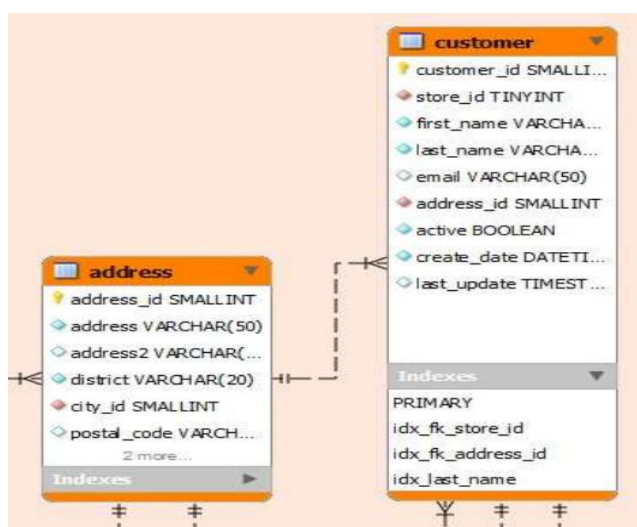
```
SELECT COUNT (DISTINCT rating) FROM film;
```

11. ПОВРЗУВАЊЕ НА ПОДАТОЦИТЕ ОД ПОВЕЌЕ ТАБЕЛИ

Се досега , секој упит со SELECT наредбата прикажување податоци само од една табела. Во релационите бази на податоци, главно работиме со повеќе поврзани табели. Поради тоа често ќе бидеме во ситуација да го применуваме SELECT во комбинација на податоци од повеќе табели.

11.1. СТУДИЈА НА СЛУЧАЈ

Ќе разгледаме две табели кои постојат во базата на податоци Sakila. Тоа се табелите customer и address:



Во склоп на табелата customer не постојат конкретни информации за, на пример името на улицата на корисникот, туку таквите податоци се групирани во посебна табела address. Двете прикажани табели поседуваат атрибут address_id, кој во табелата customer претставува надворешен клуч, чија цел е поврзување на двете табели.

За да ја добиеме адресата на корисникот ќе напишеме:

```
SELECT first_name, last_name, address FROM customer, address;
```

Еден од начините на кој можеме овој упит да го подобриме за да ги добиеме посакуваните резултати е да го наведеме условот за појавување на резултатот,

т.е. да ја наведеме релацијата меѓу табелите. Тоа може да го постигнеме со користење на клучното зборче **WHERE**:

```
SELECT first_name, last_name, address FROM customer, address WHERE
```

```
address.address_id = customer.address_id;
```

JOIN

JOIN е клучен збор во SQL, кој означува спој меѓу две табели. Оваа наредба никогаш не се појавува сама, туку задолжително доаѓа во комбинација со клучното зборче ON (или во ретки случаи и со некои други клучни зборчиња).

```
SELECT customer.first_name, customer.last_name, address.address
```

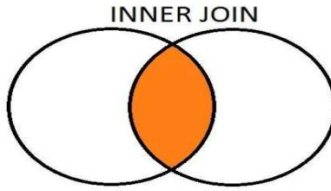
```
FROM customer JOIN address
```

```
ON address.address_id = customer.address_id;
```

Постојат неколку вида на спојување со JOIN:

- Inner Join
- Left Join
- Right Join
- Cross Join

Inner Join веќе го видовме во претходниот пример. Потполно идентичен ефект има и користењето на клучното зборче JOIN и Inner Join и во двата случаи ќе бидат селектирани редовите кои за дефинираната колона имаат вредности и во двете табели. Говорејќи хипотетички, ова значи дека, доколку за некој од корисниците нема внесено адреса, корисникот нема да се прикаже во резултатите.

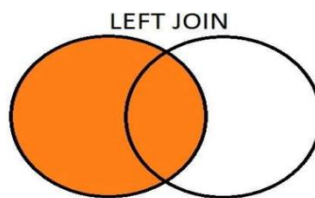


```
SELECT customer.first_name, customer.last_name, address.address, address.district
```

```
FROM customer INNER JOIN address
```

```
ON address.address_id = customer.address_id;
```

Спојувањето на табелите дефинирано со **Left Join** ќе ги врати сите записи од левата табела, па дури и доколку некои од записите во бараната колона немаат вредност.

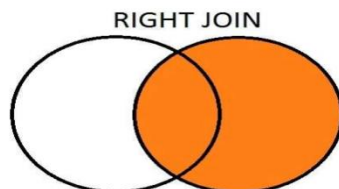


```
SELECT customer.first_name, customer.last_name, address.address, address.district
```

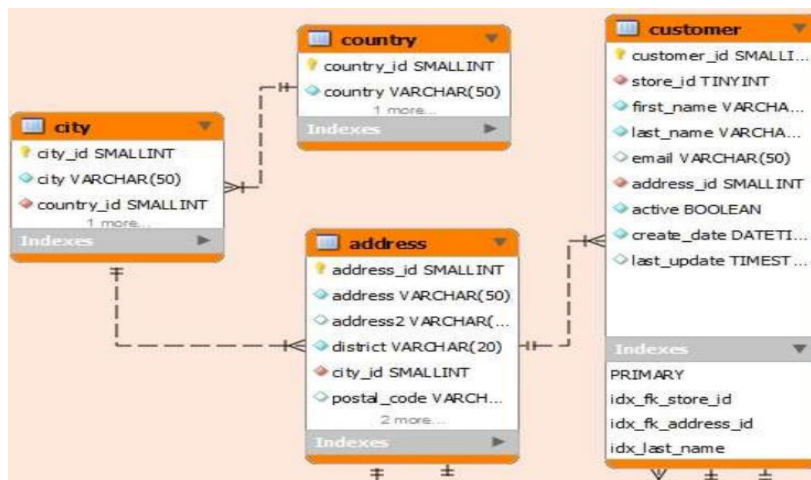
```
FROM customer LEFT JOIN address
```

```
ON address.address_id = customer.address_id;
```

Слично со **Right Join** ќе ги добиеме резултатите од десната табела, но само они кои имаат дефинирана вредност во истата колона од левата табела.



Со помош на **JOIN** може да се поврзат и повеќе од две табели:



```

SELECT customer.first_name, customer.last_name,
address.address, address.district, city.city, country.country

FROM customer

JOIN address ON address.address_id = customer.address_id

JOIN .city ON city.city_id = address.city_id

JOIN country ON country.country_id = city.country_id
  
```

Вежба 12: Потребно е да се добие листа од сите филмови со податоци за соодветниот жанр на филмот.

Решение:

Во табелата film не постои податок за категорија, т.е. жанр на филмот. Жанровите се чуваат во посебна табела category, додека релациите меѓу табелата film и category се обработени во посебна табела film_category.

```

SELECT film.title, category.name
FROM film INNER JOIN film_category
ON film.film_id = film_category.film_id
INNER JOIN category
ON category.category_id = film_category.category_id;
  
```

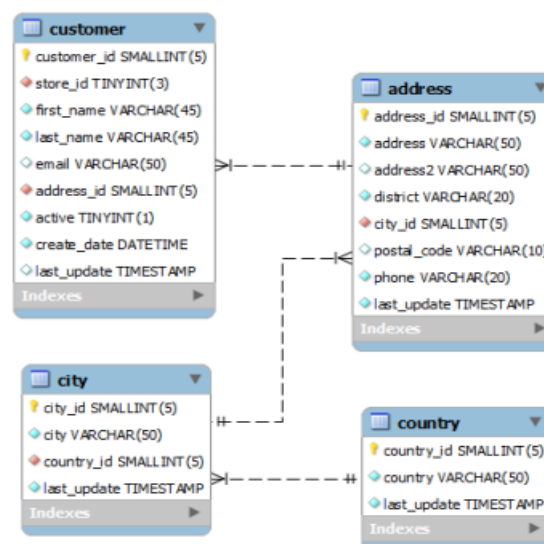
12. НАРЕДБИ ЗА ИЗМЕНА НА ПОДАТОЦИТЕ (INSERT, UPDATE, DELETE)

INSERT

Со користење на наредбата INSERT се додаваат нови записи во табелата во базата на податоци. Основната синтакса на оваа наредба е:

INSERT INTO mytable (column1, column2...) VALUES (value1, value2...)

А погледнеме ER дијаграм за сместуваче на податоците за клиентите:



Да претпоставиме дека треба во базата на податоци да внесеме запис за нов клиент, чија држава на живеење не постои во базата. Со оглед на релациите кои постојат во табелите за чување на информации за клиентите, може да заклучиме дека за внес на нов клиент, потребно е прво да направиме внес во табелата address, за што е потребно претходно да направиме внес во табелата city, а за што повторно потребно е прво да направиме внес во табелата country.

Од сето ова може да заклучиме дека е неопходно да почнеме од табелата country. Практично, тоа значи дека секогаш кога имаме зависност на две табели, пред

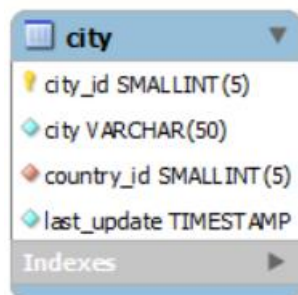
самоит внес мора да заклучиме која табела е водечка (табелата со која првенствено работиме – во нашиот пример, табелата customer), а потоа да утврдиме од која табела таа зависи, почитувајќи го редоследот (customer – address – city - country). По утврдувањето на зависностите и редоследот ќе започнеме од првата табела од крајот (во нашиот случај табелата country) и прво ќе направиме внес во неа, а потоа ќе продолжиме да внесуваме податоци во табелите се додека не дојдеме до главната табела (customer).

Внесот на податоците се постигнува со наредбата INSERT. Конкретно:

```
INSERT INTO country (country) VALUES ('Macedonia');
```

Може да се забележи дека не ги внесовме сите колони ниту вредностите за сите колони. Тоа е затоа што идентификационата колона на табелата country има подесено auto_increment својство.

Со увид во ER дијаграмот може да видиме дека е потребно да внесеме податоци и во табелите city и address. Па, ќе започнеме по ред. За да го додадеме градот во кој новиот клиент живее, ќе ги погледнеме атрибутите на оваа табела:



Значи, оваа табела содржи идентификациона колона (примарен клуч) city_id, па колона за сместување на името на градот city, колона за сместување на државата во која се наоѓа градот country_id и колона за бележење на времето кога вредноста последен пат е модификувана. Проблемот настанува сега затоа што за државата во која се наоѓа градот неопходно е да се внесе id вредност од табелата држава. Тоа е колоната country_id која е идентификациона колона во табелата country, т.е. надворешен клуч во табелата city:

```
INSERT INTO city (city, country_id) VALUES ('Skopje',
```

Почнавме да внесуваме вредности за колоната, ја наведовме вредноста за колона city, и стигнавме до проблем. Потребно е да наведеме country_id на државата која ја внесовме во базата. Како би можеле да знаеме под кој id број е заведена ако го немаме специфицирано id бројот.

Во ваква ситуација може да користиме функција која ќе ни го врати id редот кој е последен впишан во базата. Тоа е функцијата LAST_INSERT_ID:

```
INSERT INTO city (city, country_id) VALUES ('Skopje', LAST_INSERT_ID());
```

Функцијата LAST_INSERT_ID() се однесува само на моменталната конекција, така што внесовите од страна на другите корисници се игнорираат.

По ова потребно е да внесеме податок во табелата address, што ќе го направиме на истиот начин, затоа што ситуацијата се повторува:

```
INSERT INTO address (address, district, city_id, phone)
```

```
VALUES ('High Street 235', 'Central', LAST_INSERT_ID(), '0112534564');
```

На крај ќе го додадеме клиентот, со следната наредба:

```
INSERT INTO customer (store_id, first_name, last_name, address_id, create_date)
```

```
VALUES (1, 'Ivan', 'Simov', LAST_INSERT_ID(), curdate());
```

DELETE

Ова е наједноставната манипулациона наредба. Едноставно, ако напишеме:

```
DELETE FROM customer;
```

ќе биде избришана содржината на целата табела. Ова се случува затоа што немаме наведено ников услов за бришењето, т.е. никаква филтрација на податоците.

Треба да имаме на ум дека после ваквата операција не постои UNDO наредба која би ги вратило податоците, затое оваа наредба треба да ја употребуваме внимателно.

Упитот, напишан во таа форма нема никаква цел без филтрација, па наредбата DELETE најчесто (секогаш) се користи со филтрација:

DELETE FROM customer WHERE id = 10;

или

DELETE FROM customer WHERE id BETWEEN 10 AND 20;

Овде, за прв пат се стекаваме со наредбата BETWEEN. Нејзината примена е јасна – ги прикажува сите вредности кои се во зададениот ранг (10 и 20), вклучувајќи ја и самата вредност од условот, т.е. идентично на:

DELETE FROM customer WHERE id >= 10 AND <= 20;

Кога табелата поседува auto_increment својство, инкрементационата вредност која ја генерира серверот никогаш не се враќа назад. Тоа значи дека, ако имаме внесувано редови кои имале генерирани вредности 1, 2, 3, 4, 5, 6....., а потоа некои од нив имаме избришано, на пример 3, листата ќе ги има следните редови: 1, 2, 4, 5, 6. На следниот внесен ред серверот нема да му додели вредност 3 (што би било логично, бидејќи таа вредност сега не постои), туку 7, бидејќи тоа е наредната вредност во auto-инкрементацијата.

Значи, ако имаме табела со, на пример, последна инкрементирана вредност 1000, и извршиме наредба DELETE над целата табела, а потоа направиме нов внес со наредбата INSERT, вредноста на инкрементираната колона ќе биде 1001, иако внесената вредност ќе биде единствена во табелата.

Ова може да се избегне со наредбата TRUNCATE, која исто така ја брише содржината на табелата, но притоа го ресетира и бројачот на аутоинкрементационата колона:

```
TRUNCATE DATABASE my_first_db;
```

Оваа наредба за разлика од наредбата DELETE не е логирана наредба. Тоа значи дека доколку би морале да вршиме враќање на базата, оваа наредба нема да биде забележана.

UPDATE

Наредбата UPDATE служи за ажурирање на одреден ред или редови. Нејзината синтакса е прилично едноставна. Доколку сакаме да го измениме презимето на клиент, може да напишеме:

```
UPDATE customer SET last_name = 'peterson';
```

Или доколку би сакале да ги измениме и името и презимето:

```
UPDATE customer SET first_name = 'john', last_name = 'peterson';
```

Треба да бидеме внимателни со овие упити, бидејќи било кој од нив ги менува сите редови во табелите, што најчесто не е она што го сакаме. Како и за наредбата DELETE, така и за UPDATE, скоро секогаш е потребна филтрација, односно наведување на условот на упитот:

```
UPDATE customer SET first_name = 'john', last_name = 'peterson' WHERE customer_id = 5;
```

или

```
UPDATE customer SET first_name = 'john', last_name = 'peterson'
```

```
WHERE first_name = 'david' AND last_name = 'miller';
```

Вежба 13: Да се креира база на податоци test_db. Во базата да се креира табела users, која содржи три колони id, name и password и status.

Потребно е да се внесат три корисници (Peter,123; Jenny, 345; John, 678) и status на корисниците да се подеси за Peter и Jenny на 1, а на John на 0.

Решение:

```
CREATE database test_db;
```

```
use test_db;
```

```
CREATE table users (id int primary key auto_increment, name varchar(50), password varchar(15), status varchar(1));  
INSERT INTO users (name, password)  
VALUES ('Peter', '123', '1'), ('Jenny', '345', '1'), ('John', '567', '0');
```

Вежба 14: За претходно креираната табела users, потребно е да се креира упит за превземање на сите податоци за корисниците чии имиња започнуваат на буквата j

Решение: `SELECT * FROM users WHERE name LIKE 'j%';`

Вежба 15: За претходно креираната табела users, потребно е да се креира упит кој ги враќа корисниците чиј ID е поголем од 1 и помал од 5.

Решение: `SELECT * FROM users WHERE id > 1 AND id < 5;`

Вежба 16: Да се креира уште една табела во базата на податоци test_db. Табелата се вика statuses и содржи две полиња: ID и name. Потребно е да се додадат три називи на статуси во оваа табела: user, administrator и superadministrator.

Да се измени постоечката табела users со додавање на уште една колона, наречена status.

Решение:

```
CREATE table statuses (id int primary key auto_increment, name varchar(30));  
INSERT INTO statuses (name) VALUES ('user');  
INSERT INTO statuses (name) VALUES ('administrator');  
INSERT INTO statuses (name) VALUES ('superadministrator');  
ALTER TABLE users ADD column status int;
```

Вежба 17: Во табелата users да им се доделат статуси на корисниците, така што John да биде administrator, Jenny superadministrator, а Peter user.

Решение:

```
UPDATE users SET status = 2 WHERE name = 'John';  
UPDATE users SET status = 3 WHERE name = 'Jenny';  
UPDATE users SET status = 1 WHERE name = 'Peter';
```

Вежба 18: Да се прикаже името и статусот на корисникот со ID 2.

Решение:

```
SELECT users.name, statuses.name FROM users  
JOIN statuses ON users.status = statuses.id WHERE users.id = 2;
```

13.ИНДЕКСИ

Во управувањето со бази на податоци, клучна работа е брзото и прецизно претставување на податоците. Кога количините на податоците во табелите се мали, системот, во секој случај ќе функционира брзо, но со нивното зголемување, брзината станува основен проблем. Меѓутоа, брзината на достава на податоците не е таа која го прави овој процес спор, туку пронаоѓањето на самите податоци. Кога податокот еднаш ќе биде лоциран, брзината на неговото доставување е ирелевантна.

Еве како под стандардни услови, серверот доаѓа до податоците. Да речеме дека постои листа на градови (секој град е еден ред):

1	Paris
2	Bruges
3	London
4	Moscow
5	Krakow

Ако му кажеме на серверот да ја прикаже Москва, тој ќе започне секвенцијална потрага во табелата. Прво ќе го погледне првиот ред (Париз), па вториот (Бругес), па третиот... и секој од тие редови ќе го спореди со бараниот град. На крај,од четвртиот обид, би успеал да го пронајде бараниот град. Ова не звучи како брз процес. Посебно, доколку наместо пет, во листата би имало илјадници градови. Освен тоа, кога доаѓа до претрага во повеќе поврзани табели, работата значајно се зголемува и брзината драстично опаѓа.

Поради тоа, секоја база на податоци има механизам кој има само една цел: брзо лоцирање на бараниот податок. Овој механизам се нарекува индекс.

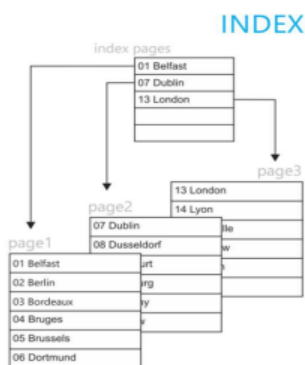
Најчесто објаснувањето на индексите започнува со споредување со содржината од книга. Тоа, освен што е сликовито, претставува и прилично добро споредување. Книгата на почетокот или на крајот содржи содржина и странична структура. За да дојдеме до една страница, колку и да е голема книгата, потребни

ни се неколку секунди. Но, за да дојдеме до одреден дел од текстот би ни требало многу повеќе време. Поради тоа, содржината на книгата структурира одредени целини и ги памети нивните позиции (страни), па лесно може да стигнеме до саканата содржина со правилна употреба на содржината.

13.1. МЕХАНИЗАМ НА ИНДЕКСИРАЊЕ

Веќе знаеме дека повеќето податоци во MySQL базата ги памети по страни. Големината на една страна е ограничена мемориски (обично 8к или 16к), но не може да се знае точно колку редови може да собере една страна, бидејќи тоа зависи од разни фактори. Пред се – од количината на податоците во редовите.

Вака изгледаат страните за табелите со градови:



Индексот се состои од редови во кои секој ред претставува една страна на вистински податоци, а полињата во тој ред претставуваат почетните редови на тие страници, заедно со бројот на редови. Кога бараме некој податок, започнува претрага по индексот.

Да речеме дека го бараме градот Лион. Во страните на индексот започнува секвенцијална претрага по вредноста Лион. Во првиот ред се проверува дали бараната вредност е поголема или еднаква во однос на вредноста од тој ред (вредностите во индексите секогаш се сортирани). Бидејќи Лион во оваа табела е на пониска позиција од Белфаст, се преминува на следниот ред. Даблин е на повисока позиција од Лион, па индексерот преминува уште еднаш на следниот ред. Кога индексерот ќе ја спореди третата табела, Лондон и Лион, условот

сеуште не е исполнет, бидејќи Лондон не е еднаков на Лион, а при тоа и понатаму е на повисока позиција. Затоа индексот преминува повторно на следната страна. Бидејќи на таа страна условите не се исполнети (Лион е на пониска позиција), индексот се враќа на претходната страна и започнува секвенцијална потрага по таа страна. Кога ќе дојде до бараниот податок, потрагата е завршена.

Кога го разгледуваме индексот, мора да имаме во предвид дека структурата на индексот е многу поголема од претставата на цртежот. Целиот индекс претставува едно големо В-стебло, со многу страни.

Сите страни од индексот кои не содржат податоци се нарекуваат non-leaf страни, додека страните кои содржат конечни податоци се leaf страни.

13.1.1. КЛАСТЕРИРАН И НЕКЛАСТЕРИРАН ИНДЕКС

Сага знаеме како функционира претрагата во индексот. Таа секојпат функционира на ист начин, но нејзината финализација може да се изврши на два начина и тие два начина прават да разликуваме два вида на индекси:

- Кластериран и
- Некластериран (секундарен).

Често, со полно право, овие два индекси се споредуваат со книга и телефонски именик. Доколку бараме некоја информација во книга, прво ќе ја пребараме содржината и во неа ќе ја најдеме страната за она што не интересира. Потоа ќе ја прелистаеме книгата и брзо ќе дојдеме до податокот, бидејќи ја знаеме страната. Доколку бараме некој податок во телефонски именик, ќе ја пронајдеме буквата која не интересира, а потоа директно ќе пристапиме на страната на која се наоѓаат сите податоци кои почнуваат со таа буква. Тоа е разликата меѓу кластериран и некластериран индекс. Кластерираните индекси на своето најниско ниво (leaf level) ги содржат самите податоци, додека некластерираните индекси содржат референци кон вистинските податоци. Од ова може да заклучиме дека

некластерираниот индекс побарува еден чекор повеќе за доаѓање до вистинските податоци, па тие се и поспори од кластерираните.

14. ПОГЛЕДИ (VIEWS)

Погледите претставуваат функционалност во MySQL која овозможува дефинирање на специјална репрезентација на една или повеќе табели. Ова значи дека со користење на погледи може да се креираат т.н. виртуелни табели, кои може да бидат комбинација од колони кои припаѓаат на различни табели. Над погледите може да се извршуваат SELECT упити, и во зависност од дефиницијата на погледот и правата на корисникот може да се вршат и измени на податоците со користење на наредбите INSERT, UPDATE и DELETE.

Генерално, двете најважни причини за користење на погледите се:

- Сигурност – Во многу ситуации имаме потреба на одредени корисници да им оневозможиме полн пристап до одредени табели. Типичен пример за ваква ситуација се табелите за чување на чувствителни податоци за вработените. Решението е креирање на поглед кој ќе ги содржи колоните до кои може да пристапат сите вработени. Вработените ќе имаат право на пристап до погледот, но не и до самата табела.
- Комфорт – Веќе досега видовме дека често постои потреба од спојување на податоците од повеќе табели. На пример, во базата на податоци Sakila, доколку би сакале да добиеме комплетен сет од податоците за корисниците, би морале да извршиме упит со кој ќе споиме податоци од неколку табели. За да ја олесниме работата на корисниците на базата на податоци или на програмерите кои ќе пишуваат апликација која ќе комуницира со базата, можеме да ги поедноставиме работите и да креираме поглед.

14.1. КРЕИРАЊЕ НА ПОГЛЕДИ

Погледите (Views) се објекти во базата на податоци и поради тоа, како и останатите објекти може да се креираат со DDL наредбата CREATE. Синтаксата е:

CREATE VIEW myView AS query;

Во оваа наредба, myView претставува назив на погледот, додека query по клучното зборче AS го претставува местото каде ќе се наоѓа упитот за добивање на податоците.

Евентуално, наредбата може да изгледа и вака:

CREATE OR REPLACE VIEW myView AS query;

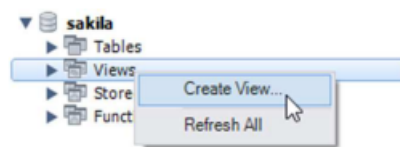
Оваа наредба ќе спречи појава на грешка доколку веќе е креиран истоимен поглед. Откако погледот е веќе креиран може да се модификува со наредбата ALTER:

ALTER VIEW myView AS query;

Погледот се брише со наредбата DROP:

DROP VIEW myView;

Поглед може да се креира и од MySQL Workbench. Потребно е од контекстното мени кое се добива со десен клик на Views да се избере опцијата Create View.



Во работната површина се пишува синтаксата со која се креира погледот.

Да се потсетиме сега на еден упит кој е потребно да го напишеме за да ги добиеме сите податоци за еден корисник од базата Sakila:

SELECT customer.first_name, customer.last_name, address.address,city.city, country.country

```
FROM customer JOIN address
```

```
ON address.address_id = customer.address_id
```

```
JOIN city ON city.city_id = address.city_id
```

```
JOIN country ON country.country_id = city.country_id;
```

На овој начин добиваме податоци од четири различни табели. За да го сместиме овој упит во поглед, потребно е само да ја додадеме наредбата CREATE VIEW AS пред упитот:

```
CREATE VIEW myView AS
```

```
SELECT customer.first_name, customer.last_name, address.address, city.city, country.country
```

```
FROM customer JOIN address
```

```
ON address.address_id = customer.address_id
```

```
JOIN city ON city.city_id = address.city_id
```

```
JOIN country ON country.country_id = city.country_id;
```

Доколку сега ја извршиме наредбата:

```
SELECT * FROM myView;
```

Како резултат ќе добиеме виртуелна табела со пет колони: first_name, last_name, address, city и country, која би содржела далеку покомплексна логика.

14.2. АЖУРИРАЊЕ НА ИЗВОРНИТЕ ПОДАТОЦИ

Погледите, освен можноста за прикажување на актуелната содржина по зададената форма, имаат и можност за ажурирање на изворот на кој се формирани, но при тоа мора да се испочитуваат некои правила. Овие правила се однесуваат на креирањето на SELECT упитот при дефинирањето на погледот.

Ова значи дека при креирањето на погледот при пишувањето на SELECT упитот мора да се испочитуваат следните правила:

- SELECT упитот не смее да ги содржи клучните зборчиња GROUP BY, DISTINCT, LIMIT, UNION или HAVING;
- Погледите кои групираат податоци од повеќе табели најчесто неможат да се користат за измена на конкретни податоци, т.е. SELECT упитот мора да управува само со една табела;
- Погледот мора да ги содржи сите колони од одредена табела кои се дефинирани како примарни или надворешни клучеви.

DDL скриптата на погледот може да ја погледнеме со наредбата:

SHOW CREATE VIEW myView;

14.3. ПРАВА

Погледите се тесно поврзани со безбедноста и правата на корисниците. При креирањето на погледот, креаторот на погледот мора да има права над табелите над кои ќе се креира погледот. Од друга страна, корисникот не мора да има SELECT права над табелите, се додека има права над погледот.

Ќе креираме еден корисник:

CREATE USER John;

Забелешка: Креираме корисник без шифра и домен, само за вежба. Во било кој друг случај ова би било лоша пракса и потребно е да се избегнува ваков начин на работа во било кое продукциско (вистинско, реално) опкружување.

Понатаму, на корисникот John ќе му доделиме SELECT права над погледот myView:

GRANT SELECT ON myView TO John;

Ако се логираме сега на серверот како John, ќе имаме право да ја извршиме следната наредба:

```
SELECT * FROM myView;
```

Притоа, нема да имаме право да ги извршиме следните наредби:

```
SELECT * FROM customers;
```

или

```
SELECT * FROM address;
```

Ваквиот пристап е добар доколку сакаме да заштитиме податоци од одредена група на корисници, а од друга страна да им овозможиме пристап до други податоци.

Исто така, потребно е да ги согледаме погледите и од перспектива на перформансите. Генерално, погледите скоро секогаш треба да се избегнуваат ако постои можност да се случи некоја од компликациите кои доведуваат до намалување на перформансите или до негативно влијание на работата на серверот. Да речеме, никако не е добро да се креира поглед кој е составен од друг поглед или да се користи поглед само за да не би пишувале некој сложен упит. Потребно е да ги разбереме погледите и да ги користиме, како што веќе рековме, во ситуации кога на некој корисник му даваме само дел од правата и дел од податоците, а до останатите не сакаме да му дозволиме пристап. Ова, секако може да се постигне и без користеше на погледи, преку други програмски јазици. На погледите ќе гледаме како на алатки за организација, кои ни овозможуваат да го класифицираме приказот на податоците (во некои случаи и други манипулации) во зависност од правата на корисникот.

15. СКЛАДИРАНИ РУТИНИ

Складираните рутини се процедури и функции кои претставуваат сетови од SQL наредби кои се сместени на серверот. Кога се користат складираните рутини, не е

потребно клиентот да повторува одредени наредби повеќе пати, туку едноставно може да ги повикува рутините кои се складирани на серверот.

Складираните рутини бараат постоење на `proc` табели во системските `sql` бази на податоци. Оваа табела се креира автоматски при инсталацијата на MySQL серверот.

Складираните рутини може да бидат особено корисни во следните ситуации:

- Кога постојат повеќе клиентски апликации напишани во различни програмски јазици кои комуницираат со иста база на податоци;
- Кога сигурноста е императив; на пример, во банкарските системи складираните процедури и функции се користат за сите операции; ова овозможува конзистентно и сигурно опкружување, а користењето на рутините овозможува секоја операција да биде адекватно логирана; во таквите сценарија, апликациите и корисниците немаат директен пристап до табелите во базата на податоци, туку само можат да извршуваат специфични складирани процедури.

Складираните рутини се блокови од SQL код кои се во состојба да извршат некоја активност на серверот. Овие блокови најчесто се апстрактни и прифаќаат параметри, така што може да се употребуваат повеќе пати и во различни ситуации.

Целта на складираните рутини е автоматизација на кодот и брзина при ракувањето со податоците. Што се однесува до автоматизацијата, веројатно е прилично јасно, бидејќи складираните рутини се многу слични на функциите од било кој модерен програмски јазик.

15.1. STORED PROCEDURES VS. USER-DEFINED FUNCTIONS

Во овој дел ќе ги разгледаме главните разлики меѓу складираните процедури (stored procedures) и кориснички дефинираните функции (user-defined functions, UDF). Во оваа прилика велиме кориснички дефинирани функции, бидејќи за

разлика од функциите со кои ќе се запознаеме сега, постојат и функции кои во MySQL се автоматски достапни, како на пример функцијата `now()`, за добивање на моменталното време.

Во следната табела се прикажани основните разлики меѓу процедури и функции:

Stored procedures	User-defined functions
Може да вратат null или произволен број на вредности	Може да вратат само една вредност
Може да имаат влезни и излезни параметри	Имаат само влезни параметри
Може да се користат SELECT и DML упити	Може да се користи само SELECT упит
Со процедурите може да бидат повикани функции	Од функциите не може да бидат повикани процедури
Може да се обработуваат исклучоци, користејќи try-catch блокови	Не може да се користат try-catch блокови
Не може да се користат во SQL наредби	Може да се користат во SQL наредби
Поддржуваат трансакции	Не поддржуваат трансакции

15.2. СКЛАДИРАНИ ПРОЦЕДУРИ

15.2.1. КРЕИРАЊЕ НА СКЛАДИРАНИ ПРОЦЕДУРИ

Складираната (stored) процедура е објект на серверот, како и секој друг. За нејзино креирање, ќе ја користиме DDL наредбата CREATE со следната синтакса:

CREATE

[DEFINER = { user | CURRENT_USER }]

PROCEDURE sp_name ([proc_parameter[,...]])

[characteristic ...] routine_body

Синтаксата за креирање на една едноставна складирана процедура би изгледала вака:

```
DELIMITER//
```

```
CREATE PROCEDURE my_procedure()
```

```
BEGIN
```

```
SELECT 'hello from stored procedure';
```

```
END//
```

```
DELIMITER;
```

Како што може да се види од примерот, креацијата на складираната процедура содржи уште некои особености (наредбата DELIMITER). Оваа наредба всушност и не спаѓа во дефинициониот дел на процедурата, туку се користи за заобиколување на следниот проблем:

Основниот delimiter во MySQL е ознаката ;. Тоа значи дека секоја линија код, доколку сакаме да бидат повеќе, мора да завршува со таа ознака. Со оглед дека складираните рутини во базата се зачувуваат како текст и обично поседуваат повеќе од една линија код, потребно е некако да бидат сместени со delimiter ;.

Проблемот настанува кога MySQL при креирањето на складирана процедура наиде на оваа ознака и наместо да ја третира како текст на рутината, тој, едноставно го третира како SQL симбол.

За да го избегнеме тоа, ја користиме SQL наредбата DELIMITER, која овозможува дефинирање на подразбирливиот delimiter на базата. Во примерот, наместо ознаката ; ставено е ознака //, што значи дека секоја линија која на крајот ја содржи оваа ознака, ќе биде третирана како SQL наредба.

На крај, ја завршуваме дефиницијата на процедурата со ознаката END// (// заменува ;) и, конечно, го враќаме delimiter-от на старата вредност со наредбата DELIMITER;

Доколку би сакале складираната процедура да ја креираме преку MySQL Workbench ја избираме опцијата Create Stored Procedure од контекстуалното мени кое се добива со десен клик на Stored Procedures.



На овој начин во прозорецот за внес на упитот ќе добиеме веќе готова синтакса за креирање на складирана процедура:

```
DELIMITER $$
```

```
CREATE PROCEDURE new_procedure()
```

```
BEGIN
```

```
END
```

Во претходно наведената синтакса може да забележиме и некои опционални параметри кои може да бидат наведени при дефинирањето на складираната процедура.

Првото клучно зборче е DEFINER; кое се користи за дефинирање на корисникот кој ја креира складираната процедура.

Ова е важно бидејќи, подразбирливо е дека процедурата ќе може да биде ажурирана или избришана само од страна на корисникот кој ја има креирано. Затоа можеме, при самото креирање, експлицитно да го нагласиме креаторот на процедурата:

```
DELIMITER//
```

```
CREATE DEFINER = John PROCEDURE my_procedure()
```

BEGIN

SELECT * FROM customer;

END//

DELIMITER;

Процедурата може да биде изградена со некои структурни параметри (CONTAINS SQL, NO SQL, READS SQL DATA, MODIFIES SQL DATA) кои ја опишуваат природата на процедурата, но немаат синтаксичко влијание на нашето ракување, туку на самиот MySQL му помагаат при креирањето на статистиката и оптимизацијата:

CREATE PROCEDURE my_procedure()

MODIFIES QSL DATA

BEGIN

// procedure body

END;

Слична цел има и параметарот DETERMINISTIC/NON DETERMINISTIC. Овој параметар исто така значајно влијае на работата на оптимизациониот Engine, но нема некое влијание врз синтаксата на процедурата. Доколку процедурата е означена како deterministic, значи дека нејзиниот излез никогаш не се менува, како во првата процедура која ја напишавме, која емитира порака “Hello from stored procedure”. Ако процедурата не е детерминистичка (што е подразбирливиот параметар), MySQL не очекува дека резултатот ќе биде секогаш ист (на пример, ако процедурата содржи функција now(), која прикажува точно време и, секако, никогаш не дава ист резултат).

15.2.2. АКТИВИРАЊЕ НА ПРОЦЕДУРА

Активација на процедурата се врши со наредбата CALL:

CALL my_procedure;

или

CALL my_procedure();

Доколку извршиме една од овие две наредби, ќе добиеме резултат извршување на блокот во процедурата my_procedure.

15.2.3. АЖУРИРАЊЕ НА ПРОЦЕДУРА

Од примерот за креирање на процедура гледаме дека телото на процедурата започнува со клучното зборче BEGIN, а завршува со END со delimiter. Ова правило важи само при креирањето на процедурата, но не и при нејзино ажурирање (ALTER):

ALTER PROCEDURE proc_name [characteristic ...];

Доколку сакаме да го ажурираме телото на процедурата, наредбата ALTER не е употреблива, туку единствениот начин е бришење на процедурата и повторно креирање. Што се однесува до наредбата ALTER, со неа може да се ажурира само структурата на процедурата, но не и нејзиното тело. Може да се каже дека наредбата ALTER на процедурите нема некоја особена вредност.

Кога еднаш ќе ја креираме складираната процедура, таа функционира само на ниво на базата во која е креирана, и тоа само за корисниците кои имаат право на нејзино користење (grant execute on procedure my_procedure to John). За овие права важат истите правила како и кај погледите. Корисникот не мора да има права над податоците за да би можел да има права над складирана процедура и преку неа да ги види податоците. Треба да бидеме внимателни при ажурирање на телото на процедурата, кога ги користиме наредбите DROP и CREATE. Со наредбата DROP се губат сите кориснички права над процедурата, дури и ако новата процедура има исто име како старата.

15.2.4. БРИШЕЊЕ НА ПРОЦЕДУРА

Синтаксата за бришење на складирана процедура е следна:

DROP PROCEDURE [IF EXISTS] sp_name

Ги користиме клучните зборчиња DROP PROCEDURE по кои се наведува името на процедурата која сакаме да ја избришеме. Како опционален параметар може да се наведат и клучните зборчиња IF EXISTS, кои ќе спречат појава на грешка во случај процедурата да не постои.

За бришење на складирана процедура потребни се Alter routine привилегии.

15.2.5. ПАРАМЕТРИЗАЦИЈА НА ПРОЦЕДУРИ

Складираната процедура има можност за прифаќање и проследување на параметри, при што може да се користат три вида на параметри: IN, OUT и INOUT.

proc_parameter:

[IN | OUT | INOUT] param_name type

Самите нивни називи се доста сликовити:

- IN ги подразбира влезните параметри, кои по прифаќањето се видливи само во телото на процедурата;
- OUT се креира во процедурата, но може да биде видлив и надвор од неа;
- INOUT се креира надвор од процедурата и е видлив преку целото време, и во процедурата и надвор од неа.

IN е подразбирливиот правец на движење на параметрите. Тоа значи дека не мораме експлицитно да го нагласуваме тоа негово својство. Намерно велиме правец на движење, а не тип, бидејќи параметрите исто така поседуваат типови (што не е исто со правецот на движење). Невозможно е да се проследи параметарот на процедурата доколку во нејзината дефиниција не постои дефиниран и неговиот очекуван тип.

Пример за параметризирана процедура:

```
CREATE PROCEDURE my_procedure (param1 int, param2 int)
```

```
    SELECT param1+param2;
```

Оваа процедура собира два броја. За да се повика оваа складирана процедура, се користи клучното зборче CALL; при проследувањето, потребно е, секако, да се проследат параметрите кои се бараат. Па повикот ќе изгледа вака:

```
CALL my_procedure (1,2);
```

Овие броеви процедурата ги прифаќа преку параметрите param1 и param2, а потоа го третира преку кодот. Забележуваме дека параметрите ја немаат стандардната нотација (со ознаката @), туку се проследуваат само по називот. Сепак, можни се комбинации на нотациите на параметрите во самата процедура:

```
SET @x=5;
```

```
SELECT param1+param2+@x;
```

Процедурата како резултат ќе го прикаже бројот 8, бидејќи влезните параметри и параметрите во процедурата се третирани идентично.

Променливите кои се наоѓаат во процедурата не се видливи надвор од неа. Значи, променливата @x ќе биде видлива само во BEGIN-END блокот.

Доколку би сакале параметрите од процедурата да ги користиме и надвор од неа, потребни ни се излезни (OUT) и излезно-влезни (INOUT) параметри.

Излезните параметри бараат параметарот да биде дефиниран пред повикот на процедурата:

```
CREATE PROCEDURE my_procedure (OUT param varchar (20))
```

```
    SET param='hello from procedure';
```


Сега параметарот е проследен преку клучното зборче OUT (што значи дека параметарот е излезен). Понатаму во кодот овој параметар се третира стандардно.

При повикувањето на процедурата, не мора да се проследува параметарот кој има вистинска вредност, туку може да го проследиме било кој параметар (па дури и Null), кој не мора да биде ни претходно дефиниран:

```
CALL my_procedure (@p);
```

```
SELECT @p;
```

По извршувањето на процедурата, вредноста на променливата p ќе биде изменета во склад со самата процедура.

Доколку ја креираме процедурата my_procedure на следниот начин:

```
CREATE PROCEDURE my_procedure (OUT param int)
```

```
SET param=param+1;
```

И ја повикаме на следниот начин:

```
SET @a=1;
```

```
CALL mp (@a);
```

```
SELECT @a;
```

Резултатот од упитот ќе биде Null, бидејќи OUT параметарот не е третиран како покажувач, туку едноставно само како параметар кој ќе биде вратен на системот. За да биде третиран параметарот како покажувач, треба да употребиме INOUT покажувач:

```
CREATE PROCEDURE my_procedure (INOUT param int)
```

```
SET param=param+1;
```

Со оваа измена, параметарот ќе биде третиран како покажувач и измените на параметарот (променливата) во текот на процедурата ќе бидат актуелни за целиот код во кој се повикува процедурата.

Сите структурални податоци за процедурата може да се добијат преку наредбата:

SHOW CREATE PROCEDURE my_procedure

А за сите процедури:

SHOW PROCEDURE STATUS

5.3. КОРИСНИЧКИ ДЕФИНИРАНИ ФУНКЦИИ (UDF)

Сите функции може да се поделат на два вида:

- Кориснички дефинирани функции и
- Вградени функции.

Вградените функции се сите функции кои ги подразбира стандардната форма на MySQL серверот, додека кориснички дефинирани се оние кои се креирани дополнително, од страна на корисникот.

Кога станува збор за кориснички дефинираните функции, на MySQL серверот тие може да бидат реализирани на два начина: преку SQL скрипта (DDL) или преки изворниот програмски јазик на MySQL серверот (C).

Креирањето на кориснички дефинирани функции (UDF) преку SQL е едноставен метод. Всушност, синтаксата е многу слична со синтаксата за креирање на процедури:

CREATE FUNCTION myFunction() RETURNS varchar(20) RETURN 'hello from UDF';

Ќе ја разгледаме оваа функција чекор по чекор:

CREATE FUNCTION myFunction()

Оваа линија е сосема идентична како при креирањето на процедура.

RETURNS varchar(20)

За разлика од процедурата, функцијата може (и мора) да врати само една вредност. Типот на оваа вредност мора да биде дефиниран при дефинирањето на самата функција. Во овој случај тоа е varchar(20).

RETURN 'hello from UDF';

Функцијата има само една линија код. Оваа линија го враќа резултатот од функцијата, а со самото тоа го прекинува и нејзиното извршување. Доколку функцијата би поседувала повеќе линии код, ќе треба да прибегнеме кон техниката која ја употребивме и кај процедурите – привремена замена на delimiter-от.

DELIMITER //

CREATE FUNCTION my_function()

RETURNS varchar(20)

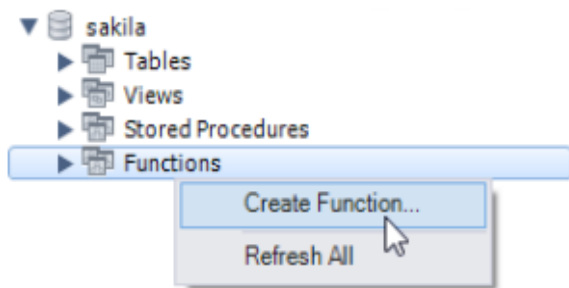
BEGIN

RETURN 'Hello';

END //

DELIMITER;

Креирањето на кориснички дефинирана функција, секако, може да се изведе и од MySQL Workbench. Доволно е од контекстуалното мени кое се добива со десен клик на Functions да се избеере Create Function...



За разлика од процедурите, функциите може да бидат вметнати и во самиот упит, што ни дава на располагање големи можности за интервенција над податоците во моментот на креирање на излезот. На пример, функцијата ја повикуваме со нејзино вметнување во упит:

```
SELECT my_function()
```

Повторно како пример ќе ја земеме базата на податоци Sakila. Доколку би сакале да утврдиме дали одреден филм е подолг од два часа, би можеле да користиме функција:

```
DELIMITER $$
```

```
CREATE DEFINER='root'@'localhost' FUNCTION
```

```
    movie_over_2_hours(film_title varchar(50)) RETURNS tinyint(1)
```

```
    DETERMINISTIC
```

```
BEGIN
```

```
    DECLARE film_duration INT;
```

```
    SELECT length INTO film_duration
```

```
    FROM film
```

```
    WHERE title = film_title;
```

```
    If film_duration > 120 then return TRUE; end if;
```

```
    If film_duration < 120 then return FALSE; end if;
```

END

Креираваме кориснички дефинирана функција со назив `movie_over_2_hours` и дефинираваме `DEFINER`. Тоа не е неопходно. Оваа функција има еден влезен параметар и тоа е називот на филмот чие траење сакаме да го провериме. Овој параметар има назив `film_title` од тип `varchar`. Како излезен параметар функцијата има тип `tinyint`.

На почетокот од телото на функцијата декларирана е една променлива во која ќе ја сместиме должината на филмот чиј назив е проследен на функцијата. Овој параметар има назив `film_duration` ОД ТИП `int`. За добивање на должината на филмот ќе го користиме стандардниот `SELECT` упит, со тоа што неговиот резултат ќе го сместиме во променливата `film_duration`. Понатаму користиме условна синтакса за да ја утврдиме дали должината на филмот е помала или поголема од 120 минути.

Од овој пример може да забележиме дека процедурите и функциите се многу слични по природа. Нивните основни разлики се во бројот на излезни параметри (функцијата може да врати еден, а процедурата повеќе) и во начинот на повикување (функцијата може да се повика од упит).

Освен складираните, постои уште еден вид на кориснички дефинирани функции: тоа се функциите напишани со изворниот јазик (C). Овие функции се многу побрзи од складираните, но нивното креирање е далеку покомплицирано, така што тие не се креираат без некоја позначајна потреба.

5.4. КОНТРОЛА НА ТЕКОТ

Користењето на јазичните конструктори е можно само во складираните рутини, т.е. во процедурите и функциите.

If

Во претходниот пример, користевме синтакса:

```
if film_duration > 120 then return TRUE; end if;
```

Ова е синтакса за една од основните наредби за контрола на текот и симбол во програмирањето воопшто. Контролниот оператор `if` , едноставно, му навестува на системот дека во наредниот тек на програмата нешто ќе се изврши условно. Во примерот рековме: Ако должината на филмот е поголема од 120 минути, тогаш целокупната функција ќе врати вредност `TRUE`.

Еве како би ја провериле вредноста на некоја променлива и на основа на тоа би извршиле некоја операција:

```
if @myVariable > 0 then  
  
    set @myVariable=@myVariable-1;  
  
    else if @myVariable < 0 then  
  
        set @myVariable=@myVariable+1;  
  
    else  
  
        set @myVariable=0;  
  
    end if;
```

На крај, во секој случај, блокот мора да го затвориме со наредбата `end if;`.

Еден од начините за контрола на текот е и `switch-case` структурата.

While

Уште еден начин за контрола на текот на програмата се јамките. Најчесто користени јамки во програмирањето се `while` и `for` (и нивните деривати).

Јамката `while` започнува со клучното зборче `while`, по што следи условниот израз и клучното зборче `do`.

Се што се наоѓа пред do ќе биде извршено при секоја итерација на јамката. Кога условот на јамката веќе нема да биде исполнет, таа завршува со исказот end while;.

```
SET @x=10;
```

```
while @x > 0 do  
  
    set @x=@x-1;  
  
    select @x;  
  
end while;
```

While јамката е карактеристична по тоа што се користи тогаш кога не знаеме точно колку итерации ќе бидат потребни за извршување на нејзиниот блок. Со оглед дека условот за завршување на јамката обично се регулира во самата јамка, многу е важно да не ја доведеме јамката во ситуација никогаш да не заврши (мртва јамка). Во примерот, ова би се случило доколку вредноста на x не ја намалуваме во секоја итерација.

Loop

Loop не се разликува многу од while јамката. На почетокот, потребно е да се иницијализира јамката со доделување на назив (my_loop: loop). Потоа, блокот на код започнува и трае се до наредбата end loop (end loop my_loop;). Јамката се извршува се додека не биде прекината со наредбата leave (leave my_loop;).

```
SET @x=10;
```

```
my_loop: loop  
  
    set @x=@x-1;  
  
    select @x;  
  
    if @x<=0 then
```

```
        leave my_loop;

    end if;

end loop my_loop;
```

Repeat

Repeat е јамка која се извршува се додека не биде исполнет одреден услов. Карактеристично за неа е тоа што дури условот и никогаш да не биде исполнет, телото на јамката ќе се изврши барем еднаш. Од другите програмски јазици, оваа јамка ја познаваме како `do while` јамка.

Repeat започнува со клучното зборче `repeat`, по што следи телото на јамката. Телото завршува со наредбата `until`, по која се наведуваат условот и наредбата (`until @x = 0 end repeat`):

```
repeat

    set @x=@x-1;

    select @x;

until @x = 0 end repeat;
```

Iterate и leave

Овие две наредби овозможуваат преминување на следната итерација или безусловно напуштање на јамката. Како параметар го прифаќаат името на јамката:

`iterate my_loop;` -Преминува на следната итерација

`leave my_loop;` -Ја напушта јамката

For each

Оваа јамка извршува итерација преку табеларна структура, овозможувајќи одредена акција на секој податок посебно и таа важи само за тригерите:

For each row set @x = @x + 1;

Вежба 19: Да се креира процедура за кориснички внес, со назив `usp_insertuser`, која како параметар го прифаќа името на корисникот. Корисникот ќе биде внесен со статус `user`.

Решение:

```
create PROCEDURE insertuser (newname varchar (50))
BEGIN
insert into users (name, status) values (newname,1);
END;
```

Вежба 20: Да се креира процедура која креира корисник според параметризирано име. Доколку во табелата веќе постои корисник со тоа име, тој нема да биде креиран. Корисникот се внесува со статус `user`.

Решение:

```
create PROCEDURE insertuser (newname varchar (50))
BEGIN
declare usersCount int;
select count(id) from users where name = newname into usersCount;
if usersCount < 1 then
insert into users (name, status) values (newname,1);
end if;
END;
```

Вежба 21: Да се креира процедура која ќе внесува корисник во база. Како параметри процедурата прифаќа име и шифра. Доколку корисник со тоа име не постои, ќе се внесе, во спротивно, шифрата ќе му биде заменета со нова. Корисникот се внесува со статус `user`.

Решение:

```
Create PROCEDURE insertuser (newname varchar (50), newpass varchar(50))
BEGIN
declare usersCount int;
select count (id) from users where name = newname into usersCount;
```

```
if usersCount < 1 then
insert into users (name, password, status) values (newname, password,1);
else
update users set password = newpass where name = newname;
end if;
END;
```

6. ВГРАДЕНИ ФУНКЦИИ

Функционалностите кои досега ги разгледувавме се кориснички дефинирани. Освен овие функционалности, постојат и т.н. вградени функционалности, т.е. (build in) функции. Ако погледнеме наназад ќе видиме и дека досега доста пати ги имаме употребувано. Од тој аспект постојат два вида на функции: скаларни и агрегатни.

6.1. СКАЛАРНИ ФУНКЦИИ

Скаларните функции се функции кои главно примаат една вредност како параметар и враќаат една вредност како излез.

```
SELECT trim (' scalar function ');
```

Резултатот од оваа функција ќе биде стринг со отстранети празни места од почетокот и од крајот. Функцијата како параметар прифаќа една вредност, а тоа е почетниот стринг и враќа обработен стринг, што исто така е една вредност.

6.2. АГРЕГАТНИ ФУНКЦИИ

Агрегатните функции управуваат со повеќе вредности, но враќаат една (најчесто нумеричка) вредност како резултат:

```
SELECT count (*) FROM customer
```

Карактеристична агрегатна функција е функцијата count, прикажана во примерот погоре. Таа како параметар прима ѕвездичка, која ги симболизира сите колони од некоја табела, како што знаеме.

Забелешка: Овие две функции овде се претставени само демонстративно, со цел разбирање на разликата меѓу скаларни и агрегатни функции.

Освен оваа поделба, агрегатните функции се делат на пет категории:

1. Функции за управување со датумот;
2. Функции за управување со нумерички вредности;
3. Функции за управување со стрингови;
4. Сумарни функции;
5. Контролни функции.

17. КУРСОРИ И ТРИГЕРИ

Тригерите се кориснички дефинирани блокови код кои се извршуваат во моментот на интервенција на табелите (INSERT, UPDATE, DELETE) и овозможуваат манипулација со податоците пред вистинското ажурирање да биде зачувано. Тие се делат на три категории (INSERT, UPDATE и DELETE) и две подкатегории (BEFORE и AFTER) со чија зависност нудат одредени опции.

Еден тригер е објект во една база на податоци и важи само за таа база. Доколку базата на податоци се избрише, се бришаат и сите нејзини тригери.

При креирањето на тригер, го поврзуваме со одредена табела и акција која сакаме да ја следиме. Притоа, не може да креираме повеќе истоимени тригери на една табела, ниту да поставиме исти типови на тригери на ист настан во една табела (на пример, два BEFORE INSERT тригера на една табела).

```
CREATE TABLE mytable
```

```
(
```

```
id int primary key auto_increment,
```

```
name varchar (50)
```

```
)
```

7.1. BEFORE ТРИГЕРИ

Карактеристика на овие тригери е тоа што настаните се случуваат пред податокот да дојде до табелата. Тоа ни овозможува да го пресретнеме податокот и да извршиме евентуална интервенција над него. За таа цел, во моментот на внес во меморијата се креира табела чија структура е иста како и структурата на целната табела, со таа разлика што единствените податоци во оваа табела се податоците кои ги внесуваме или ажурираме:

```
CREATE TRIGGER checkName BEFORE INSERT ON mytable
```

```
FOR EACH ROW SET new.name = concat (new.name, “_test”);
```

Во примерот, првата линија ја претставува дефиницијата на тригерот (CREATE TRIGGER checkName), со назнака дека станува збор за BEFORE тригер за INSERT наредба (BEFORE INSERT), кој се наоѓа на табелата mytable.

Потоа започнува блокот код; со оглед дека блокот содржи една линија код, едноставно го надоврзуваме на дефиницијата. Доколку блокот содржи повеќе од една линија код, ќе ни бидат потребни begin и end ознаки.

Самиот блок содржи итератор (FOR EACH ROW) кој означува дека следната акција (или блок) ќе биде извршувана се додека има редови во табелата. Називот на табелата new, на која овде се интервенира, претставува споменатата табела во меморијата која ја креира серверот, односно тригер. Оваа табела секогаш го носи ова име, со чија помош ја поставуваме референцата.

Бидејќи сакаме да внесеме податок во табелата, серверот ни дава можност прво да го провериме и евентуално да го ажурираме тој податок во табелата new. По завршувањето на измените на табелата new, односно, кога тригерот ќе го заврши своето извршување, податоците од табелата new ќе се сместат во вистинската табела (mytable).

Погоре прикажаниот пример, на секое внесено име додава наставка _test.

Во телото на тригерот може да се најде и покомплексна логика. Би можеле, на пример, да ја провериме должината на внесеното име и доколку името е пократко од 5 карактери, да извршиме кодирање на името:

DELIMITER //

```
CREATE TRIGGER checkName BEFORE INSERT ON mytable
```

```
FOR EACH ROW
```

```
BEGIN
```

```
if length (new.name) < 5 then
```

```
set new.name = concat (new.name, “_test”);
```

```
END IF;
```

```
END//
```

DELIMITER;

Како и при останатите повеќелински дефиниции на функционалност и кај тригерите користиме привремена измена delimiter.

Бидејќи станува збор за наредбата INSERT, во меморијата се наоѓа само една табела – new. Доколку тригерот е поставен на наредба UPDATE, во меморијата ќе се наоѓа и табела – old, додека во случај на DELETE тригер, не постои табела new, туку само табела old.

```
DROP TRIGGER checkName;

DELIMITER //

CREATE TRIGGER checkName BEFORE UPDATE ON mytable

    FOR EACH ROW

        BEGIN

            if length (old.name) < 4 then

                set new.name = password (new.name);

            END IF;

        END//

DELIMITER;
```

7.2. AFTER ТРИГЕРИ

Овие тригери се активираат по внесот на податоците. Значи, кога се наоѓаме во кодот на тригерот, ќе имаме пристап само до веќе внесени податоци.

After тригерите се корисни доколку сакаме паралелно со главната табела да ажурираме и некоја архивирана, log или backup табела:

```
DROP TRIGGER checkName;

DELIMITER //

CREATE TRIGGER checkName AFTER INSERT ON mytable

    FOR EACH ROW

        BEGIN

            insert into backup_table values (new.name)
```

END IF;

END//

DELIMITER;

Кога еднаш ќе го поставиме тригерот на табелата, тогаш тој е дел од самата табела во контекст на нејзините трансакции. Тоа значи дека при ажурирањето на податоците, во предвид се зема и извршувањето на тригерот и дека акцијата ќе се смета за неуспешна доколку дојде до грешка при неговата активација.

Доколку дојде до грешка при активацијата на BEFORE тригерот, нема да се случи ни активација на AFTER тригерот (доколку се наоѓаат на иста наредба во иста табела).

7.3. КУРСОРИ

Курсорите се посебен тип на податоци кои овозможуваат секвенцијално управување со податоците добиени со одреден упит. Иако велиме *типови податоци*, управувањето со курсори не е толку едноставно како управувањето со прости типови на податоци. Тие имаат табеларна структура и бараат специфичен сет на наредби. Курсорите се користат за да се итерира низ голема колекција на податоци. Некои особености на MySQL курсорите се:

- Со курсорот не може да се врши ажурирање на податоци, туку само читање;
- Низ редовите може да се преминува само во една насока и тоа онака како што е дефинирано со SELECT упитот;
- Курсорите може да се користат само во складираните рутини (процедури и кориснички дефинирани функции) и тригери.

Синтаксата за креирање на курсор во процедура е следна:

DELIMITER \$\$

```
CREATE DEFINER = 'root' @ 'localhost' PROCEDURE 'film_cursor'()

BEGIN

DECLARE sum float DEFAULT 0;

DECLARE counter int DEFAULT 0;

DECLARE a float;

DECLARE e int DEFAULT 0;

DECLARE c CURSOR FOR SELECT length FROM film;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET e = 1;

OPEN c;

Repeat

FETCH c INTO a;

IF NULL ISNULL (a) THEN

SET sum = sum + a;

END IF;

SET counter = counter + 1;

UNTIL e END repeat;

CLOSE c;

SELECT sum/counter;

END
```

На почетокот на процедурата декларираме неколку променливи со клучното зборче DECLARE. Променливата sum служи за зачувување на збирот на должините на филмовите. Променливата counter служи за чување на вкупниот

број на филмови во базата. Овие два податока ќе ни бидат потребни за пресметување на просечната должина на филмовите.

Следната променлива е променливата `a`, која служи за вчитување на вредноста од колоната `length`.

За дефинирање на курсорот потребно ни е неговото име и табеларните податоци со кои ќе манипулира.

DECLARE `c` CURSOR FOR SELECT `length` FROM `film`;

Бидејќи користиме наредба `DECLARE` и се наоѓаме во процедура, променливата може да ја дефинираме само со нејзиното име. Пракса е името на променливата која го носи курсорот да биде `c`, иако може да се употреби и било кој друг назив.

Откако ќе го дефинираме курсорот, за да можеме да работиме со податоците, треба истиот и да го отвориме. Откако ќе го отвориме курсорот, тој останува отворен се додека експлицитно не го затвориме. Курсорот се отвара со наредбата `open`, а се затвара со наредбата `close`:

OPEN `c`;

CLOSE `c`;

За контрола на позицијата при итерирањето, потребен ни е маркер:

DECLARE CONTINUE HANDLER FOR NOT FOUND SET `e` = 1;

Маркерот (`e`) потребно е претходно да биде дефиниран со тип и вредност:

DECLARE `e` int DEFAULT 0;

Променливата која ќе го претставува маркерот, мора да биде дефинирана пред самиот курсор.

Секој ред од курсорот се чита со наредбата `fetch`:

FETCH `c` INTO `a`;

Откако ќе ги прочита сите редови, маркерот ќе ја прикаже вредноста која му ја имаме дефинирано на почетокот.

Сето ова е треба да се изврши во една јамка. Логиката во курсорот извршува собирање на должините на сите филмови и таа вредност ја чува во променливата `sum`. Преку променливата `counter` се врши зголемување на вредноста, секој пат за еден. На крај со наредбата `SELECT` се врши прикажување на резултатот.

Овој резултат може да се добие на многу поедноставен начин, со употреба на функцијата `AVG`.

```
SELECT AVG(length) FROM film;
```

Поради тоа, мислењата на MySQL стручњаци се поделени во врска со употребата на курсорите. Некои се залагаат за нивната употреба, додека други сметаат дека употребата на курсорите е непотребно комплицирање на логиката.

18. ТРАНСАКЦИИ

Трансакциите овозможуваат една или повеќе SQL команди да се енкапсулираат во едноставна операција, односно целина. Тие претставуваат важна компонента кога станува збор за интегритетот на базата на податоци. Трансакциите се користат и за оневозможување на измени на еден ист податок од страна на два различни корисници во ист момент. Трансакциите имаат 4 фундаментални својства:

- **Atomicity** (атомичност или неделивост) – овој поим означува дека трансакцијата е неделива, т.е. дека може или да се изврши или да не се изврши; половично извршување не е можно.
- **Consistency** (конзистентност) – овој поим означува дека по извршувањето на трансакцијата базата ќе остане во конзистентна состојба; доколку трансакцијата предизвика некоја недозволена состојба или неадекватно ажурирање на податоците, се ќе биде вратено во почетната состојба.

- Isolation (изолација) - овој поим означува дека повеќе трансакции може да се извршуваат симултано, т.е. истовремено без негативни последици.
- Durability (издржливост) – поим кој се однесува на особината на трансакциите дека по нивното комплетно извршување веднаш ќе бидат зачувани, дури и доколку се случи отказ на системот и прекин на работата на базата на податоци.

18.1. КОНТРОЛА НА ТРАНСАКЦИИТЕ

По default, MySQL серверот се наоѓа во таканаречен autocommit mode. Ова значи дека секоја SQL команда се извршува во склоп на една мала трансакција. Затоа може да се каже дека наредбите се обвиваат трансакционо, т.е. или ќе бидат извршени или нема да бидат извршени и нема да бидат извршени половично. Значи, доколку, на пример ја извршуваме наредбата DELETE, ние всушност извршуваме една трансакција.

Во реалноста, најчесто се работи со трансакции кои се составени од неколку SQL наредби, и тоа на два начини. Првион начин подразбира исклучување на autocommit mode.

18.1.1. КОНТРОЛА AUTOCOMMIT MODE

За да го исклучиме autocommit mode, ја користиме системската променлива autocommit (системските променливи во SQL се означуваат со @@).Кога autocommit е подесен на 0 (исклучено), дури ни основните наредби нема да бидат извршени пред ние тоа да го одобриме. Со други зборови, кога autocommit mode е исклучен, сите SQL наредби се сметаат за една трансакција се додека трансакцијата не се потврди со клучното зборче COMMIT или се откаже со клучното зборче ROLLBACK.

Да погледнеме еден пример од базата на податоци Sakila и табелата country:

```
SET @@autocommit = 0;
```

```
INSERT INTO country (country) VALUES ('Macedonia');
```

```
INSERT INTO country (country) VALUES ('Croatia');
```

```
INSERT INTO country (country) VALUES ('Bulgaria');
```

```
ROLLBACK;
```

Доколку го извршиме овој упит и ја погледнеме табелата country, нема да ги видиме промените кои тукушто ги додадовме. Да го разгледаме упитот:

Во првата линија, со наредбата SET ја поставуваме вредноста на системската променлива autocommit на 0 и на тој начин го исклучуваме autocommit mode. Тоа практично значи дека поединечните SQL команди веќе не се сметаат за една трансакција, туку се чека на една од клаузулите COMMIT/ROLLBACK за да се заврши трансакцијата.

Со следните три линии вршиме класично додавање на податоци во табелата од базата. На крај со клучното зборче ROLLBACK го поништуваме ефектот од трансакцијата, што всушност е и причината за тоа што податоците во базата ги немаше. Доколку трансакцијата завршуваше со COMMIT, ефектот ќе беше видлив.

На овој начин се решава и проблемот на банкомат така што ќе извршиме бришење во базата, а потоа ќе ја активираме исплатата на банкоматот и дури потоа ќе го потврдиме целиот процес и ќе ја затвориме комплетната трансакција.

Сепак, постои и друг начин, можеби и подобар за постигнување на саканиот резултат.

18.1.2. START TRANSACTION И ROLLBACK/COMMIT

Ако ја разгледаме трансакцијата од претходниот пример, ќе забележиме дека таа нема почеток. Нејзиниот почеток, всушност е почетокот на скриптата. Се од почетокот на скриптата до наредбата COMMIT/ROLLBACK спаѓа под една трансакција. Кога се активира некоја од овие две наредби трансакцијата повторно

започнува, се до следната наредба COMMIT/ROLLBACK. Тоа е така затоа што овие трансакции се имплицитни.

За да биде една трансакција експлицитна, потребно е да го нагласиме нејзиниот почеток и крај со наредбите START TRANSACTION и COMMIT/ROLLBACK:

START TRANSACTION;

```
///sql dml
```

COMMIT/ROLLBACK

Ниту една манипулациона наредба нема да биде извршена во START/COMMIT блокот пред наредбата COMMIT. Намерно напоменуваме дека станува збор за манипулациони наредби, затоа што манипулациониот сет е практично единствениот сет кој може да се извршува трансакционо. Доколку во една трансакција сместиме DDL НАРЕДБА (на пример CREATE), таа би била извршена безусловно.

Така трансакцијата од претходниот пример која ја постигнавме со исклучување на autocommit mode, со променета синтакса за експлицитна трансакција, би изгледала вака:

START TRANSACTION;

```
INSERT INTO country (country) VALUES ('Macedonia');
```

```
INSERT INTO country (country) VALUES ('Croatia');
```

```
INSERT INTO country (country) VALUES ('Bulgaria');
```

ROLLBACK;

Секако, и овој пат нема ништо да се случи со податоците, т.е. нема да дојде до INSERT, бидејќи трансакцијата ја завршивме со ROLLBACK.

18.2. КОНТРОЛНИ ТОЧКИ

На трансакцијата може да и се постават и контролни точки и така да се контролира нејзиниот тек. Контролните точки се означуваат со клучното зборче `savepoint`. Претходниот пример би изгледал вака:

```
START TRANSACTION;
```

```
INSERT INTO country (country) VALUES ('Macedonia');
```

```
SAVEPOINT created_one_country;
```

```
INSERT INTO country (country) VALUES ('Croatia');
```

```
SAVEPOINT created_second_country;
```

```
INSERT INTO country (country) VALUES ('Bulgaria');
```

```
SAVEPOINT created_third_country;
```

```
ROLLBACK TO SAVEPOINT created_one_country;
```

Кога ќе го извршиме прикажаниот упит, во табелата `country` ќе биде инсерирана само државата во првиот запис, бидејќи трансакцијата ја завршивме со наредбата `ROLLBACK TO SAVEPOINT created_one_country`. Со оваа наредба ја завршуваме трансакцијата, така што сите наредби пред контролната точка се извршуваат и примаат, а они по контролната точка остануваат неизвршени.

Трансакциите се затвараат автоматски доколку дојде до извршување и на некоја од следните команди: `ALTER TABLE`, `CREATE INDEX`, `CREATE TABLE`, `DROP DATABASE`, `DROP TABLE`, `LOCK TABLES`, `RENAME TABLE`, `SET AUTOCOMMIT = 1`, `TRUNCATE`, `UNLOCK TABLES`. Од друга страна, трансакцијата не се затвара доколку дојде до обична грешка при извршувањето на упитите.

18.3. КОНКУРЕНЦИЈА

При конкурентното извршување на трансакциите може да настанат четири вида на проблеми:

1. Dirty Read / Uncommitted Read

Ова се случува доколку една трансакција пристапува на податоци кои се под друга трансакција. Постои реална можност овие податоци да не се точни, што може да доведе до компликација при работата и на едната и на другата трансакција.

2. Non-Repeteable Read / Inconsistent Analysis

Овој случај е сличен со претходниот. Се случува доколку една трансакција повторува едно исто читање, додека во меѓувреме друга трансакција извршува ажурирање на податоците, поради што доаѓа до неправилни податоци во првата трансакција.

3. Phantom Read

Се случува кога во текот на една трансакција се додаваат податоци од страна на друга трансакција. Тоа доведува до појава на неидентификувани редови, а со самото тоа и до неправилности во редот.

4. Lost Update

Кога две трансакции ажурираат податоци со UPDATE наредбата, податоците се ажурираат според последната извршена трансакција.

18.3.1. ЗАКЛУЧУВАЊЕ (LOCKS)

Наброените проблеми во врска со конкуренцијата се решаваат на различни начини во зависност од типот на табелата кој се користи. Најинтелигентен начин е употребата на трансакции, кои се подржани само од страна на табелите InnoDB.

Кај MyISAM табелите, за решавање на наброените проблеми, потребно е да употребиме техника на заклучување. Кај овој тип на табели може да се употреби само техниката на заклучување на ниво на цела табела, напосто речено, табелата која ќе биде заклучена, т.е. на која е применет lock, во текот на одреден период ќе биде достапна само за корисникот кој го има активирано lock-от. За тоа

време другите корисници нема да бидат во состојба да извршуваат било какви промени над податоците од заклучената табела, а во некои ситуации, во зависност од типот на заклучувањето, нема да бидат во состојба ни да читаат од податоците.

За да се резервира една табела за ексклузивно користење од страна на еден корисник, се користи синтаксата LOCK TABLE[S].

При повикување на оваа наредба, постојат неколку типови на заклучување кои може да се активираат:

- READ – сите корисници може да читаат податоци од заклучената табела, но никој не може да врши измени, вклучувајќи го и корисникот кој ја има активирано LOCK командата;
- READ LOCAL – исто како и претходно опишаниот READ, со таа разлика што може да се извршуваат неконфликтни INSERT упити;
- WRITE – само корисникот кој ја има активирано LOCK командата може да чита и врши измени на табелата, додека сите останати корисници се практично блокирани и не можат да вршат ни читање ни пишување;
- LOW PRIORITY WRITE – исто како и WRITE, со тоа што ваков LOCK не може да се активира се додека не завршат сите моментално активни LOCK-ови.

За отклучување на табелата се користи наредбата UNLOCK TABLE[S] без дополнителни параметри. Оваа команда ќе ги заврши сите активни LOCK-ови на моменталниот корисник.

При извршување на само една SQL команда не е потребно да се применува заклучување, бидејќи тие секако се атомични и се извршуваат на начин на кој не може да дојдат во конфликт со друга наредба.

18.3.2. ТРАНСАКЦИИ И ЗАКЛУЧУВАЊЕ

Веќе повеќе пати досега напоменавме дека една од главните предности во користењето на трансакциите е решавањето на проблемот со конкуренцијата, секако, доколку користиме InnoDB. Доколку делениот ресурс е заклучен со shared lock, сите трансакции може да го читаат, но само актуелната трансакција (онаа која го има активирано lock-от) може и да пишува (овие lock-ови се случуваат на ниво на редови и траат само толку колку што е потребно за редот да биде прочитан; карактеристични се за SELECT упитот).

Од друга страна, доколку дојде до активација на ексклузивниот lock, ниту една трансакција нема да може да му пристапи на ресурсот, се додека трансакцијата која го има активирано не заврши. Овој облик на заклучување настанува при INSERT, UPDATE и DELETE наредбите.

Да погледнеме на еден пример како функционира конкуренцијата меѓу две трансакции на InnoDB табела:

Konekcija A	Konekcija B	Trenutak
USE innotest INSERT INTO table1 VALUES (1, 10) SELECT * FROM table1 colA = 1 colB = 10	USE innotest	0
START TRANSACTION UPDATE table1 SET colB=11 WHERE colA=1		1
SELECT * FROM table1 colA = 1 colB = 11	SELECT * FROM table1 colA = 1 colB = 10	2
	BEGIN UPDATE table1 SET colB=colB+3 WHERE colA=1	3
COMMIT		4
SELECT * FROM table1 colA = 1 colB = 11	SELECT * FROM table1 colA = 1 colB = 14	5
	ROLLBACK	6
SELECT * FROM table1 colA = 1 colB = 11	SELECT * FROM table1 colA = 1 colB = 11	7

Во табелата се прикажани акциите кои се одвиваат во две конекции кои постојат паралелно. Во колоната за времето дадени се ознаки и лесно мже да се забележи

дека еден ред претставува една единица време. Уитите се претставени со карактеристичен фонт, додека резултатите се претставени со *italic* фонт.

1. На почетокот двете конекции селектираат иста табела за користење; конекцијата A внесува вредност во table1, и тоа вредностите 1 и 10 во колоните colA и colB, респективни; во конекцијата A се врши селектирање на инсертираните вредности.
2. Конекцијата A ја започнува трансакцијата; трансакцијата започнува со наредба за ажурирање на веќе внесените вредности, така што вредноста во колоната B наместо 10, ќе биде 11.
3. Следниот чекор на трансакцијата е селектирање на ажурираните вредности; во тој момент конекцијата B ги побарува вредностите од colA и colB; практично, овие SELECT наредби се извршуваат во ист момент; конекцијата A ги добива вредностите кои се ажурирани со претходната линија во трансакцијата, додека конекцијата B ги добива старите вредности.
4. Во овој момент конекцијата B ја започнува трансакцијата; во трансакцијата се врши ажурирање на colB така што нејзината вредност се зголемува за 3.
5. Во овој момент конекцијата A ја активира наредбата COMMIT, ккоја го означува крајот на трансакцијата.
6. Во овој момент двете конекции извршуваат SELECT наредба над колоните colA и colB, со тоа што конекцијата B тоа го извршува во трансакцијата; конекцијата A добива вредности кои се ажурирани со трансакцијата на оваа конекција, додека конекцијата B ги добива вредностите кои се под промена во трансакцијата која уште трае; при тоа во предвид е земена и промената која е направена со трансакцијата на конекцијата A, затворена со наредбата COMMIT, па вредноста на колоната B е $10+1+3=14$.
7. Во овој момент конекцијата B ја активира наредбата ROLLBACK и го поништува ефектот од трансакцијата.

8. На крај двете трансакции вршат SELECT над податоците и добиваат исти вредности.

Ваквото дефолтно однесување на InnoDB механизмот на конкуренција нема да ги задоволи сите потребни сценарија и ситуации кои може да се појават. Затоа постојат неколку специјални наредби за да се повлијае на однесувањето на заклучувањата.

SELECT...LOCK IN SHARE MODE

Дефолтното однесување на InnoDB табелите е SELECT командите да се извршуваат истиот момент, па дури и над блокирани ресурси. Едноставно, не се зема во предвид дека веќе постои отворена трансакција и дека на тој начин може да се преземат неадекватни податоци. Вакво однесување се појавува и во горната табела, и тоа во моментот 2, кога конекцијата B го селектира ресурсот кој е изменет со трансакцијата која постои во конекцијата A. Резултатот е различна вредност за колоната colB во однос на вредноста која ја добива конекцијата A.

Доколку SELECT наредбата се изврши со дополнителна синтакса LOCK IN SHARE MODE, тогаш SELECT наредбата пред извршувањето ќе чека се додека не се извршат сите трансакции кои се во тек, под услов да доаѓа до судир на ресурси, т.е. трансакцијата да манипулира со редовите кои наредбата SELECT ги побарува.

Доколку во моментот 2, наместо постоечката наредба, го извршевме упитот:

```
SELECT * FROM table1 LOCK IN SHARE MODE
```

Тогаш резултатот од овој упит би се прикажал дури по завршувањето на трансакцијата која постои во конекцијата A, т.е. во моментот 4.

SELECT... FOR UPDATE

Ова е уште една дополнителна наредба која може да се искористи за формулирање на SELECT упит. Таа се користи за поставување на ексклузивно

заклучување при користење на SELECT упит, исто како што тоа се случува подразбирливо при UPDATE упит. На овој начин би можеле ова да го искористиме во трансакцијата на конекцијата А и тогаш конекцијата В не би можела да ја добие вредноста од колоната colB веднаш, и покрај користењето на SELECT наредбата.

18.3.3. TIMEOUT И БРЗИНА НА ПРИСТАП

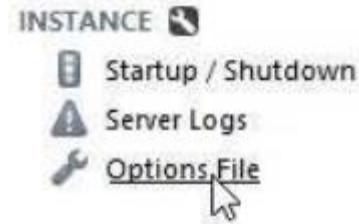
Понекогаш, обично при лошо реализирана конкуренција, доаѓа до дојава на **deadlock**. Тоа е случај кога две трансакции се блокираат една со друга. Да речеме дека едната трансакција држи заклучен ресурс на кој другата трансакција чека за да продолжи со извршувањето, а другата трансакција го држи заклучен ресурсот кој и е потребен на првата трансакција за да продолжи со извршувањето. Конечно, двете трансакции бесконечно чекаат една на друга.

InnoDB може да детектира deadlock и во тој случај ќе изврши автоматски повик на трансакциите кои се наоѓаат во него. Но, доколку во трансакцијата учествуваат и табели кои го немаат овој механизам за складирање или кога табелите се заклучени рачно со lock table директивата, InnoDB, едноставно, не ја детектира ваквата ситуација и не е во состојба да препознае дека трансакциите се взаемно заклучени.

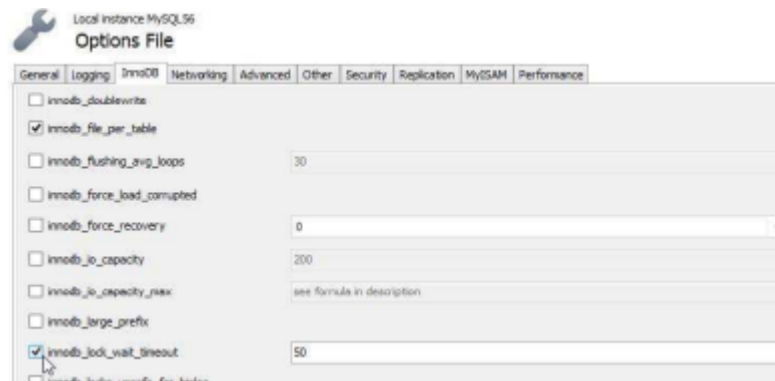
Затоа може да се одреди временски период кој претставува колку трансакцијата ќе чека за целниот ресурс да се ослободи пред да го напушти.

Атрибутот кој ја содржи оваа вредност е innodb_lock_wait_timeout, но до него не може да се дојде преку упити, туку мора да биде иницијализиран при стартувањето (рестартувањето) на серверот.

За да дојдеме до оваа опција во MySQL Workbench, потребно е да ја избереме опцијата Options File од одделот Instance.



По ова треба да се пронајде ставката е `innodb_lock_wait_timeout` и да се постави вредност, која се извршува во секунди и го означува времето колку одредена трансакција ќе чека на ресурс пред да се изврши `rollback`.



18.3.4. ИЗОЛИЦИОНИ НИВА (ISOLATED LEVELS)

Изолационите нива влијаат на начинот на извршување на командата која се наоѓа во трансакцијата. Тие се некој вид на изолациони шеми под кои трансакциите се извршуваат. Во следната листа, изолационите нива се сортирани по ексклузивност. Првоприкажаните се оние со кои се постигнува максимална брзина на пристап (нема взаемно блокирање), а на крајот се оние со кои се постигнува максимално осигурување од симултано менување на податоците од страна на повеќе клиенти:

- **READ UNCOMMITTED** – Заклучувањето трае само додека траат измените. Доколку се случуваат измени во една трансакција, веднаш се видливи и за другите трансакции, иако сеуште не се прифатени. Поради ова изолационо ниво се нарекува **dirty read**.

- **READ COMMITTED** – Промените во трансакцијата се видливи за другите трансакции дури по наредбата **COMMIT**, т.е. по успешното завршување на трансакцијата. Ова значи дека **SELECT** во трансакцијата може да има различни вредности во зависност од тоа дали податоците се променети во некоја друга трансакција која е завршена.
- **REPEATABLE READ** – Трансакцијата ги заклучува ексклузивно ресурсите на кои пишува и споделените (shared) ресурси од кои чита. Заклучувањето трае се додека трае и трансакцијата, па не може да дојде до измена на податоците во текот на трансакцијата. Ова е дефолтното изолационо ниво на MySQL серверот.
- **SERIALIZABLE** – Подразбира најголема стапка на изолација. Секоја трансакција е изолирана и ги држи ресурсите ексклузивно.

Изолационото ниво се подесува на ниво на серверот и може да се види со наредбите:

```
SHOW GLOBAL VARIABLES LIKE 'tx_isolation';
```

или

```
SELECT @@global.tx_isolation;
```

За измена на изолационото ниво се користи наредбата **set transaction isolation level**:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Вака напишаната наредба важи само на ниво на моменталната конекција и има идентичен резултат како:

```
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

За изолационото ниво да важи за сите конекции на ниво на серверот, ќе треба да го декларираме како глобален:

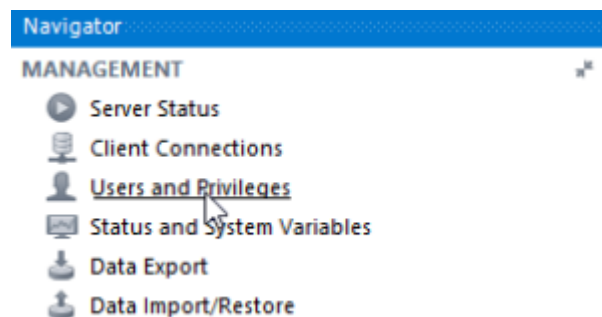
SET GLOBAL TRANSACTION ISOLATION LEVEL SERIALIZABLE;

19. КОРИСНИЦИ И ПРАВА НА ПРИСТАП

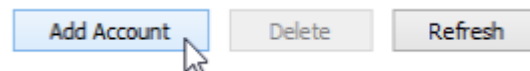
Се досега сите манипулации над базата на податоци ги вршевме со користење на еден кориснички налог. Во реални услови ќе креираме конкретни различни корисници за различни типови на апликации кои ќе комуницираат со базата на податоци. Различните видови на корисници ќе имаат различни права на пристап над објектите на серверот.

19.1. КРЕИРАЊЕ НА КОРИСНИК

За да креираме нов корисник со користење на MySQL Workbench алатот, потребно е прво да се конектираме на серверот преку root налогот. Потоа ја одбираме опцијата Users and Privileges од одделот Management во левиот дел од прозорецот.



По ова потребно е да ја избереме опцијата Add Account од дното на прозорецот:



Во полињата се внесуваат соодветните податоци за името на корисникот и неговата шифра, па се притиска Apply:

Во тој момент MySQL Workbench на серверот му го праќа упитот:

```
CREATE USER 'test_user'@'%' IDENTIFIED BY '123'
```

Овој упит за серверот значи да додаде нов корисник, чие име ќе биде `test_user`, хост `%`, а шифра `123`. Хостот во овој контекст претставува адресата од која корисникот ќе може да пристапува на серверот. Карактерот `%` значи дека корисникот може да пристапува на серверот од било која адреса.од безбедносен аспект ова не е добра пракса. Доколку би сакале за еден корисник да пристапува од неколку повеќе IP адреси, ќе мора за секој хост да направиме корисник со исти пристапни податоци, а различен хост.

Вака креираниот корисник на серверот нема никакви права, а за да се увериме дека тој постои, ќе напишеме:

```
USE mysql;
```

```
SELECT * FROM user;
```

По притискање на тастерот Execute, ќе се појави листа од сите корисници на серверот, заедно со новокреираниот корисник `test_user`.

19.2. ПРАВА НА ПРИСТАП

За да ги промениме привилегиите на креираниот корисник ќе напишеме:

```
GRANT SELECT, UPDATE, CREATE, DELETE ON *.* TO 'test_user'@'%'
```

Упитот започнува со наредбата GRANT (одобрување), потоа следи листата на привилегии (ние внесовме SELECT, UPDATE,CREATE и DELETE, но листата на

привилегии е подолга), а потоа следи адресата на базата над која корисникот ќе има привилегии. Секвенцата на карактери *.* означува дека ќе има привилегии над сите бази на податоци на серверот. Следи клучното зборче TO, по кое го испишуваме називот на корисникот по веќе познатиот пример. Називот на корисникот под наводници, ознаката @ и хостот, исто така под наводници.

На сличен начин може и да ги одземеме правата од корисникот:

```
REVOKE SELECT, UPDATE, CREATE, DELETE FROM *.* TO 'test_user'@'%'
```

Наместо наредбата GRANT, ставивме наредба REVOKE, а наместо TO напишавме FROM.

Може да се додадат или одземат сите привилегии одеднаш:

```
GRANT ALL PRIVILEGES ON *.* TO 'test_user'@'%'
```

Вакво нешто може да постигнеме и со ажурирање на табелата user:

```
UPDATE USER SET select_priv='Y', insert_priv='Y'
```

```
.....
```

```
WHERE USER='test_user' and host='%'
```

Точките во вториот ред претставуваат листа на рачно внесени привилегии.

По измените на привилегиите на овој начин, неопходно е да се изврши и наредбата: Flush Privileges. По тоа, серверот ќе ги прифати новите привилегии на корисникот.

Сепак, наједноставниот начин за додавање и одземање на привилегии е со користење на MySQL Workbench.

Навраќајќи се во одделот за манипулација со корисниците, каде освен Login табот, со кој работевме и ги внесовме основните податоци за новиот корисник, може да забележиме уште неколку табови:

Details for account test_user@%

[Login](#)[Account Limits](#)[Administrative Roles](#)[Schema Privileges](#)

Во склоп на табот Account Limits може да ги поставиме граничните вредности кои се однесуваат на комуникацијата на корисникот со серверот. Со користење на оваа опција може да го ограничиме бројот на операции или логирања кои корисникот може да ги изврши за еден час.

Max. Queries:	0	Number of queries the account can execute within one hour.
Max. Updates:	0	Number of updates the account can execute within one hour.
Max. Connections:	0	The number of times the account can connect to the server per hour.
Concurrent Connections:	0	The number of simultaneous connections to the server the account can have.

За нас, моментално, најважен таб е Administrative Roles. Тој е поделен на две логички целини. Првата целина се однесува на можноста за доделување на одредени права на корисникот.

Role	Description
☐ DBA	grants the rights to perform all tasks
☐ MaintenanceAdmin	grants rights needed to maintain server
☐ ProcessAdmin	rights needed to assess, monitor, and kill any user process running in server
☐ UserAdmin	grants rights to create users logins and reset passwords
☐ SecurityAdmin	rights to manage logins and grant and revoke server and database level permission
☐ MonitorAdmin	minimum set of rights needed to monitor server
☐ DBManager	grants full rights on all databases
☐ DBDesigner	rights to create and reverse engineer any database schema
☐ ReplicationAdmin	rights needed to setup and manage replication
☐ BackupAdmin	minimal rights needed to backup any database

Во системот веќе постојат однапред креирани улоги, а при избор на некои од нив, во десниот дел од прозорецот одредени глобални привилегии се чекираат. Во десниот дел од прозорецот се излистани сите глобални привилегии кои еден корисник може да ги посадува. На корисникот може да му доделуваме произволни привилегии директно од оваа листа и при тоа во листата на улоги се креира сосема нова custom улога.

Листата со сите привилегии е прикажана во следната табела:

Privilege	Column	Context
CREATE	Create_priv	databases, tables, or indexes
DROP	Drop_priv	databases, tables, or views
GRANT OPTION	Grant_priv	databases, tables, or stored routines
LOCK TABLES	Lock_tables_priv	databases
REFERENCES	References_priv	databases or tables
EVENT	Event_priv	databases
ALTER	Alter_priv	tables
DELETE	Delete_priv	tables
INDEX	Index_priv	tables
INSERT	Insert_priv	tables or columns
SELECT	Select_priv	tables or columns
UPDATE	Update_priv	tables or columns
CREATE TEMPORARY TABLES	Create_tmp_table_priv	tables
TRIGGER	Trigger_priv	tables
CREATE VIEW	Create_view_priv	views
SHOW VIEW	Show_view_priv	views
ALTER ROUTINE	Alter_routine_priv	stored routines
CREATE ROUTINE	Create_routine_priv	stored routines
EXECUTE	Execute_priv	stored routines
FILE	File_priv	file access on server host
CREATE TABLESPACE	Create_tablespace_priv	server administration
CREATE USER	Create_user_priv	server administration
PROCESS	Process_priv	server administration
PROXY	see proxies_priv table	server administration
RELOAD	Reload_priv	server administration
REPLICATION CLIENT	Repl_client_priv	server administration
REPLICATION SLAVE	Repl_slave_priv	server administration
SHOW DATABASES	Show_db_priv	server administration
SHUTDOWN	Shutdown_priv	server administration
SUPER	Super_priv	server administration
ALL [PRIVILEGES]		server administration
USAGE		server administration

Доделувањето на привилегии во MySQL се заснова на релационен принцип. Одредени објекти во системот имаат дефинирани сетови на права за некои други објекти во системот. Ваквиот тип на менаџмент на пристап се нарекува Access Control List.

Во една табела се наоѓаат сите корисници, а во друга – сите објекти. За да знаеме кој корисник над кој објект има привилегии, потребна ни е трета табела, која ќе ги спојува овие две. Овој пристап го симболизира комплетниот релационен концепт на базите на податоци, бидејќи тоа е стандард за остварување на однос повеќе спрема повеќе (many to many).

Бришењето на корисник е исто како и бришењето на секој друг објект од серверот:

DROP USER test_user;

Вежба 26: Потребно е да се креира база на податоци application_04.

Оваа база треба да ги има следните три корисника:

1. app_04_admin, со шифра хуз, кој ќе може да се логира само од локалниот компјутер и ќе ги има сите права над базата;
2. app_04_user, со шифра 123, кој ќе може да се логира само од одредени IP адреси и ќе има можност за select, insert, update и delete наредби во базата.
3. App_04_viewer, со шифра abc, кој ќе може да се логира од сите IP адреси и ќе има можност само за select наредби во базата.

Решение:

```
create database application_04;
create user 'app_04_admin'@'localhost' identified by 'xyz';
grant all privileges on application_04 .* to 'app_04_admin'@'localhost';
create user 'app_04_user'@'ip address' identified by '123';
grant select, insert, update, delete on application_04 .* to
'app_04_user'@'ip address';
create user 'app_04_viewer'@'%' identified by 'abc';
grant select on application_04 .* to 'app_04_viewer'@'%';
```

Вежба 27: Потребно е да се изменат правата доделени во претходната вежба така што:

1. Корисникот app_04_admin има право само на select, update, delete и insert;
2. app_04_user има право само на select наредби;
3. App_04_viewer веќе не постои.

Решение:

```
show grants for 'app_04_user'@'ip address';
revoke update, insert, delete on *.* from 'app_04_user'@'ip address';
drop user 'app_04_viewer'@'%';
show grants for 'app_04_admin'@'localhost';
revoke all on application_04 .* from 'app_04_admin'@'localhost';
grant select, insert, update, delete on application_04 .* to
'app_04_admin'@'localhost';
```

20. БЕЗБЕДНОСТ ВО MYSQL

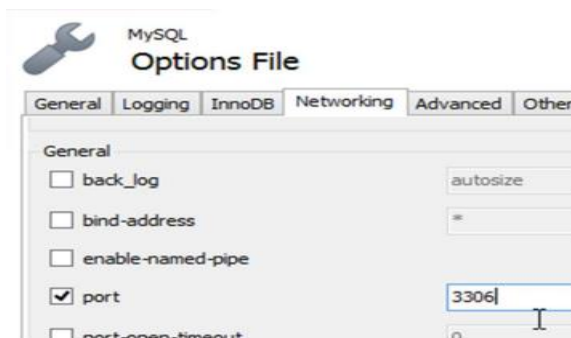
Еден од најбитните аспекти во одржувањето на еден сервер е неговата безбедност. Во зависност од типот на системот базата на податоци може да содржи многу чувствителни податоци, а голема одговорност за безбедноста на податоците му припаѓа на администраторот на базата на податоци. Практично, во моментот кога MySQL серверот ќе се инсталира и ќе стане достапен на мрежата, тој станува изложен на потенцијални напади.

20.1. ПОРТА

MySQL работи на TCP/IP портата 3306. Оваа порта е дефолтна порта за MySQL, но не и задолжителна. Доколку MySQL е дел од систем кој не бара стандардни параметри, оваа порта може да биде променета. На тој начин ќе се подигне безбедноста на серверот, бидејќи потенцијалните напаѓачи нема да го најдат серверот таму каде што обично се наоѓа (на портата 3306). Тоа нема да го спречи нападот, но може да го одложи.

Корисниците за кои ќе биде достапна оваа порта треба да бидат филтрирани со firewall. Инаку, базата на податоци се наоѓа на еден слој, додека самата логика обично се наоѓа на некој друг слој. Кога крајниот корисник му пристапува на системот, неговиот максимум треба да биде управување со логиката преку посредничка апликација, бидејќи нема никаква потреба тоа да се извршува директно, без посредник. Уште помалку да управува директно со податоците, поради што и нема никаква потреба тие да бидат директно изложени на надворешниот свет. Во секој случај, ова што не смееме да дозволиме никогаш да се случи е да дозволиме директен пристап до серверот од страна на непроверен хост.

Контролата на портата се врши во зависност од системот на кој MySQL серверот се наоѓа (најчесто со помош на firewall), додека портата на која MySQL ќе работи може да се подеси од MySQL Workbench – опција Options File, таб Networking.



Овој податок може да се промени и рачно со ажурирање на фајлот `my.ini` (кој се наоѓа во инсталациониот фолдер на MySQL серверот). Подесувањата за портата се наоѓаат во следниот оддел:

```
[mysqld]
```

```
# The TCP/IP Port the MySQL Server will listen on port=3306
```

20.2. КОРИСНИЦИ

Како што веќе рековме, безбедносниот метод на MySQL серверот е базиран на ACL (Access Control List) методата. Тоа значи дека за секој корисник постојат одредени права кои ги има во однос на некој објект. Кога ги менуваме привилегиите на корисниците, ние всушност само го менуваме односот кои корисниците го имаат спрема одредени објекти.

Како што ни веќе знаеме, `root` корисникот е супер администратор на MySQL серверот, кој ги поседува сите достапни привилегии. Иницијално, `root` корисникот е без лозинка, така што секој може да изврши конектирање на серверот како `root` корисник и на тој начин да ги оствари сите привилегии.

За да ја подигнеме безбедноста, потребно е да поставиме шифра за `root` корисникот. Тоа може да се постигне со следниве наредби:

```
SET PASSWORD FOR 'root'@'localhost'= PASSWORD('newpwd');
```

```
SET PASSWORD FOR 'root'@'127.0.0.1'= PASSWORD('newpwd');
```

SET PASSWORD FOR 'root'@'::1' = PASSWORD('newpwd');

Да погледнеме кои уште кориснички налози се креирани освен root корисникот. Во табелата на корисници може да видиме и кориснички налог без корисничко име и шифра. Тоа е т.н. anonymous корисник, кој овозможува конектирање на серверот без корисничко име и лозинка, но само од локалниот компјутер (localhost). Овој корисник е без права. За подигнување на безбедноста овој корисник може да се отстрани или да му се додели лозинка.

Исто така, добра пракса е да ја искористиме наредбата за преглед на корисничките права на серверот, така што ќе утврдиме дали правата се адекватни:

SHOW GRANTS;

20.3. АПЛИКАЦИЈА

Со MySQL серверот се управува преку апликација, така што апликацијата ќе праќа барања до серверот и ќе прзема одговори од него. Праќањето на упити кон серверот е критичен процес, посебно ако резултатот е некој кориснички дефиниран упит.

На пример корисникот го внесува своето име и шифра. Апликација името и шифрата ги сместува во упит и го проследува на базата на податоци. Ова функционира добро се додека корисникот навистина се обидува да ги внесе своето корисничко име и шифра, но корисникот може да се обиде и покрај бараните податоци да “прошверцува” и некоја дополнителна SQL наредба која може да влијае на работата на серверот (т.н. SQL Injection). Затоа, податоците кои доаѓаат до серверот мора да бидат адекватно филтрирани во самата апликација.

Потребно е да се придржуваме до следните правила:

- Не смее да се дозволи корисниците преку апликацијата да праќаат единечни наводници (') на серверот;

- Не смее да и се дозволи на апликацијата да се обиде во полето да внесе вредност со несоодветен тип;
- Не смее да и се дозволи на апликацијата да се обиде во полето да внесе вредност која ја надминува големината на целниот тип;
- Не смее да и се доделат на апликацијата високи кориснички привилегии или да и се додели root налогот; апликацијата мора да има свој посебен налог со привилегии кои и овозможуваат само она што и е неопходно за работа;
- По креирањето на апликацијата и доделување на корисници во базата, потребно е да се ограничи полето за дејствување на тој корисник само на адресата на апликацијата.

20.4. ПОДАТОЦИ

Секаде каде тоа е можно треба да не и се дозволува на апликацијата директна манипулација со податоците преку упити, туку да се ограничи на користење на функционалностите на базата (складирани процедури, погледи,...), а самите упити од најниско ниво да се извршуваат во контекст на самата база, на серверот, а не на апликацијата.

За чување на податоци со критична содржина (на пример, шифри), потребно е, доколку има можност да се чуваат криптирани (со некоја еднонасочна енкрипција (на пример, md5 или sha)). За ова може да се искористат вградените функции md5() и sha().

Препорачливо е, доколку апликацијата и базата се наоѓаат на интернет на раздвоени компјутери, односно, доколку се поврзуваат повеќе сервери преку интернет, да се користи SSL енкрипција.

Вежба 28: На компјутерот постои инстанца од MySQL сервер. На истиот сервер постои и web апликација.

Потребно е да се измени структурата на системот, така што базата на податоци и web апликацијата да бидат на различни компјутери. Потребно е да се обезбедат дополнителни елементи кои би ја зголемиле безбедноста на серверот.

Решение:

Во my.ini фајлот пожелно е да се промени портата од дефолтна на нестандартна:

```
[mysqld]
```

```
Port=3306
```

(на пример, 1434)

Корисникот root треба да ги одземе правата на работата од оддалечениот компјутер, доколку тој евентуално ги поседува:

```
DROP USER 'root'@'%';
```

или

```
DROP USER 'root'@'ip address';
```

Се креира корисник за базата:

```
Create user 'peter'@'web server ip address' identified by '123';
```

Доделување на минимални привилегии на корисникот за целната база:

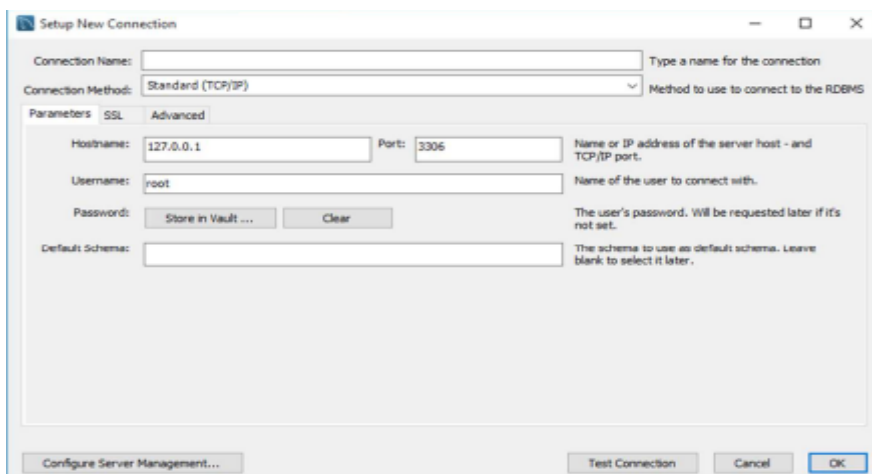
```
grant select, update, delete on application_03 .* to  
'peter'@'web server ip address';
```

21. КОНЕКЦИИ

Бидејќи постојат два ентитета клиент-сервер во опкружувањето на MySQL, а тоа се од една страна корисниците, а од друга серверот, потребно е на некој начин да се изврши поврзување на овие два ентитета. Во моментот кога му пристапуваме на MySQL серверот, ние бараме од него да отвори посебен процес, независен од сите останати процеси.

21.1. КРЕИРАЊЕ НА КОНЕКЦИЈА

Веќе ни е познато дека прозорецот за креирање на конекција се добива со кликање на тастерот плус (+) на почетниот прозорец од MySQL Workbench. По ова се прикажува прозорец за креирање на нова конекција:



Првата опција се однесува на називот на конекцијата. Веднаш под неа се наоѓа опцијата Connection Method, која го одредува методот кој ќе биде користен за конектирање на базата.

Кога клиентот и серверот се наоѓаат на различни машини, работите се многу едноставни: комуникацијата меѓу два компјутери, т.е. две програми мора да се остварува преку TCP/IP протокол. За ова да биде можно, мора да бидат исполнети два услова:

- Компјутерите мора да бидат поврзани преку TCP/IP, што многу лесно може да се провери со командата ping;
- Дефолтната порта 3306 (или некоја друга порта која е подесена) не смее да биде блокирана од страна на firewall.

Доколку клиентот и серверот се наоѓаат на ист компјутер, работите малку се комплицираат, бидејќи во тој случај постојат малку поголем број на опции за воспоставување на конекцијата:

- TCP/IP – овој мрежен протокол исто така може да биде користен и за комуникација меѓу програми на иста машина;
- socket фајл - socket документот овозможува ефикасна комуникација на Linux/Unix, каде овој вид на комуникација е дефолтен, со оглед дека е поефикасен од комуникацијата преку TCP/IP;

- `named pipes` – претставуваат, во некоја рака, одговор на `Windows`-от на `socket` комуникацијата на `Linux/Unix`; сепак, ваквиот вид на комуникација дефолтно е оневозможен, па неопходно е да се активира пред употребата; освен тоа, и `MySQL` клиентот мора да подржува ваков вид на комуникација, па тој скоро никогаш не се користи во реална употреба;
- `shared memory` – и овој вид на комуникација функционира само на `Windows` оперативниот систем и исто така претставува пандан на `socket` фајловите; функционира така што клиентот и серверот делат ист мемориски простор; сепак, и овој вид на комуникација скоро никогаш не се користи поради идентични проблеми како оние што постојат кај `named pipes`.

Останатите опции на прозорецот ни се веќе познати:

- `hostname` – адреса на компјутерот на кој е инсталиран `MySQL` серверот;
- порта на компјутерот на која е инсталиран `MySQL`;
- `password` – корисничка лозинка;
- `default schema` – дефолтна база на податоци која ќе биде селектирана при активирање на конекцијата.

Не е задолжително лозинката на корисникот да биде внесена веднаш; доколку не биде внесена, таа ќе ни биде побарувана при секое конектирање на серверот. Доколку, сепак, одлучиме да ја внесеме веднаш, потребно е да ја избереме опцијата `Store in Vault`.

Следниот таб во прозорецот е за `SSL`. За да разбереме како `MySQL` го користи `SSL`, потребно е да поседуваме некои основни знаења за `SSL` и `X509` концептот.

По дефолт, `MySQL` воспоставува некриптирана конекција меѓу клиентот и серверот. Тоа значи дека било кој има пристап до мрежата може да ја следи комуникацијата меѓу `MySQL` клиентот и серверот. Секој кој има пристап на мрежата може дури и да ги измени податоците кои патуваат од клиентот кон

серверот. За да ја подобриме безбедноста на податоците, може да користиме енкрипција на податоци со користење на опцијата `compress`, но ваквиот пристап не помага многу кога станува збор за напади.

Кога е потребно вршење на размена на информации во сигурен амбиент, некриптираната конекција не доаѓа во предвид. Енкрипцијата претставува начин на кој податоците кои патуваат преку мрежата може да се направат нечитливи.

SSL е протокол кој користи различни алгоритми за енкрипција за да се обезбеди безбеден пренос на чувствителните податоци преку јавната мрежа. SSL исто така инкорпорира алгоритам кој овозможува проверка на идентитетот со користење на X.509 стандардот.

X.509 овозможува идентификација на интернет. Најчесто се користи во апликации за електронска продажба. Во ваквиот систем постои ентитет кој се нарекува Certificate Authority (CA), кој извршува доделување на електронски сертификати. Сертификатите се засноваат на асиметрична енкрипција, каде постојат два клуча, јавен и таен. За да се авторизира втората страна, потребно е да го покаже сертификатот со јавен клуч кој може да биде декриптиран единствено со користење на тајниот клуч кој се чува кај издавачот на сертификатот.

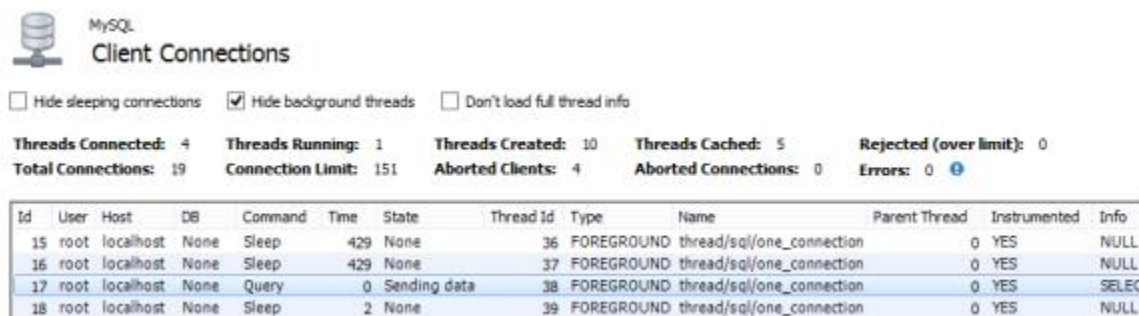
Аналогно на ова, постојат и опции во табот SSL:

21.2. ПРЕГЛЕД НА КОНЕКЦИЈАТА

Значи, штом му пристапиме на MySQL серверот, тој отвара нов процес за нас, независен од сите други останати процеси. При отворањето на процесот, тој му доделува и единствен број. Овие процеси може да ги погледнеме со упитот:

SHOW processlist;

или со помош на MySQL Workbench, преку опцијата Server Statuses во одделот Management, а прикажаните конекции изгледаат како на сликата подолу:



The screenshot shows the 'Client Connections' window in MySQL Workbench. It includes summary statistics at the top and a table of active connections below.

Summary Statistics:

- Threads Connected: 4
- Threads Running: 1
- Threads Created: 10
- Threads Cached: 5
- Rejected (over limit): 0
- Total Connections: 19
- Connection Limit: 151
- Aborted Clients: 4
- Aborted Connections: 0
- Errors: 0

Active Connections Table:

Id	User	Host	DB	Command	Time	State	Thread Id	Type	Name	Parent Thread	Instrumented	Info
15	root	localhost	None	Sleep	429	None	36	FOREGROUND	thread/sql/one_connection	0	YES	NULL
16	root	localhost	None	Sleep	429	None	37	FOREGROUND	thread/sql/one_connection	0	YES	NULL
17	root	localhost	None	Query	0	Sending data	38	FOREGROUND	thread/sql/one_connection	0	YES	SELEC
18	root	localhost	None	Sleep	2	None	39	FOREGROUND	thread/sql/one_connection	0	YES	NULL

Податоците за еден процес ги подразбираат следните ставки:

- Id – идентификатор на конекцијата;
- User – корисничко име;
- Host – хост од кој корисникот доаѓа;
- DB – база која корисникот моментално ја користи;
- Command – тип на актуелна команда;
- Time – време за кое процесот ја извршува актуелната команда;
- State – статус на процесот;
- Info - актуелен упит на процесот.

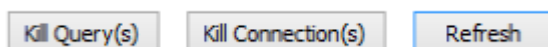
Дали конекцијата моментално нешто работи може да видиме од полето Command. Ова поле содржи информација за типот на акцијата која процесот ја извршува. Двата типа кои најчесто ќе ги гледаме се query и sleep. Query значи дека процесот моментално извршува упит, додека sleep значи дека процесот не извршува ништо и чека исклучување. Ваквиот процес не го оптеретува серверот и не ја задржува конекцијата и може безбедно да се исклучи. Понекогаш, sleep командата на процесот значи и дека се чуваат податоци за некоја истечена конекција (pooling), за да се забрза евентуално повторниот процес на конектирање. Вредностите на командните параметри на процесот се многу повеќе од споменатите две.

21.3. ЗАТВАРАЊЕ НА КОНЕКЦИЈАТА

Конекцијата може да биде затворена во било кој момент со наредбата:

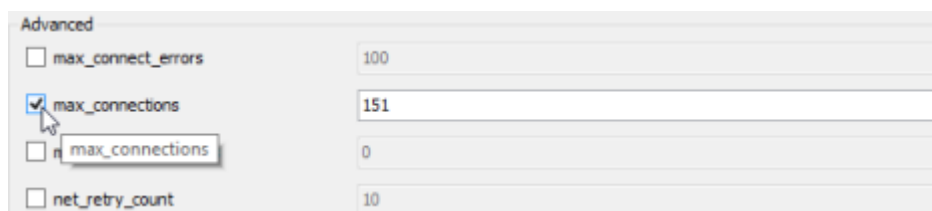
KILL connection_ID;

connection_ID претставува id на конекцијата. Затварање на конекцијата може да се постигне и со користење на тастерот во дното од прозорецот:



21.4. ПОДЕСУВАЊЕ НА КОНЕКЦИЈАТА

MySQL серверот нема физички ограничувања што се однесува на бројот на процеси (нишки threads). Овој број зависи од можностите на опкружувањето. Обврска на администраторот да води сметка за тоа колку конекции ќе дозволи во базата, односно, колку ќе го оптовари серверот. Ова може да се подеси преку MySQL Workbench, со користење на опцијата max_connection, која може да се пронајде со избор на опцијата Option File од одделот Configuration, а потоа со избор на табот Networking:



Ќе подесиме сите иницијални подесувања на MySQL серверот да може да се подесат и во иницијалниот фајл my.ini.

Проблемот со процесите е тоа што не само администраторот на базата може да влијае на нив. Всушност, отворањето и затварањето на конекцијата се случува во апликацијата која се конектира на базата, така што голема одговорност лежи и на авторот на таа апликација.

Да речеме дека една web апликација има страна на која се наоѓа една конекција. Таа апликација во базата е дефинирана како еден корисник. Но, што ако повеќе корисници ја отворат истата страна? Во тој случај, еден корисник на базата (web апликацијата) ќе отвори и повеќе конекции.

Посебна опасност лежи во т.н. “цурење на конекцијата”. Доколку апликацијата која го отвара процесот, поради грешки во кодот, не го затвори процесот и таквите процеси почнат да се насобираат, може да дојде до преголем број на конекции кои може да го оптоварат серверот. Серверот, секако, може после извесно време да ги исклучи неактивните конекции.

Покрај опцијата `max_connection`, важни ни се и опциите кои се наоѓаат во Advanced и Timeout Settings одделот, Networking табот.

Advanced подесувањата веќе ги спомнавме (контрола на бројот на конекции). Освен `max_connection` подесувањата, овде може да ги регулираме и максималниот број на конекции по корисник (Max Conn per User), колку обиди ќе бидат дозволени за реконекција доколку дојде до загушување на портата (Retry Count) и дозволениот број на прекини на конекцијата пред серверот да ја оневозможи конекцијата од актуелниот домен (Max Conn Errors).

Timeout подесувањата, генерално се однесуваат на времето за кое серверот ќе чека за да се изврши некој очекуван настан пред да го оневозможи.

Connection Timeout претставува времето на чекање од почетокот до крајот на воспоставување на некоја конекција со серверот. Доколку конектирањето трае подолго од зададениот број на секунди, конекцијата нема да биде воспоставена. Во ова и во останатите подесувања, нула значи неограничено време. Доколку некоја конекција постои, но не е активна, серверот може да ја прекине. Interactive и Wait Timeout се опции кои му кажуваат на серверот колку да чека пред таквата конекција да ја прекине. Read и WriteTimeout означуваат колку секунди серверот ќе чека за следниот податок пред да ја прекине конекцијата.

Timeout Settings	
☐ connect_timeout	10
☐ interactive_timeout	28800
☐ net_read_timeout	30
☐ net_write_timeout	60
☐ wait_timeout	28800

Поголемиот дел од овие наредби спаѓаат во глобални подесувања, кои може да се изменат и со наредбата Set Global:

SET GLOBAL connect_timeout = 15;

Сите глобални променливи може да се видат со наредбата:

SHOW GLOBAL VARIABLES;

22. РЕПЛИКАЦИЈА

Репликацијата овозможува синхронизација на податоците меѓу два или повеќе сервери кои работат на иста или различни машини. Во продукциски услови, скоро секогаш тоа се две различни машини.

Различните системи за управување со бази на податоци на различен начин ја реализираат репликацијата, така што потребно е да се запознаеме со начинот на функционирање на репликацијата на секој поединечен систем за управување со бази на податоци.

MySQL подржува т.н. master/slave систем за репликација. Master постои како примарно складиште, над кое може да се врши и читање и пишување на податоци. Ова е системот кој е одговорен за сите промени на податоците, додека slave серверот (или серверите) се користат за репликација на податоците од master. За разлика од master, slave серверите се користат само за читање на податоци

Лесно може да заклучиме дека секој од овие елементи претставува еден MySQL сервер. При тоа, master може да биде само еден, додека бројот на slave сервери може да биде произволен, практично неограничен, односно во границите на можностите на системот.

Репликацијата функционира така што за многу кратко време сите промени на податоците на master серверот се реплицираат на сите slave сервери.

Репликацијата функционира дури и доколку компјутерите кои учествуваат во овој систем имаат инсталирано различни оперативни системи.

22.1. КОРИСТЕЊЕ НА РЕПЛИКАЦИЈА

Постојат две главни причини поради кои користиме репликација, а тоа се безбедноста и брзината.

Безбедност

Благодарение на репликацијата, базата на податоци е достапна на неколку компјутери. Доколку slave компјутерот престане да работи, целиот систем продолжува да работи без никакви проблеми. Кога slave компјутерот ќе продолжи со работа, ќе се изврши неопходната синхронизација која е пропуштена во меѓувреме.

Доколку master компјутерот откаже, т.е. прекине со работа, тогаш интегритетот на податоците може да биде гарантиран од страна на slave компјутерите. Може дури и целиот систем да се конфигурира така што slave компјутерите ќе ја преземат улогата на master доколку тој откаже. Во такви ситуации, целиот систем е достапен само за читање.

Со помош на репликацијата олеснето е и правењето на резервни копии на податоците (backup) и податоците се секогаш ажурирани.

Брзина

Со користење на репликација може да се постигне и зголемување на брзината. Позахтевните упити се делат меѓу slave компјутерите, додека master системот се користи исклучиви за ажурирање на податоците. Доколку сакаме да ја користиме ваквата функционалност, нашиот клиентски програм мора однапред да подготви упити кои ќе можат да се извршуваат дистрибуирано.

Предности во користењето на репликацијата се:

- растеретување на компонентите на големите бази;
- безбедност/backup на податоците;
- анализа на податоците;
- оддалечена дистрибуција на податоците.

22.2. ФУНКЦИОНИРАЊЕ НА РЕПЛИКАЦИЈА

Освен во случај на кластер MySQL конфигурација, репликацијата е едносмерен, асинхронен процес, во кој податоците се движат од master серверот кон slave серверите.

За да може да функционира репликацијата потребно е на master серверот да биде активиран бинарен log (дневник). Во моментот кога slave ќе почнат да слушаат на master, тие се позиционираат на заедничка точка на логот (секоја позиција во логот е маркирана со број). Во текот на извршувањето на SQL наредните на master серверот (и во master логот), доаѓа до промена на позицијата на маркерот во логот. Периодично, slave серверот го прегледува master логот и го споредува со претходната верзија. Се што се случува меѓу две проверки се извршува и на slave и на тој начин се добива точно пресликување на податоците.

Репликацијата во MySQL може да се активира на повеќе различни начини, од обични SQL наредби и конзолни наредби до употреба на апликации чија единствена намена е управување со репликации.

22.3. КРЕИРАЊЕ НА РЕПЛИКАЦИЈА

Ќе дефинираме најпроста репликација, т.е. репликација која ќе се состои од еден master и еден slave.

Креирање на корисник

По одредувањето на целта на репликацијата (кога веќе знаеме каква ќе биде нејзината архитектура, колку сервери ќе има во неа, кои податоци ќе бидат земени во предвид), потребно е да креираме корисник на репликацијата. Тоа не е неопходен чекор и корисникот не мора да биде креиран специјално за таа намена, но тоа строго се препорачува од безбедносни причини.

Овој корисник ќе го репрезентира slave серверот на master серверот и поради тоа мора да има привилегија REPLICATION SLAVE. Да речеме дека корисникот за кого ја имаме креирано репликацијата гер, е со шифра 123:

```
GRANT REPLICATION SLAVE ON *.* TO 'rep'@'localhost' IDENTIFIED BY '123';
```

Доменот на корисникот е localhost (бидејќи станува збор за проба), што е редок случај во пракса (slave серверите ќе бидат на други компјутери).

Подесување на master репликацискиот сервер

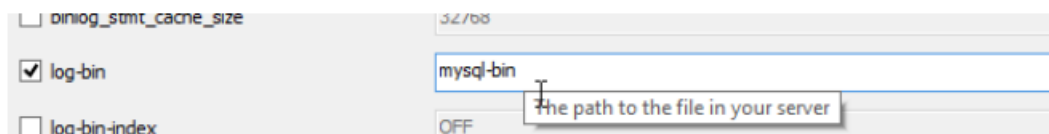
По корисникот за репликација, потребно е да се подготват и самите сервери, пред се master. Минимум потребни услови за серверот да биде учесник во репликација како master е да поседува репликациски id и бинарен лог.

Тоа може да го постигнеме со директна интервенција на my.ini фајлот или со користење на MySQL Workbench. Без оглед на начинот на кој тоа ќе го направиме, резултатот ќе биде следниве две линии код во my.ini фајлот:

```
log-bin=mysql-bin
```

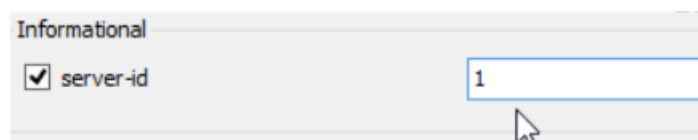
```
server-id=1
```

Преку MySQL Workbench ова може да се постигне на следниов начин. Дефолтно бинарниот лог за MySQL серверот не е подесен, така што потребно е да се дефинира фајл во кој ќе се чува. За да го направиме тоа, потребно е прво да ја избереме опцијата Options File од одделот INSTANCE. По ова потребно е да ја пронајдеме и да ја чекираме опцијата bin-log и да го дефинираме називот на фајлот во кој ќе биде сместен:



Називот на логот не е важен, се додека се вклопува во конвенцијата на фајлови за оперативниот систем.

Преостанува уште да видиме на кој начин се поставува id на серверот неопходен за функционирање на репликацијата. Потребно е да го избереме табот GENERAL, каде постои опцијата server-id:



Id-то мора да биде целобројна вредност, поголема од нула, единствена за секој сервер во репликацијата.

По подесувањето на овие параметри, потребно е да се рестартира серверот.

Подесување на slave системот

За slave серверот исто така е потребно да се обезбедат некои податоци. Id, автентикациски податоци за master серверот (автентикација за корисникот на rep), бинарен лог и relay log. (Бинарниот лог за slave не е неопходен, но relay log е, бидејќи во него се наоѓаат сите разлики меѓу бинарниот лог на master и податоците на slave.) Измените на my.ini фајлот се:

server-id = 2

log-bin = bin

log-bin-index = log-bin

log-error = error

relay-log = relay

relay-log-info-file = relay-log

relay-log-index = relay-log-index

По ова, потребно е да ги извршиме следните наредби:

CHANGE MASTER TO

MASTER_HOST='localhost',

MASTER_USER='replication_user',

MASTER_PASSWORD='123',

MASTER_PORT=3306,

MASTER_LOG_FILE='mysql-bin.000003',

MASTER_LOG_POS=120;

MASTER_HOST се однесува на хостот на кој master серверот се наоѓа. MASTER_USER е корисничкото име на корисникот кој го креираме на master за потребите на репликацијата. MASTER_PASSWORD се однесува на шифрата на корисникот. Портата на master серверот е дефинирана со MASTER_PORT подесувањето.

Податоците за називот на лог фајлот (MASTER_LOG_FILE) и неговата позиција (MASTER_LOG_POS) може да се најдат така што на master серверот ќе се изврши:

SHOW MASTER STATUS;

На овој начин се добива табела во која не интересираат две колони – file и position:

File	Position
mysql-bin.000003'	120

Овие податоци ќе ги искористиме за поставување вредности за MASTER_LOG_FILE и MASTER_LOG_POS во горната наредба.

Опишаната процедура е доволна за репликација ако се работи за потполно “чисти” сервери, каде уште нема податоци. Рестартот на slave серверот автоматски ќе ја активира и репликацијата.

Доколку сакаме да ја започнеме репликацијата со постоечки податоци (што е многу веројатно), потребно е на slave да му кажеме за кои позиции треба да ги следи промените на логот, што најлесно може да се направи на следниот начин:

Прво ја преземаме скриптата од master базата со помош на конзолниот алат mysqldump, кој се наоѓа во MySQL bin фолдерот:

```
mysqldump -uroot -master-data my_database > backup.sql
```

- -uroot е корисничко име (-u е прекинувач за корисничко име, а root е називот на корисникот. Бидејќи на root не му доделивме шифра, не ни е потребен дополнителниот прекинувач -p.)
- --master-data е многу важна опција, која го содржи називот на master лог фајлот и позицијата на маркерот. Оваа опција во скриптата ќе ја произведе следната линија:

```
CHANGE MASTER TO MASTER_LOG_FILE='mysql-bin.000001',
```

```
MASTER_LOG_POS=548;
```

- `my_database` е назив на базата. За проба, во оваа скрипта ја ставивме само `my_database`, но можевме да ги ставиме и сите бази (`--all-databases`).
- `backup.sql` го специфицира називот и локацијата на фајлот во кој ќе биде сместена скриптата. Бидејќи не го наведовме називот на фолдерот, овој фајл ќе заврши во фолдерот кај апликацијата `mysqldump` (bin фолдерот).

По извршувањето на овие наредби, доколку е се како што треба, би требало да биде произведен фајл `backup.sql`. По ова, овој фајл ќе ни биде потребен на `slave` серверот. Кога фајлот ќе биде на серверот ќе ја активираме неговата скрипта со следната конзолна наредба:

```
mysql -uroot > backup.sql
```

или на било кој друг начин, на пример со стартирање на скриптата од MySQL Workbench. По ова потребно е само да му пристапиме на `slave` серверот и да ја активираме наредбата:

SLAVE START;

Од овој момент, сите измени на `master` ќе бидат рефлексирани и на `slave` серверот. Репликацијата на `slave` серверот се прекинува со наредбата:

STOP SLAVE;

23. ВАСКУП И МИГРАЦИЈА

Миграција е поим кој е малку поширок од поимот `backup`, но може и да се поклопат. При миграција, најчесто е потребно да се изврши миграција на комплетната SQL структура, односно на сите објекти на серверот –и, секако, на сите податоци.

Кога станува збор за backup, тој може да се изврши така што, освен податоците, кои се примарен интерес на бекапот, да се изврши бекап и на самата структура на базата.

Бекап стратегијата во еден систем зависи од структурата на системот. Некои системи побаруваат често освежување на бекапот, со минимум губиток на податоци; некои не побаруваат чест бекап, а кај некои дури и не постои потреба за бекап.

Овие три особености лесно може да се препознаат на реални примери. Банка во која секојдневно се извршуваат мноштво на финансиски трансакции мора да има цврста и сигурна база, со сигурен и ефикасен бекап систем кој ги бекапира податоците многу фреквентно. Една историска архива, од друга страна, не побарува толку фреквентен бекап систем. Овде податоците ретко се менуваат, и освежувањето на бекапот може да се извршува многу поретко за разлика во случајот со банка. Базата за влезници би била актуелна додека манифестацијата не заврши. Потоа, ни базата ни бекапот не би биле потребни (освен за архива).

Освен што зависи од структурата и намената на базата, начинот на кој ќе ги бекапираме MySQL базите зависи и од нашите можности за пристап до таа база.

23.1. КОПИРАЊЕ НА ФАЈЛОВИ

Наједноставниот и најбрзиот начин за бекап на податоци и миграција е рачно копирање на сите фајлови во базата. Му пристапуваме на data фолдерот на MySQL серверот, го пронаоѓаме фолдерот со саканата база (називот му е ист како и на самата база) и го копираме.

Освен што мора да се извршува рачно, овој начин на бекап има уште две битни негативни особини. Прво, може да го извршиме само доколку имаме пристап до фајл системот на компјутерот, и друго, се отвора само ако базата не содржи табели под InnoDB системот на складирање. Знаеме дека податоците од InnoDB базата се наоѓаат во заеднички фајл data во фолдерот ibdata1, што значи дека овој фајл содржи податоци и од другите бази. Доколку базата е фреквентна,

најдобро е пред копирањето на фајловите да се оневозможи внес на податоци, за да не дојде до губиток на податоци внесени во текот на извршувањето на бекапот. Еден начин да го направиме тоа е со користење на наредбата lock:

FLUSH TABLES WITH READ LOCK;

Оваа наредба е спој на две наредби (lock и flush). Со flush tables на серверот му велíme да ги затвори сите табели и да ги прекине моменталните уписи во нив. Наредбата read lock, која е варијација на наредбата lock table myTable read, забранува упис во табелата.

Што се однесува на враќањето на податоците, тие едноставно се копираат од една локација на друга.

23.2. DUMP ТАБЕЛИ ВО ТЕКСТ ФАЈЛ

Табелите од базата може да се експортираат во текст фајл – секако, под услов да постојат права за користење на фајл системот за актуелниот корисник. Следната наредба ја емитува содржината од табелата customer од базата sakila во фајлот c:/customer.txt:

SELECT * INTO OUTFILE 'customer.txt' FROM customer;

Овој фајл содржи податоци од табелата. Празното место меѓу колоните (полињата) во овој фајл ќе бидат остварени со табови, додека новите редови ќе бидат остварени со специјални карактери.

Ова може да се промени со дополнителни опции FIELDS TERMINATED BY и LINES TERMINATED BY (корисничко дефинирање на делимитер за поле и редови):

SELECT * INTO OUTFILE 'customer.txt'

FIELDS TERMINATED BY '|' LINES TERMINATED BY ' ' FROM customer;

Овој пристап е добар за чување на податоците, но не и на комплетната структура на базата.

Доколку сакаме да ги вчитаме податоците назад во базата од ваков фајл, може да користиме неколку начини.

Првиот начин е користење на наредбата `LOAD DATA INFILE`. Со оваа наредба се извршува вчитување на податоците од некој фајл во табела:

```
LOAD DATA INFILE 'customer.txt' INTO TABLE customer;
```

Другиот начин за вчитување на податоците назад во табелата е со користење на алатот `mysqlimport`. Доколку `bin` фолдерот од MySQL е додаден во системската варијабла `PATH`, потребно е во Command Prompt да внесеме:

```
mysqlimport -u root -local sakila customer.txt
```

Доколку фолдерот `bin` не е додаден во системската `PATH` варијабла, потребно е пред извршувањето на горната наредба да се изврши позиционирање во `bin` фолдерот. Важно е да се напомене дека при ваквото враќање на податоците називите на текстуалниот фајл и табелата во која податоците се вчитуваат мора да бидат идентични.

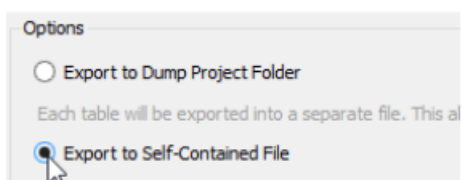
23.3. DUMP БАЗИ ВО ФАЈЛ

Оваа техника веќе ни е позната преку подготовката на базата за репликација. За да се изврши `dump` база во фајл се користи конзолната апликација `mysqldump`, употребувајќи ја следната синтакса:

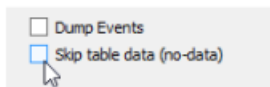
```
mysqldump -u root sakila > sakilaDump.sql
```

Не треба да заборавиме дека `mysqldump` не е SQL наредба, туку конзолна апликација, која се наоѓа во `bin` фолдерот на MySQL, така што ова е потребно да се изврши од Command Prompt.

Ова може да се постигне и преку MySQL Workbench, со користење на опцијата Data Export од одделот Management, по што се отвара прозор за избор на базата која сакаме да ја експортираме и на тој начин да извршиме бекап или миграција. По ова може да избереме дали сакаме извозот на базата на податоци да се изврши така што податоците и структурата на секоја табела ќе се наоѓаат во поединечни фајлови (една табела – еден фајл) или пак сите табели во еден фајл. Креирањето на посебен фајл за секоја табела овозможува поголема слобода во подоцнежната манипулација, т.е. restore.



Може да се избере експорт и без податоците, т.е. извоз само на структурата на базата на податоци, со чекирање на опцијата Skip table data (no-data).



Бекапот започнува со притискање на тастерот Start Export, и по завршувањето на бекапот може да се увериме дека на избраната патека постои бекап фајл со екстензија .sql.

Фајлот со скриптата има екстензија sql, но освен за распознавање на содржината на фајлот, не подразбира никаква посебна карактеристика, туку станува збор за обичен текстуален фајл.

Забелешка: При бекап на базата на овој начин не доаѓа до бекап на складираните процедури.

Постојат два начини за враќање на базата извезена на овој. Првиот е со користење на SQL Monitor. Доколку во .sql фајлот е извршен бекап само на една база на податоци, потребно е да се покрене SQL Monitor со следните параметри:

```
>mysql -u root databasename < backup.sql
```

databasename се однесува на називот на базата на податоци, при што, за да се изврши поврзувањето, неопходно е базата со наведеното име да постои на серверот. Доколку .sql фајлот содржи повеќе од една база на податоци, доволно е да напишеме:

```
mysql -u root < backup.sql
```

Кога се врши dump на повеќе од една база на податоци, mysqldump ги додава неопходните CREATE DATABASE наредби за креирање на база во фајлот, па не е потребно да се креира база нити да се наведува било какво име.

Другиот начин за повраќање на податоците е секако, користење на MySQL Workbench. Потребно е да ја избереме опцијата Data Import/Restore од Management одделот. Во прозорот кој се отвара може да избереме:

- Import from dump project folder – restore од фолдерот – за ситуации кога export се состои од повеќе поединечни фајлови;
- Import from Self-Contained File – за ситуации кога се експортира еден фајл.

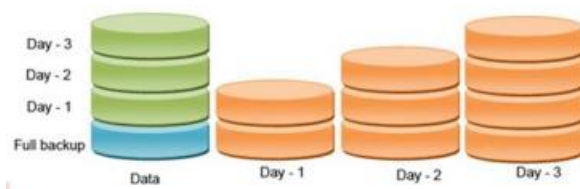
Import започнува со притискање на тастерот Start Import и може да забележиме дека се повикува mysql.exe фајлот, што е MySQL Monitor.

23.4. INCREMENTAL BACKUP

До сега опишаните техники кои може да се користат за бекап, но и за миграција подразбираат секогаш бекап на комплетната база на податоци. Да замислиме сега ситуација дека вршиме бекап во правилни временски интервали, на пример еднаш неделно. Освен тоа што секојпат би вршеле бекап на комплетната база – што е непотребно – би постоела и реална опасност за податоците. Податоците меѓу два бекапи се под константен ризик; по моментот откако ќе направиме бекап,

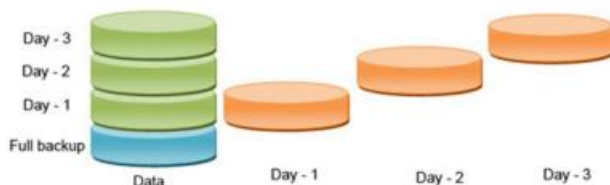
веќе следниот внесен податок е под ризик и ќе биде обезбеден дури по следниот бекап. За сериозни бази на податоци тоа е недопустливо.

Бекапот, кој секојпат се врши над комплетната база и нејзината содржина, се нарекува full backup.



Едно од решенијата се чести бекапи на комплетната база, што само ќе го намали коефициентот на незаштитеност, но нема да го отстрани проблемот во потполност. Освен тоа, доколку базата е голема, ваквото нешто одзема доста време и простор.

Друго решение е инкрементиран бекап (incremental backup). Тој исто така подразбира чест бекап, но не на целата база, туку само на промените направени од последниот бекап, што е значајно поефикасна варијанта.



Ова решение е остварливо со помош на бинарен лог. Знаеме дека, доколку бинарниот лог е активен, секоја sql наредба извршена на серверот е логирана. Тоа значи дека треба да бидат снимени сите наредби извршени од последниот бекап, што е многу помалку од комплетната база.

Во моментот на активација, бинарниот лог креира фајл (во data фолдерот на MySQL серверот) кој како екстензија има реден број. На пример:mysql-bin.000001. Понекогаш (на пример, при секој рестарт), MySQL серверот ќе го затвори логот и ќе направи нов фајл, со реден број зголемен за еден: mysql-bin.000002. Сите овие

фајлови, освен последниот се неактивни и може да ги преземеме и користиме како додаток на бекапот. Но, и понатаму останува проблемот за последниот фајл, кој во себе ги содржи сите наредби извршени од последниот “исечок” на логот.

Затоа, логот мора да го “исечеме” рачно во моментот кога сакаме да извршиме преземање на фајловите на логот.

Ова е многу едноставна процедура. Треба само да ги стартираме SQL наредбите FLUSH LOGS или конзолната апликација (`mysqladmin -u root flush-logs`). По активацијата на овие наредби, ќе се појави уште еден лог фајл во data фолдерот (при секоја активација се додава нов фајл). Сега само треба да ги преместиме сите фајлови, освен последниот, на сигурна локација.

Всушност, бекапот се овие лог фајлови. Тие ги содржат сите INSERT, UPDATE, DELETE наредби.

Како најдобар бекап систем треба да се користи комбинација од FULL и INCREMENTAL. На пример, најдобро е full бекап да се врши еднаш неделно, а incremental еднаш дневно, или full еднаш дневно, а incremental на секој час – секако, се зависи од потребите.

При разгледување на бекап стратегиите, треба да се земе во предвид и репликацијата, која е можеби и најдобар начин за бекап на високофреквентни продукциски бази.

Restore на логот може да се изврши со помош на конзолната апликација `mysqlbinlog`. Оваа наредба го претвора бинарниот лог во SQL наредби и ги проследува на излезот.

ЗАКЛУЧОК

Овој проект е креиран со цел да ги демонстрира стекнатите знаења и вештини од предметот Бази на податоци. Содржината е базирана на реални сценарија и е дизајнирана така што може да послужи како основа за развој на поголеми и покомплексни системи. Секој дел од

проектот е внимателно изработен со цел да се обезбеди логичка целина, а истовремено да се прикаже применливоста на наученото.

КОРИСТЕНА ЛИТЕРАТУРА

1. Elmasri, R., & Navathe, S. B. (2017). Fundamentals of Database Systems. Pearson.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts. McGraw-Hill.
3. Date, C. J. (2003). An Introduction to Database Systems. Addison-Wesley.
4. MySQL Documentation: <https://dev.mysql.com/doc/>
5. Coronel, C., & Morris, S. (2015). Database Systems: Design, Implementation, & Management. Cengage Learning.